BÁO CÁO BÀI TẬP THỰC HÀNH
XỬ LÝ ẢNH SỐ

Phân tích và phân đoạn ảnh màu · Xử lý ảnh hình thái học
Phát hiện biên và phân ngưỡng · Phân đoạn ảnh dựa trên vùng

Họ và tên	
Mã sinh viên	
Lớp	.....
Notebook / mã nguồn	(điền đường dẫn Colab/Drive của bài làm)
Công cụ	Python 3, NumPy, OpenCV, scikit-image, scikit-learn, Matplotlib (Google Colab / Jupyter)

Mục lục

Bài 1. Phân tích và phân đoạn ảnh màu
Bài 2. Xử lý ảnh hình thái học
Bài 3. Phát hiện biên và phân ngưỡng
Bài 4. Phân đoạn ảnh dựa trên vùng

Ghi chú về dữ liệu: ảnh đầu vào là các ảnh mẫu chuẩn của thư viện scikit-image (coffee, astronaut, coins, camera, page) được lưu trong thư mục `images/`; có thể thay bằng ảnh của bộ dữ liệu LMS cùng tên file mà không cần sửa mã. Mỗi nội dung trình bày theo thứ tự: mô tả và tham số -> mã nguồn -> ảnh đầu vào / kết quả -> nhận xét.

Bài 1. Phân tích và phân đoạn ảnh màu

Nội dung	Tham số khảo sát
Chuyển không gian màu	RGB -> HSV (OpenCV), HSI (tự cài đặt), CIELAB (skimage); hiển thị R,G,B / H,S,V / H,S,I / L*,a*,b*
Phân đoạn 2 nhóm/ảnh	coffee: sứ đỏ & gỗ; astronaut: bộ đồ cam & nền trắng-xám. Hộp ngưỡng RGB, HSV, HSI (chi tiết trong mã)
Độ bền chiếu sáng	ảnh tối ×0,6 và gamma 0,7, giữ nguyên ngưỡng, đo IoU
Khảo sát 3 bộ ngưỡng HSV	chặt / vừa / lỏng: coffee (H≤10,S≥0,85,V≥0,35 | H≤16,S≥0,6,V≥0,15 | H≤26,S≥0,4,V≥0,08); astronaut (H 10-20,S≥0,7,V≥0,65 | H 5-25,S≥0,55,V≥0,4 | H 0-40,S≥0,3,V≥0,2)

Nội dung: (1) chuyển RGB → HSV / HSI / CIELAB và hiển thị từng kênh; (2) phân đoạn 2 nhóm đối tượng theo màu và so sánh kết quả khi đặt ngưỡng trong RGB với HSV / HSI; (3) khảo sát 3 bộ tham số ngưỡng.

Ảnh đầu vào (2 ảnh màu, thu nhỏ cạnh dài còn 400 px):

- `coffee.png` - tách sứ đỏ (cốc, đĩa) và mặt bàn gỗ (hai màu cùng họ đỏ-cam nhưng khác sắc độ).
- `astronaut.png` - tách bộ đồ phi hành gia màu cam và nền trắng/xám (khác về độ bão hòa).

Mã nguồn - cài đặt và hàm tiện ích

```
# ===== Cài đặt & hàm tiện ích dùng chung =====
import os, time, warnings
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import data, color, filters, morphology, measure, segmentation, util
warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 80, "font.size": 9, "axes.titlesize": 9})

# Thư mục ảnh đầu vào / kết quả.
# Muốn dùng ảnh trong bộ dữ liệu LMS: chép ảnh vào images/ và đổi tên file ở ô "Ảnh đầu vào".
IMG_DIR, OUT_DIR = "images", "outputs/bai1"
for d in (IMG_DIR, OUT_DIR):
    os.makedirs(d, exist_ok=True)


def load_rgb(name, fallback, max_side=None):
    """Đọc ảnh màu RGB từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, cv2.cvtColor(fallback(), cv2.COLOR_RGB2BGR))
    img = cv2.cvtColor(cv2.imread(p, cv2.IMREAD_COLOR), cv2.COLOR_BGR2RGB)
    if max_side and max(img.shape[:2]) > max_side:
        s = max_side / max(img.shape[:2])
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def load_gray(name, fallback, max_side=None):
    """Đọc ảnh xám uint8 từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, fallback())
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if max_side and max(img.shape) > max_side:
        s = max_side / max(img.shape)
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def show(images, titles=None, ncols=4, name=None, cell=3.0, cmap="gray"):
    """Hiển thị lưới ảnh; ảnh 2-D hiển thị theo thang xám cố định; lưu vào OUT_DIR/name."""
    n = len(images)
    ncols = min(ncols, n)
    nrows = int(np.ceil(n / ncols))
    fig, axs = plt.subplots(nrows, ncols, figsize=(cell * ncols, (cell * images[0].shape[0] / images[0].shape[1] + 0.35) *
        nrows), squeeze=False)
    for i, ax in enumerate(axs.ravel()):
        ax.axis("off")
        if i >= n:
            continue
        im = images[i]
        if im.ndim == 2:
            if im.dtype == np.uint8:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=255)
            elif im.dtype == bool:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=1)
            else:
                ax.imshow(im, cmap=cmap)
        else:
            ax.imshow(im)
        if titles is not None:
            ax.set_title(titles[i])
    fig.tight_layout()
    if name:
```

```
        fig.savefig(os.path.join(OUT_DIR, name), dpi=110, bbox_inches="tight")
    plt.show()
```

1.0 Ảnh đầu vào

Mã nguồn

```
coffee = load_rgb("coffee.png", data.coffee, max_side=400)
astro = load_rgb("astronaut.png", data.astronaut, max_side=400)
print("coffee:", coffee.shape, "| astronaut:", astro.shape)
show([coffee, astro], ["coffee.png", "astronaut.png"], ncols=2, cell=5, name="b1_input.png")
```

Kết quả (văn bản)

```
coffee: (267, 400, 3) | astronaut: (400, 400, 3)
```

Kết quả (hình)

coffee.png



astronaut.png



1.1 Chuyển đổi không gian màu và hiển thị các kênh

- **HSV:** `cv2.cvtColor(..., COLOR_RGB2HSV)` trên ảnh float32 (H ∈ [0,360), S, V ∈ [0,1]).
- **HSI (tự cài đặt):** I=(R+G+B)/(3), S=1-(3min(R,G,B))/(R+G+B), H=θ nếu B≤ G, ngược lại 360°-θ, với θ=arccos(((1)/(2)[(R-G)+(R-B)])/(√((R-G)^2+(R-B)(G-B)))).
- **CIELAB:** `skimage.color.rgb2lab` (nguồn sáng D65): L ∈ [0,100], a*, b* ∈ ≈[-128,127].

Kênh Hue được hiển thị bằng bảng màu cầu vồng (S = V = 1) để dễ quan sát; các kênh còn lại là ảnh xám.

Mã nguồn

```
def rgb2hsv_f(rgb):
    """RGB uint8 -> H (độ, 0-360), S, V (0-1) dùng OpenCV."""
    hsv = cv2.cvtColor(rgb.astype(np.float32) / 255.0, cv2.COLOR_RGB2HSV)
    return hsv[..., 0], hsv[..., 1], hsv[..., 2]


def rgb2hsi(rgb):
    """RGB uint8 -> H (độ, 0-360), S, I (0-1) theo công thức giáo trình."""
    f = rgb.astype(np.float64) / 255.0
    R, G, B = f[..., 0], f[..., 1], f[..., 2]
    eps = 1e-10
    I = (R + G + B) / 3.0
    S = 1.0 - 3.0 * np.minimum(np.minimum(R, G), B) / (R + G + B + eps)
    num = 0.5 * ((R - G) + (R - B))
    den = np.sqrt((R - G) ** 2 + (R - B) * (G - B)) + eps
    theta = np.degrees(np.arccos(np.clip(num / den, -1.0, 1.0)))
    H = np.where(B <= G, theta, 360.0 - theta)
    H = np.where(S < 1e-6, 0.0, H)                      # xám: Hue không xác định
    return H, S, I


def hue_rgb(H):
    """Ảnh màu thể hiện Hue (S = V = 1)."""
    return (color.hsv2rgb(np.dstack([H / 360.0, np.ones_like(H), np.ones_like(H)])) * 255).astype(np.uint8)


def to_u8(x, lo=0.0, hi=1.0):
    return np.clip((x - lo) / (hi - lo) * 255, 0, 255).astype(np.uint8)


def channels_figure(rgb, name, fname):
    Hv, Sv, Vv = rgb2hsv_f(rgb)
    Hi, Si, Ii = rgb2hsi(rgb)
    L, a, b = [c for c in np.moveaxis(color.rgb2lab(rgb), -1, 0)]
    imgs = [rgb, rgb[..., 0], rgb[..., 1], rgb[..., 2],
            rgb, hue_rgb(Hv), to_u8(Sv), to_u8(Vv),
            rgb, hue_rgb(Hi), to_u8(Si), to_u8(Ii),
```

```
            rgb, to_u8(L, 0, 100), to_u8(a, -60, 80), to_u8(b, -60, 80)]
    tits = [f"{name}", "R", "G", "B",
            f"{name}", "H (HSV)", "S (HSV)", "V",
            f"{name}", "H (HSI)", "S (HSI)", "I",
            f"{name}", "L*", "a* (xanh lá ← → đỏ)", "b* (xanh dương ← → vàng)"]
    show(imgs, tits, ncols=4, cell=3.0, name=fname)
    return (Hv, Sv, Vv), (Hi, Si, Ii), (L, a, b)

hsv_c, hsi_c, lab_c = channels_figure(coffee, "coffee", "b1_channels_coffee.png")
```

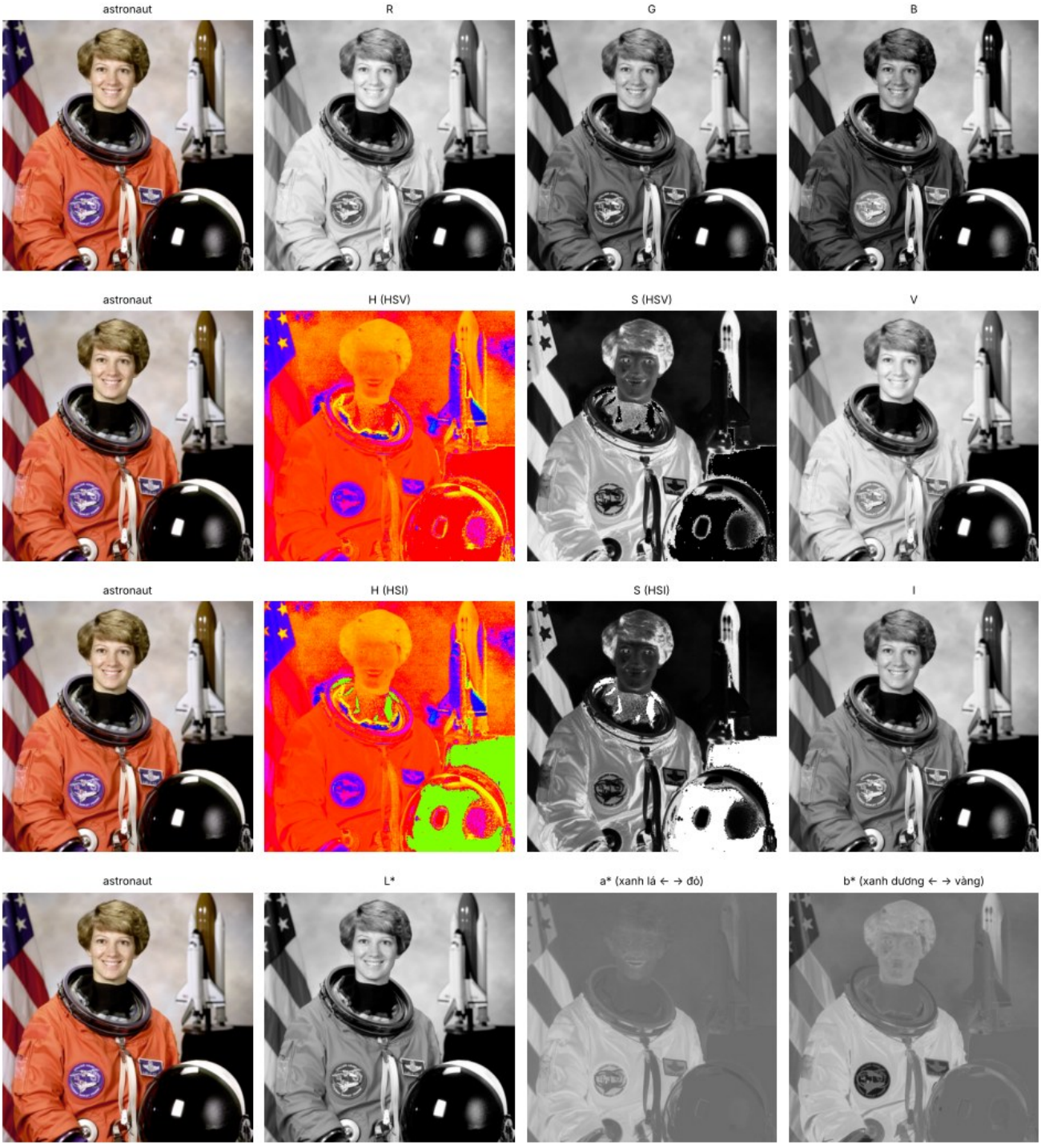
Kết quả (hình)



Mã nguồn

```
hsv_a, hsi_a, lab_a = channels_figure(astro, "astronaut", "b1_channels_astronaut.png")
```

Kết quả (hình)



Mã nguồn

```
# kiểm chứng cài đặt HSI tự viết: các cạnh/giá trị đặc biệt và so với HSV
t = np.array([[[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 255], [128, 128, 128], [255, 128, 0]]], np.uint8)
H, S, I = rgb2hsi(t)
Hh, Sh, Vh = rgb2hsv_f(t)
print("điểm màu:        đỏ      lục    lam    trắng    xám    cam")
print("H (HSI)   :", np.round(H[0], 1))
print("H (HSV)   :", np.round(Hh[0], 1))
print("S (HSI)   :", np.round(S[0], 2))
print("S (HSV)   :", np.round(Sh[0], 2))
print("I / V     :", np.round(I[0], 2), "/", np.round(Vh[0], 2))
print("Khác biệt Hue HSV - HSI trung bình (độ) trên coffee: %.2f; trên astronaut: %.2f" %
      (np.abs(hsv_c[0] - hsi_c[0]).mean(), np.abs(hsv_a[0] - hsi_a[0]).mean()))
```

Kết quả (văn bản)

```
điểm màu:        đỏ      lục    lam    trắng    xám    cam
H (HSI)   : [  0.  120.  240.    0.    0.   30.1]
H (HSV)   : [  0.  120.  240.    0.    0.   30.1]
S (HSI)   : [1. 1. 1. 0. 0. 1.]
S (HSV)   : [1. 1. 1. 0. 0. 1.]
I / V     : [0.33 0.33 0.33 1.   0.5  0.5 ] / [1.  1.  1.  1.  0.5 1. ]
Khác biệt Hue HSV - HSI trung bình (độ) trên coffee: 0.69; trên astronaut: 10.07
```

Nhận xét 1.1

- **RGB** trộn lẫn màu và độ sáng: ở `coffee`, kênh R đều cao ở cả đĩa đỏ sáng lẫn mặt bàn gỗ, nên từng kênh R/G/B riêng lẻ khó dùng để phân biệt hai vật.
- **HSV/HSI** tách màu (Hue) khỏi sáng tối (V/I): ở `coffee`, Hue của sứ đỏ ≈ 10° còn gỗ ≈ 22–30°, nhìn kênh H đã phân biệt rõ; kênh S thấp ở vùng trắng/xám (nền, vành cốc, tàu con thoi) và cao ở bộ đồ cam. Kênh H **nhiễu mạnh ở vùng gần đen hoặc gần xám** (mũ bảo hiểm đen, nền xám) vì Hue không xác định khi S ≈ 0 hoặc V ≈ 0 -> khi phân đoạn phải kèm điều kiện trên S và V.
- **HSI vs HSV:** hàm tự cài đặt cho đúng các giá trị đặc biệt (đỏ/lục/lam/trắng/xám/cam đều khớp công thức; Hue cam = 30.1° ở cả hai). Hue của HSV và HSI gần như trùng trên `coffee` (lệch TB 0,69°) nhưng lệch TB 10,07° trên `astronaut` chủ yếu ở vùng tối/xám (mũ bảo hiểm hiện màu xanh lá ở kênh H của HSI). S_HSI nhỏ hơn S_HSV (màu cam (214,99,63): 0,50 so với 0,71) và I (trung bình) tối hơn V (max).
- **CIELAB:** L* là độ sáng cảm nhận; a* cao (đỏ) ở bộ đồ cam/sứ đỏ, b* cao (vàng) ở gỗ và tóc. Biên độ a*, b* nhỏ nên phải kéo giãn tương phản khi hiển thị. Khoảng cách Euclid trong Lab gần với cảm nhận màu nên được dùng ở Bài 4 (region growing, K-means).

1.2 Phân đoạn 2 nhóm đối tượng theo màu: RGB so với HSV / HSI

Mỗi nhóm được mô tả bằng một "hộp" ngưỡng trong từng không gian màu:

- **RGB:** `lo ≤ R,G,B ≤ hi` cho từng kênh (+ điều kiện chênh lệch kênh nếu cần).
- **HSV:** khoảng Hue `[h0, h1]` (xử lý vòng 360°), `S ≥ s_min`, `V ≥ v_min` (hoặc `S ≤ s_max` cho màu xám/trắng).
- **HSI:** tương tự HSV nhưng với `I` thay `V` và `S` theo công thức HSI. Lưu ý S_HSI thường nhỏ hơn S_HSV và I≤ V nên ngưỡng `s_min`, `v_min` phải hạ thấp tương ứng.

Mặt nạ được làm sạch bằng opening (ellipse 3×3) để loại điểm nhiễu rời rạc.

Mã nguồn

```
def hue_in(H, h0, h1):
    return (H >= h0) & (H <= h1) if h0 <= h1 else (H >= h0) | (H <= h1)

def clean(m, k=3):
    return cv2.morphologyEx(m.astype(np.uint8), cv2.MORPH_OPEN, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))) > 0

def m_rgb(rgb, rlo=0, rhi=255, glo=0, ghi=255, blo=0, bhi=255, rg_diff=None, spread=None):
    R, G, B = [rgb[..., i].astype(int) for i in range(3)]
    m = (R >= rlo) & (R <= rhi) & (G >= glo) & (G <= ghi) & (B >= blo) & (B <= bhi)
    if rg_diff is not None:
        m &= (R - G) >= rg_diff
    if spread is not None:                                  # max - min kênh <= spread (gần xám)
        m &= (rgb.max(-1).astype(int) - rgb.min(-1).astype(int)) <= spread
    return clean(m)

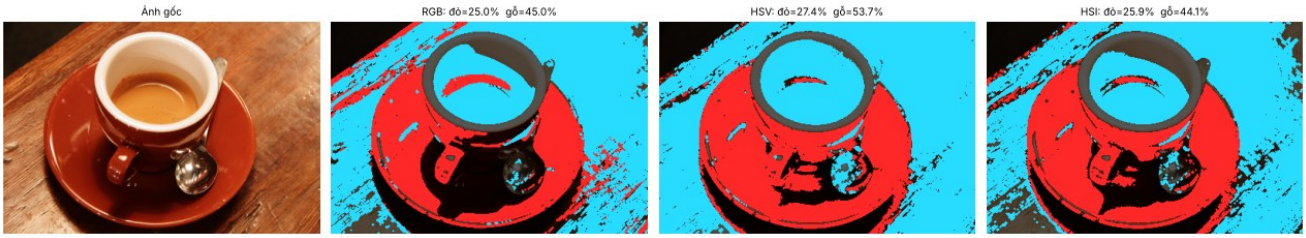
def m_hs(H, S, V, h0=0, h1=360, smin=0.0, smax=1.0, vmin=0.0, vmax=1.0):
    return clean(hue_in(H, h0, h1) & (S >= smin) & (S <= smax) & (V >= vmin) & (V <= vmax))

def paint(rgb, masks, colors):
    o = (rgb * 0.35).astype(np.uint8)
    for m, c in zip(masks, colors):
        o[m] = c
    return o

def iou(a, b):
    return (a & b).sum() / max((a | b).sum(), 1)

# --- ảnh coffee: nhóm 1 = sứ đỏ, nhóm 2 = mặt bàn gỗ ---
Hv, Sv, Vv = hsv_c; Hi, Si, Ii = hsi_c
coffee_masks = {
    "RGB": (m_rgb(coffee, rlo=90, glo=0, ghi=75, bhi=70, rg_diff=60),                 # đỏ đậm: R cao, G/B thấp
            m_rgb(coffee, rlo=140, glo=70, ghi=200, bhi=130, rg_diff=30)),           # nâu cam sáng
    "HSV": (m_hs(Hv, Sv, Vv, 0, 16, smin=0.6, vmin=0.15),
            m_hs(Hv, Sv, Vv, 17, 40, smin=0.3, vmin=0.3)),
    "HSI": (m_hs(Hi, Si, Ii, 0, 16, smin=0.6, vmin=0.07),
            m_hs(Hi, Si, Ii, 17, 40, smin=0.3, vmin=0.25)),
}
imgs, tits = [coffee], ["Ảnh gốc"]
for k, (m1, m2) in coffee_masks.items():
    imgs.append(paint(coffee, [m1, m2], [(255, 40, 40), (40, 220, 255)]))
    tits.append(f"{k}: đỏ={100 * m1.mean():.1f}%  gỗ={100 * m2.mean():.1f}%")
show(imgs, tits, ncols=4, cell=4.0, name="b1_seg_coffee.png")
print("Chú thích màu: ĐỎ = sứ đỏ, XANH LƠ = mặt bàn gỗ (nhóm còn lại tối đi).")
print("IoU giữa các không gian màu (nhóm sứ đỏ):", {f"{a}-{b}": round(iou(coffee_masks[a][0], coffee_masks[b][0]), 3) for a, b
    in [("RGB", "HSV"), ("HSV", "HSI"), ("RGB", "HSI")]})
print("IoU giữa các không gian màu (nhóm gỗ)   :", {f"{a}-{b}": round(iou(coffee_masks[a][1], coffee_masks[b][1]), 3) for a, b
    in [("RGB", "HSV"), ("HSV", "HSI"), ("RGB", "HSI")]})
```

Kết quả (hình)



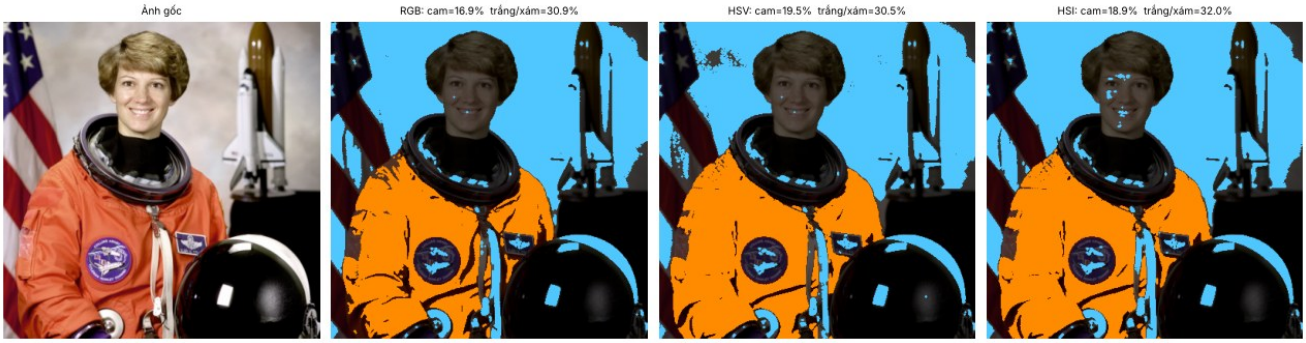
Kết quả (văn bản)

```
Chú thích màu: ĐỎ = sứ đỏ, XANH LƠ = mặt bàn gỗ (nhóm còn lại tối đi).
IoU giữa các không gian màu (nhóm sứ đỏ): {'RGB-HSV': np.float64(0.591), 'HSV-HSI': np.float64(0.885), 'RGB-HSI': np.float64(0.648)}
IoU giữa các không gian màu (nhóm gỗ)   : {'RGB-HSV': np.float64(0.826), 'HSV-HSI': np.float64(0.821), 'RGB-HSI': np.float64(0.869)}
```

Mã nguồn

```
# --- ảnh astronaut: nhóm 1 = bộ đồ cam, nhóm 2 = nền trắng/xám ---
Hv, Sv, Vv = hsv_a; Hi, Si, Ii = hsi_a
astro_masks = {
    "RGB": (m_rgb(astro, rlo=170, glo=50, ghi=140, bhi=110, rg_diff=70),
            m_rgb(astro, rlo=150, glo=150, blo=150, spread=30)),
    "HSV": (m_hs(Hv, Sv, Vv, 5, 25, smin=0.55, vmin=0.4),
            m_hs(Hv, Sv, Vv, smax=0.12, vmin=0.6)),
    "HSI": (m_hs(Hi, Si, Ii, 5, 25, smin=0.35, vmin=0.25),
            m_hs(Hi, Si, Ii, smax=0.1, vmin=0.6)),
}
imgs, tits = [astro], ["Ảnh gốc"]
for k, (m1, m2) in astro_masks.items():
    imgs.append(paint(astro, [m1, m2], [(255, 140, 0), (80, 200, 255)]))
    tits.append(f"{k}: cam={100 * m1.mean():.1f}%  trắng/xám={100 * m2.mean():.1f}%")
show(imgs, tits, ncols=4, cell=4.0, name="b1_seg_astronaut.png")
print("Chú thích màu: CAM = bộ đồ, XANH NHẠT = nền trắng/xám (nhóm còn lại tối đi).")
print("IoU (nhóm bộ đồ cam):", {f"{a}-{b}": round(iou(astro_masks[a][0], astro_masks[b][0]), 3) for a, b in [("RGB", "HSV"),
    ("HSV", "HSI"), ("RGB", "HSI")]})
print("IoU (nhóm nền trắng):", {f"{a}-{b}": round(iou(astro_masks[a][1], astro_masks[b][1]), 3) for a, b in [("RGB", "HSV"),
    ("HSV", "HSI"), ("RGB", "HSI")]})
```

Kết quả (hình)



Kết quả (văn bản)

```
Chú thích màu: CAM = bộ đồ, XANH NHẠT = nền trắng/xám (nhóm còn lại tối đi).
IoU (nhóm bộ đồ cam): {'RGB-HSV': np.float64(0.854), 'HSV-HSI': np.float64(0.967), 'RGB-HSI': np.float64(0.878)}
IoU (nhóm nền trắng): {'RGB-HSV': np.float64(0.932), 'HSV-HSI': np.float64(0.917), 'RGB-HSI': np.float64(0.961)}
```

So sánh độ bền vững với thay đổi chiếu sáng

Giả lập ảnh tối hơn (nhân cường độ RGB × 0.6) và sáng hơn/lệch (gamma 0.7) rồi áp **đúng các ngưỡng cũ**. Với mỗi không gian màu, đo IoU giữa mặt nạ trên ảnh đã đổi sáng và mặt nạ trên ảnh gốc (1.0 = không đổi). Ngưỡng nằm trên kênh sắc độ (H, S) kỳ vọng ít bị ảnh hưởng hơn so với ngưỡng thẳng trên R, G, B.

Mã nguồn

```
def seg_all(rgb, which):
    """Chạy lại ba bộ luật ngưỡng ở trên cho ảnh rgb (which: 'coffee' | 'astro')."""
    hv, hi = rgb2hsv_f(rgb), rgb2hsi(rgb)
    if which == "coffee":
        return {"RGB": (m_rgb(rgb, rlo=90, glo=0, ghi=75, bhi=70, rg_diff=60), m_rgb(rgb, rlo=140, glo=70, ghi=200, bhi=130,
            rg_diff=30)),
                "HSV": (m_hs(*hv, 0, 16, smin=0.6, vmin=0.15), m_hs(*hv, 17, 40, smin=0.3, vmin=0.3)),
                "HSI": (m_hs(*hi, 0, 16, smin=0.6, vmin=0.07), m_hs(*hi, 17, 40, smin=0.3, vmin=0.25))}
    return {"RGB": (m_rgb(rgb, rlo=170, glo=50, ghi=140, bhi=110, rg_diff=70), m_rgb(rgb, rlo=150, glo=150, blo=150,
        spread=30)),
            "HSV": (m_hs(*hv, 5, 25, smin=0.55, vmin=0.4), m_hs(*hv, smax=0.12, vmin=0.6)),
            "HSI": (m_hs(*hi, 5, 25, smin=0.35, vmin=0.25), m_hs(*hi, smax=0.1, vmin=0.6))}

def darker(rgb):
    return (rgb.astype(np.float32) * 0.6).astype(np.uint8)

def gamma(rgb, g=0.7):
    return (255 * (rgb / 255.0) ** g).astype(np.uint8)

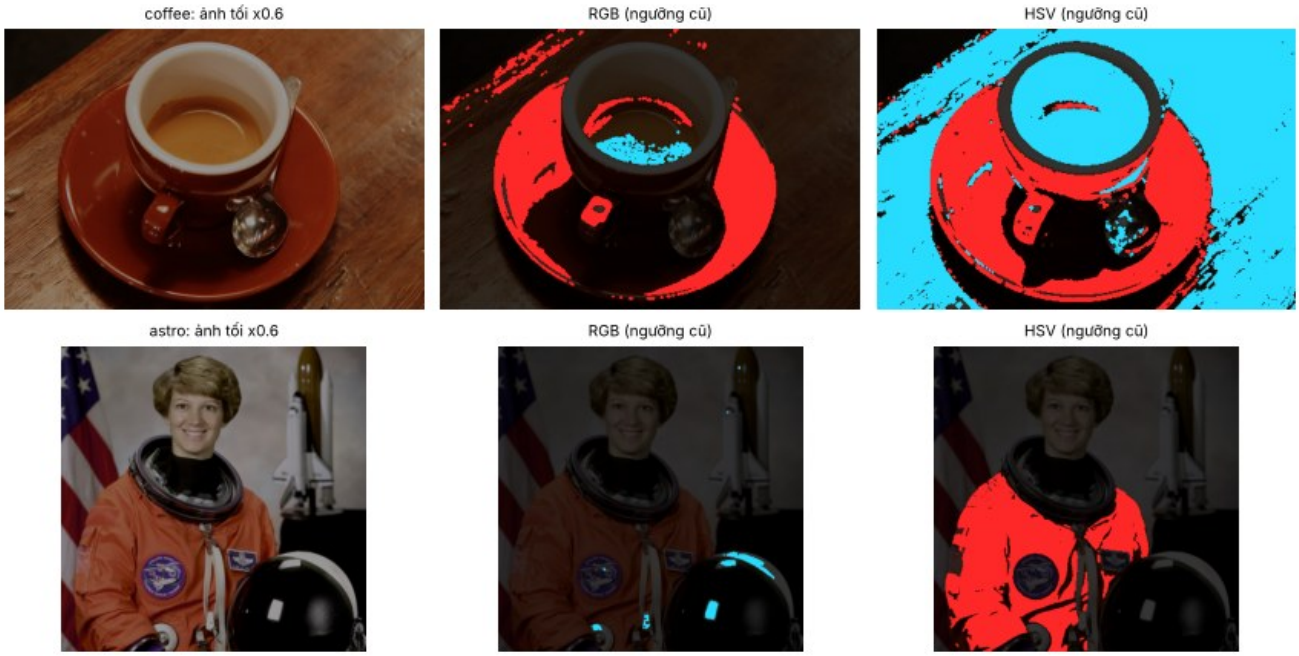
print(f"{'ảnh / nhóm':26s} {'biến đổi':12s} {'RGB':>7s} {'HSV':>7s} {'HSI':>7s}   (IoU so với kết quả trên ảnh gốc)")
```

```
rob_imgs, rob_tits = [], []
for which, img, base, gnames in [("coffee", coffee, coffee_masks, ("sứ đỏ", "gỗ")), ("astro", astro, astro_masks, ("bộ đồ cam",
    "nền trắng"))]:
    for tname, tf in [("tối x0.6", darker), ("gamma 0.7", gamma)]:
        mod = tf(img)
        res = seg_all(mod, which)
        for gi in range(2):
            print(f"{which + ' / ' + gnames[gi]:26s} {tname:12s} " + " ".join(f"{iou(res[k][gi], base[k][gi]):7.3f}" for k in
                ("RGB", "HSV", "HSI")))
        if tname == "tối x0.6":
            rob_imgs += [mod] + [paint(mod, list(res[k]), [(255, 40, 40), (40, 220, 255)]) for k in ("RGB", "HSV")]
            rob_tits += [f"{which}: ảnh tối x0.6", "RGB (ngưỡng cũ)", "HSV (ngưỡng cũ)"]
show(rob_imgs, rob_tits, ncols=3, cell=3.6, name="b1_robust.png")
```

Kết quả (văn bản)

```
ảnh / nhóm                 biến đổi         RGB     HSV     HSI   (IoU so với kết quả trên ảnh gốc)
coffee / sứ đỏ             tối x0.6       0.512   0.822   0.825
coffee / gỗ                tối x0.6       0.028   0.971   0.607
coffee / sứ đỏ             gamma 0.7      0.452   0.728   0.639
coffee / gỗ                gamma 0.7      0.636   0.862   0.689
astro / bộ đồ cam          tối x0.6       0.000   0.889   0.736
astro / nền trắng          tối x0.6       0.022   0.000   0.000
astro / bộ đồ cam          gamma 0.7      0.565   0.594   0.523
astro / nền trắng          gamma 0.7      0.876   0.860   0.870
```

Kết quả (hình)



Nhận xét 1.2 (nhóm: `coffee` = sứ đỏ / gỗ; `astronaut` = bộ đồ cam / nền trắng-xám)

- **Hình dạng kết quả:** HSV cho vùng sứ đỏ 27,4% ảnh (RGB 25,0%; HSI 25,9%) và phủ cả phần đĩa nằm trong bóng vì Hue không đổi khi tối đi; hộp RGB bỏ sót phần này (R thấp) nên IoU RGB–HSV của nhóm sứ đỏ chỉ 0,59 (HSV–HSI: 0,89). Với bộ đồ cam IoU RGB–HSV = 0,85, HSV–HSI = 0,97 (sau khi hạ ngưỡng S, I của HSI, xem nhận xét trên); với nền trắng cả ba cách đều 0,92–0,96.
- **Độ bền khi đổi chiếu sáng** (giữ nguyên ngưỡng; IoU so với kết quả ảnh gốc): khi tối đi ×0,6, HSV giữ được 0,82 (sứ đỏ), 0,97 (gỗ), 0,89 (bộ đồ cam) trong khi RGB chỉ còn 0,51; 0,03; 0,00. HSV cao hơn RGB ở 6/8 phép thử (chênh trên 0,2 ở 5 phép thử). Ngoại lệ: nhóm **nền trắng** được định nghĩa bằng độ sáng cao (V ≥ 0,6 / R,G,B ≥ 150 / I ≥ 0,6) nên cả ba cách đều hỏng khi ảnh tối (IoU ≈ 0), và khi đổi gamma RGB (0,88) nhỉnh hơn HSV (0,86) chút ít. Tức là chỉ ngưỡng đặt trên H, S mới thật sự bền với chiếu sáng; ngưỡng trên V/I/RGB thì không.
- **HSV vs HSI:** hiệu quả tương đương, nhưng phải hạ ngưỡng S và I cho HSI. Vì I là trung bình (không phải max) nên HSI nhạy với giảm sáng hơn HSV (bộ đồ cam khi tối ×0,6: 0,74 so với 0,89).
- **Hạn chế chung:** ngưỡng màu thuần túy không dùng thông tin không gian, nên nhóm gỗ (Hue 17–40°) cũng gom cả cà phê lỏng và lòng cốc (Hue ≈ 28°); Hue là đại lượng vòng (đỏ 0°/360° phải xử lý bao quanh) và không ổn định ở S, V thấp.

1.3 Khảo sát 3 bộ tham số ngưỡng (HSV)

Với mỗi ảnh, nhóm chính được phân đoạn bằng ba bộ ngưỡng HSV: **chặt** (strict), **vừa** (medium), **lỏng** (loose). Bộ "vừa" chính là bộ dùng ở mục 1.2.

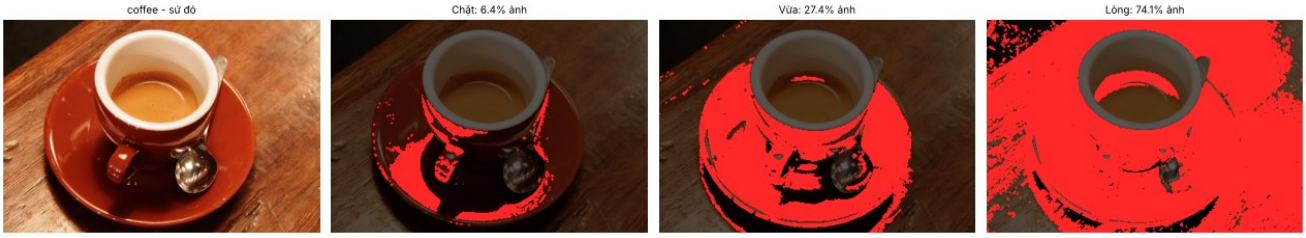
Mã nguồn

```
param_sets_coffee = {   # nhóm sứ đỏ
    "Chặt:  H∈[0,10], S≥0.85, V≥0.35": dict(h0=0, h1=10, smin=0.85, vmin=0.35),
    "Vừa:   H∈[0,16], S≥0.60, V≥0.15": dict(h0=0, h1=16, smin=0.60, vmin=0.15),
    "Lỏng:  H∈[0,26], S≥0.40, V≥0.08": dict(h0=0, h1=26, smin=0.40, vmin=0.08),
}
param_sets_astro = {    # nhóm bộ đồ cam
    "Chặt:  H∈[10,20], S≥0.70, V≥0.65": dict(h0=10, h1=20, smin=0.70, vmin=0.65),
    "Vừa:   H∈[5,25], S≥0.55, V≥0.40":  dict(h0=5, h1=25, smin=0.55, vmin=0.40),
    "Lỏng:  H∈[0,40], S≥0.30, V≥0.20":  dict(h0=0, h1=40, smin=0.30, vmin=0.20),
}
for nm, img, hsv, ps, col in [("coffee - sứ đỏ", coffee, hsv_c, param_sets_coffee, (255, 40, 40)),
                              ("astronaut - bộ đồ cam", astro, hsv_a, param_sets_astro, (255, 140, 0))]:
    imgs, tits = [img], [nm]
    print(f"--- {nm} ---")
    ref = m_hs(*hsv, **list(ps.values())[1])
    for pn, kw in ps.items():
        m = m_hs(*hsv, **kw)
        imgs.append(paint(img, [m], [col]))
        tits.append(f"{pn.split(':')[0]}: {100 * m.mean():.1f}% ảnh")
        print(f"{pn:36s} diện tích = {100 * m.mean():5.1f}% | IoU so với bộ 'vừa' = {iou(m, ref):.3f}")
    show(imgs, tits, ncols=4, cell=4.0, name="b1_params_" + nm.split()[0] + ".png")
```

Kết quả (văn bản)

```
--- coffee - sứ đỏ ---
Chặt:  H∈[0,10], S≥0.85, V≥0.35      diện tích =   6.4% | IoU so với bộ 'vừa' = 0.235
Vừa:   H∈[0,16], S≥0.60, V≥0.15      diện tích =  27.4% | IoU so với bộ 'vừa' = 1.000
Lỏng:  H∈[0,26], S≥0.40, V≥0.08      diện tích =  74.1% | IoU so với bộ 'vừa' = 0.369
```

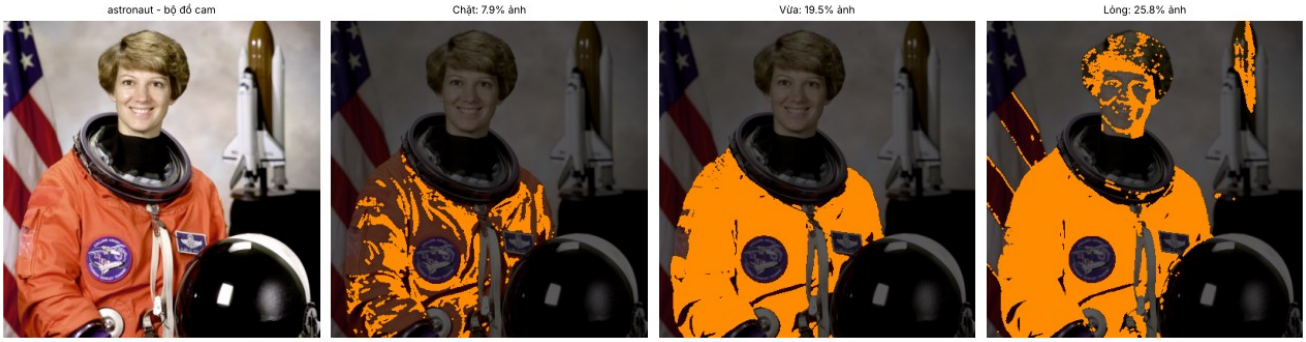
Kết quả (hình)



Kết quả (văn bản)

```
--- astronaut - bộ đồ cam ---
Chặt:  H∈[10,20], S≥0.70, V≥0.65     diện tích =   7.9% | IoU so với bộ 'vừa' = 0.406
Vừa:   H∈[5,25], S≥0.55, V≥0.40      diện tích =  19.5% | IoU so với bộ 'vừa' = 1.000
Lỏng:  H∈[0,40], S≥0.30, V≥0.20      diện tích =  25.8% | IoU so với bộ 'vừa' = 0.758
```

Kết quả (hình)



Nhận xét 1.3

- `coffee` – sứ đỏ: bộ **chặt** chỉ còn 6,4% ảnh (IoU với bộ vừa 0,24): H ≤ 10° và S ≥ 0,85 loại hầu hết mặt đĩa/thân cốc, chỉ giữ vài mảng đỏ đậm -> bỏ sót nhiều. Bộ **vừa** (27,4%) phủ gần đủ đĩa và thân cốc. Bộ **lỏng** (74,1%, H ≤ 26°, S ≥ 0,4) tràn sang mặt bàn gỗ (Hue 22–30°) -> mất khả năng phân biệt hai nhóm, nhiễm nặng.
- `astronaut` – bộ đồ cam: bộ chặt (7,9%, IoU 0,41) chỉ lấy những nếp vải sáng nhất; bộ vừa (19,5%) lấy gần trọn bộ đồ; bộ lỏng (25,8%, IoU 0,76) bắt đầu lấn sang **da mặt, tóc, sọc cờ và thùng nhiên liệu nâu** của tàu con thoi (Hue 20–40°, S 0,3–0,5).
- **Rút ra:** độ rộng khoảng Hue quyết định mức nhiễm sang màu lân cận; `S_min` loại các điểm nhạt/trắng; `V_min` loại vùng quá tối (ở đó Hue không tin cậy). Thu hẹp ngưỡng tăng độ chính xác nhưng giảm độ phủ (trade-off precision/recall). Bộ "vừa" là cân bằng tốt nhất ở cả hai ảnh.

Ảnh chụp màn hình minh chứng - Bài 1

Chụp trên nền cửa sổ thư mục chứa mã sinh viên: (1) cửa sổ thư mục hiển thị tên thư mục/mã SV và file `bai1.ipynb`; (2) notebook đang chạy với kết quả hiển thị; (3) thời gian/tên tài khoản trên màn hình nếu có.

[Chèn ảnh chụp màn hình tại đây]

Đặt file vào thư mục **screenshots/** với tên `bai1_1.png`, `bai1_2.png`... rồi chạy lại `python tools/make_report.py`.

Bài 2. Xử lý ảnh hình thái học

Nội dung	Tham số khảo sát
Erosion/dilation/opening/closing	tự lập trình NumPy; SE rect 3/15, ellipse 7, cross 5; so sánh OpenCV (bằng nhau từng pixel) + thời gian
SE hình dạng	rect, ellipse, cross, diamond, hline, vline (kích thước 7×7)
SE kích thước	ellipse 3, 7, 15, 25
Thuật toán	hình bao (SE rect3/cross3), làm đầy lỗ (SE cross3), thành phần liên thông (4/8), convex hull, thinning/skeletonize

Nội dung: (1) tự lập trình erosion / dilation / opening / closing bằng NumPy và đối chiếu với OpenCV; (2) thử nghiệm các phần tử cấu trúc (SE) khác hình dạng, kích thước; (3) cài đặt các thuật toán hình thái: trích xuất hình bao (boundary), làm đầy lỗ (hole filling), thành phần liên thông (connected components), convex hull và thinning.

Ảnh đầu vào: `coins.png` (303×384, xám - ảnh mẫu scikit-image, đồng xu trên nền tối) và `page.png` (ảnh trang chữ in, dùng cho thinning). Ảnh nhị phân được tạo bằng ngưỡng Otsu.

Mã nguồn - cài đặt và hàm tiện ích

```
# ===== Cài đặt & hàm tiện ích dùng chung =====
import os, time, warnings
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import data, color, filters, morphology, measure, segmentation, util
warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 80, "font.size": 9, "axes.titlesize": 9})

# Thư mục ảnh đầu vào / kết quả.
# Muốn dùng ảnh trong bộ dữ liệu LMS: chép ảnh vào images/ và đổi tên file ở ô "Ảnh đầu vào".
IMG_DIR, OUT_DIR = "images", "outputs/bai2"
for d in (IMG_DIR, OUT_DIR):
    os.makedirs(d, exist_ok=True)


def load_rgb(name, fallback, max_side=None):
    """Đọc ảnh màu RGB từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, cv2.cvtColor(fallback(), cv2.COLOR_RGB2BGR))
    img = cv2.cvtColor(cv2.imread(p, cv2.IMREAD_COLOR), cv2.COLOR_BGR2RGB)
    if max_side and max(img.shape[:2]) > max_side:
        s = max_side / max(img.shape[:2])
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def load_gray(name, fallback, max_side=None):
    """Đọc ảnh xám uint8 từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, fallback())
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if max_side and max(img.shape) > max_side:
        s = max_side / max(img.shape)
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def show(images, titles=None, ncols=4, name=None, cell=3.0, cmap="gray"):
    """Hiển thị lưới ảnh; ảnh 2-D hiển thị theo thang xám cố định; lưu vào OUT_DIR/name."""
    n = len(images)
    ncols = min(ncols, n)
    nrows = int(np.ceil(n / ncols))
    fig, axs = plt.subplots(nrows, ncols, figsize=(cell * ncols, (cell * images[0].shape[0] / images[0].shape[1] + 0.35) *
        nrows), squeeze=False)
    for i, ax in enumerate(axs.ravel()):
        ax.axis("off")
        if i >= n:
            continue
        im = images[i]
        if im.ndim == 2:
            if im.dtype == np.uint8:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=255)
            elif im.dtype == bool:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=1)
            else:
                ax.imshow(im, cmap=cmap)
        else:
            ax.imshow(im)
        if titles is not None:
            ax.set_title(titles[i])
    fig.tight_layout()
    if name:
        fig.savefig(os.path.join(OUT_DIR, name), dpi=110, bbox_inches="tight")
    plt.show()
```

2.0 Ảnh đầu vào và ảnh nhị phân

Mã nguồn

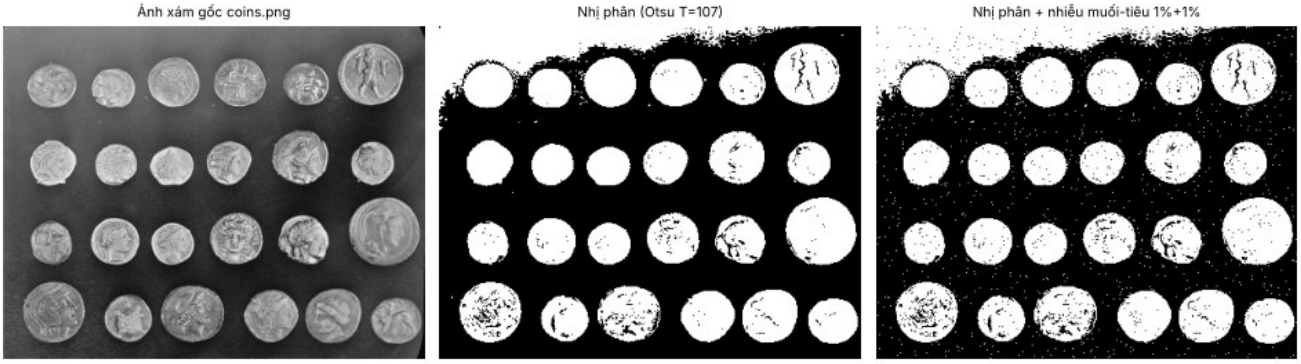
```
coins = load_gray("coins.png", data.coins)
T_otsu, bw = cv2.threshold(coins, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)   # 0/255, uint8
print("Kích thước:", coins.shape, "| ngưỡng Otsu =", T_otsu, "| số điểm trắng =", int((bw > 0).sum()))

# ảnh nhị phân có thêm nhiễu muối-tiêu (dùng để thử opening/closing)
rng = np.random.default_rng(0)
noisy = bw.copy()
r = rng.random(bw.shape)
noisy[r < 0.01] = 255          # nhiễu muối (điểm trắng)
noisy[r > 0.99] = 0            # nhiễu tiêu (điểm đen)
show([coins, bw, noisy], ["Ảnh xám gốc coins.png", f"Nhị phân (Otsu T={int(T_otsu)})", "Nhị phân + nhiễu muối-tiêu 1%+1%"],
     ncols=3, cell=4, name="b2_input.png")
```

Kết quả (văn bản)

```
Kích thước: (303, 384) | ngưỡng Otsu = 107.0 | số điểm trắng = 45117
```

Kết quả (hình)



2.1 Tự lập trình erosion, dilation, opening, closing bằng NumPy

- Phần tử cấu trúc `se` (ma trận 0/1, gốc ở tâm). Với mỗi điểm 1 của `se` ta dịch ảnh một đoạn tương ứng rồi lấy **min** (erosion) hoặc **max** (dilation) trên các ảnh đã dịch -> vector hóa toàn ảnh, không cần vòng lặp theo pixel.
- Erosion: (f⊖ B)(x)=min_s∈ B f(x+s); Dilation: (f⊕ B)(x)=max_s∈ B f(x-s) (SE được **lật** qua gốc).
- Biên ảnh: erosion đệm giá trị lớn nhất (255), dilation đệm 0 (giống mặc định của OpenCV).
- Opening = dilation(erosion(f)); Closing = erosion(dilation(f)). Cùng một code chạy cho ảnh nhị phân (0/255) lẫn ảnh xám.

Tham số: hình dạng và kích thước SE (`make_se(shape, size)`).

Mã nguồn

```
def make_se(shape, size):
    """Phần tử cấu trúc kích thước size x size (size lẻ)."""
    c = size // 2
    if shape == "rect":
        return np.ones((size, size), np.uint8)
    if shape == "ellipse":
        return cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (size, size))
    if shape == "cross":
        return cv2.getStructuringElement(cv2.MORPH_CROSS, (size, size))
    if shape == "diamond":
        yy, xx = np.ogrid[:size, :size]
        return (np.abs(yy - c) + np.abs(xx - c) <= c).astype(np.uint8)
    se = np.zeros((size, size), np.uint8)
    if shape == "hline":
        se[c, :] = 1
    elif shape == "vline":
        se[:, c] = 1
    else:
        raise ValueError(shape)
    return se


def se_img(se):
    """Ảnh minh họa SE: ô thuộc SE màu trắng, ô ngoài SE màu đen, viền xám để thấy rõ khung."""
    c = np.full((se.shape[0] + 2, se.shape[1] + 2), 90, np.uint8)
    c[1:-1, 1:-1] = se * 255
    return cv2.resize(c, None, fx=8, fy=8, interpolation=cv2.INTER_NEAREST)


def _shifted(img, se, pad_value):
    """Sinh các bản dịch của ảnh tương ứng với từng điểm 1 của se (se có gốc ở tâm)."""
    kh, kw = se.shape
    cy, cx = kh // 2, kw // 2
    p = np.pad(img, ((cy, kh - 1 - cy), (cx, kw - 1 - cx)), constant_values=pad_value)
    H, W = img.shape
```

```
    for dy, dx in zip(*np.nonzero(se)):
        yield p[dy:dy + H, dx:dx + W]


def erosion(img, se):
    out = None
    for s in _shifted(img, se, 255):
        out = s.copy() if out is None else np.minimum(out, s)
    return out


def dilation(img, se):
    se_r = se[::-1, ::-1]                    # dilation dùng SE lấy đối xứng qua gốc
    out = None
    for s in _shifted(img, se_r, 0):
        out = s.copy() if out is None else np.maximum(out, s)
    return out


def opening(img, se):
    return dilation(erosion(img, se), se)


def closing(img, se):
    return erosion(dilation(img, se), se)


def timeit(fn, *a, rep=5):
    t = time.perf_counter()
    for _ in range(rep):
        r = fn(*a)
    return r, (time.perf_counter() - t) / rep * 1000   # ms
```

So sánh với OpenCV (ảnh nhị phân và ảnh xám, nhiều SE)

`np.array_equal` = True nghĩa là kết quả **trùng khít từng pixel**.

Mã nguồn

```
ops = {
    "erosion":  (erosion,  lambda im, k: cv2.erode(im, k)),
    "dilation": (dilation, lambda im, k: cv2.dilate(im, k)),
    "opening":  (opening,  lambda im, k: cv2.morphologyEx(im, cv2.MORPH_OPEN, k)),
    "closing":  (closing,  lambda im, k: cv2.morphologyEx(im, cv2.MORPH_CLOSE, k)),
}
print(f"{'ảnh':8s} {'SE':12s} {'phép':9s} {'giống OpenCV':13s} {'t_NumPy(ms)':>12s} {'t_OpenCV(ms)':>13s}")
all_equal = True
for img_name, img in [("nhị phân", noisy), ("xám", coins)]:
    for shape, size in [("rect", 3), ("ellipse", 7), ("cross", 5), ("rect", 15)]:
        se = make_se(shape, size)
        for op, (mine, ref) in ops.items():
            a, t1 = timeit(mine, img, se)
            b, t2 = timeit(ref, img, se)
            eq = np.array_equal(a, b)
            all_equal &= eq
            print(f"{img_name:8s} {shape + str(size):12s} {op:9s} {str(eq):13s} {t1:12.2f} {t2:13.3f}")
print("\n=> Tất cả trường hợp trùng OpenCV:", all_equal)

# SE bất đối xứng: dilation của ta (SE được lật) = cv2.dilate với SE đã lật
se_asym = np.array([[1, 0, 0], [1, 1, 0], [0, 0, 0]], np.uint8)
print("SE bất đối xứng - dilation trùng cv2.dilate(SE lật):",
      np.array_equal(dilation(bw, se_asym), cv2.dilate(bw, se_asym[::-1, ::-1])))
```

Kết quả (văn bản)

```
ảnh      SE           phép      giống OpenCV   t_NumPy(ms)  t_OpenCV(ms)
nhị phân rect3        erosion   True                  0.47         0.053
nhị phân rect3        dilation  True                  0.23         0.037
nhị phân rect3        opening   True                  0.39         0.061
nhị phân rect3        closing   True                  0.44         0.061
nhị phân ellipse7     erosion   True                  0.54         0.081
nhị phân ellipse7     dilation  True                  0.52         0.080
nhị phân ellipse7     opening   True                  1.00         0.145
nhị phân ellipse7     closing   True                  1.01         0.176
nhị phân cross5       erosion   True                  0.21         0.034
nhị phân cross5       dilation  True                  0.20         0.052
nhị phân cross5       opening   True                  0.63         0.064
nhị phân cross5       closing   True                  0.71         0.114
nhị phân rect15       erosion   True                  4.35         0.067
nhị phân rect15       dilation  True                  4.39         0.089
nhị phân rect15       opening   True                  6.66         0.168
nhị phân rect15       closing   True                  6.51         0.198
xám      rect3        erosion   True                  0.36         0.041
xám      rect3        dilation  True                  0.21         0.033
xám      rect3        opening   True                  0.55         0.114
xám      rect3        closing   True                  0.49         0.070
xám      ellipse7     erosion   True                  0.83         0.101
xám      ellipse7     dilation  True                  0.63         0.098
xám      ellipse7     opening   True                  1.51         0.177
xám      ellipse7     closing   True                  1.28         0.219
xám      cross5       erosion   True                  0.27         0.035
xám      cross5       dilation  True                  0.23         0.045
xám      cross5       opening   True                  0.44         0.068
xám      cross5       closing   True                  0.59         0.090
```

Kết quả (văn bản)

```
xám      rect15       erosion   True                  4.48         0.110
```

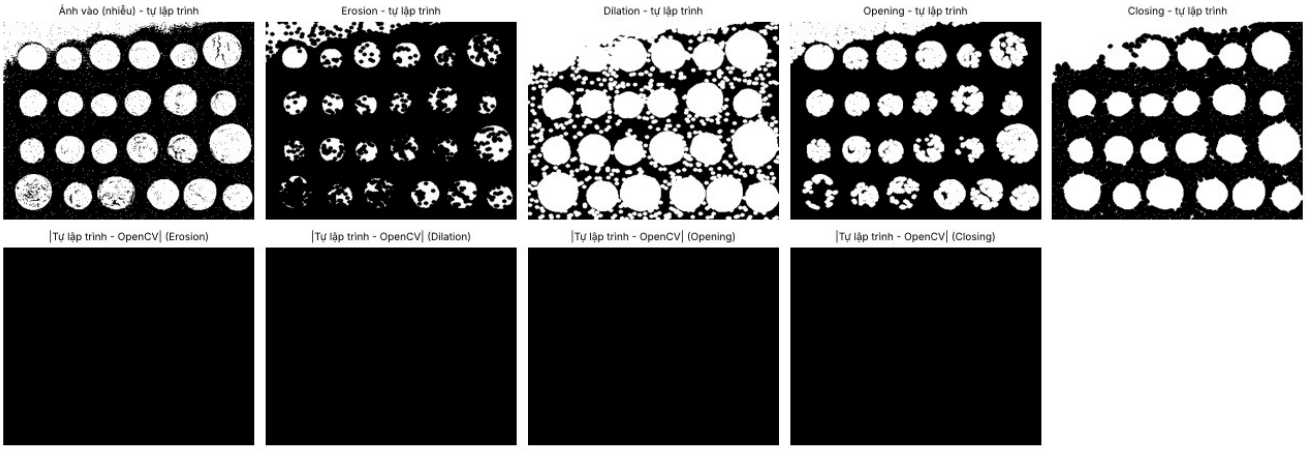
```
xám      rect15       dilation  True                 4.21         0.076
xám      rect15       opening   True                 7.55         0.126
xám      rect15       closing   True                 5.86         0.138

=> Tất cả trường hợp trùng OpenCV: True
SE bất đối xứng - dilation trùng cv2.dilate(SE lật): True
```

Mã nguồn

```
se = make_se("ellipse", 7)
res = [noisy, erosion(noisy, se), dilation(noisy, se), opening(noisy, se), closing(noisy, se)]
cvr = [noisy, cv2.erode(noisy, se), cv2.dilate(noisy, se), cv2.morphologyEx(noisy, cv2.MORPH_OPEN, se),
       cv2.morphologyEx(noisy, cv2.MORPH_CLOSE, se)]
tt = ["Ảnh vào (nhiễu)", "Erosion", "Dilation", "Opening", "Closing"]
show(res + [np.abs(a.astype(int) - b.astype(int)).astype(np.uint8) * 255 for a, b in zip(res[1:], cvr[1:])],
     [t + " - tự lập trình" for t in tt] + [f"|Tự lập trình - OpenCV| ({t})" for t in tt[1:]],
     ncols=5, cell=3.2, name="b2_vs_opencv.png")
print("Tổng sai khác tuyệt đối so với OpenCV:", [int(np.abs(a.astype(int) - b.astype(int)).sum()) for a, b in zip(res[1:],
    cvr[1:])])
```

Kết quả (hình)



Kết quả (văn bản)

```
Tổng sai khác tuyệt đối so với OpenCV: [0, 0, 0, 0]
```

Nhận xét 2.1

- Cả 32 trường hợp thử (2 ảnh: nhị phân và xám × 4 SE × 4 phép) cho kết quả **trùng OpenCV từng pixel** (tổng sai khác tuyệt đối = 0). Với SE bất đối xứng, dilation phải dùng SE đã lật qua gốc (định nghĩa chuẩn); khi đó mới trùng `cv2.dilate` với SE lật - đã kiểm chứng.
- Cách vector hóa (min/max trên các bản dịch) chạy được cho cả ảnh nhị phân và ảnh xám với cùng một đoạn mã. Thời gian NumPy 0,2 - 6 ms, OpenCV 0,03 - 0,15 ms, tức chậm hơn khoảng 6 - 50 lần: chi phí tăng tuyến tính theo số điểm của SE (rect 3×3 = 9 điểm ≈ 0,2 - 0,4 ms; rect 15×15 = 225 điểm ≈ 3 - 6 ms), còn OpenCV dùng SIMD và phân tách SE chữ nhật. Opening/closing tốn gấp đôi erosion/dilation.

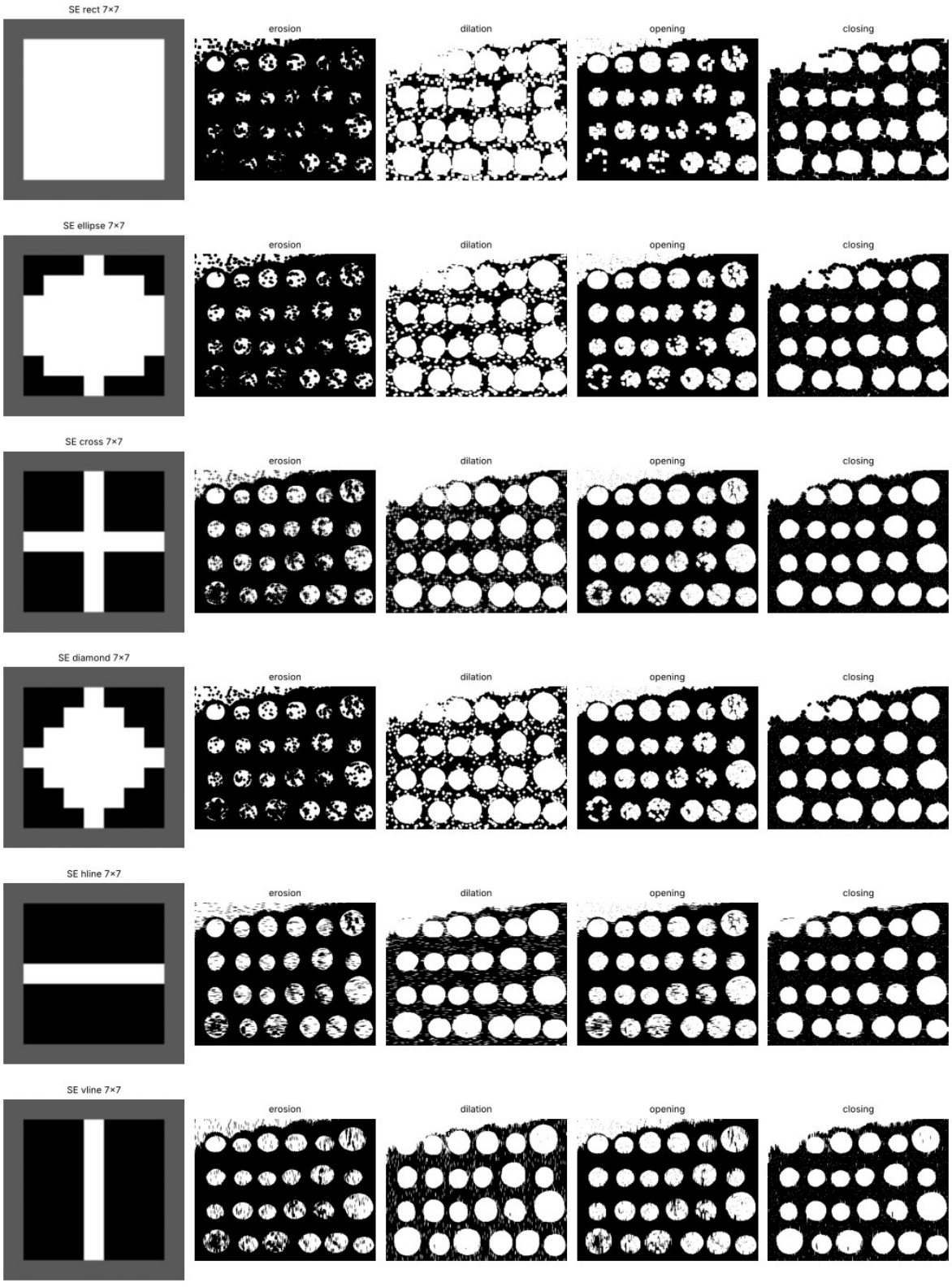
2.2 Thử nghiệm phần tử cấu trúc khác hình dạng và kích thước

(a) Cố định kích thước 7, đổi hình dạng SE (rect / ellipse / cross / diamond / hline / vline). **(b)** Cố định hình ellipse, đổi kích thước 3, 7, 15, 25. Ảnh vào là ảnh nhị phân có nhiễu muối-tiêu.

Mã nguồn

```
shapes = ["rect", "ellipse", "cross", "diamond", "hline", "vline"]
imgs, tits = [], []
for sh in shapes:
    se = make_se(sh, 7)
    imgs += [se_img(se), erosion(noisy, se), dilation(noisy, se), opening(noisy, se), closing(noisy, se)]
    tits += [f"SE {sh} 7x7", "erosion", "dilation", "opening", "closing"]
show(imgs, tits, ncols=5, cell=2.6, name="b2_se_shapes.png")
```

Kết quả (hình)



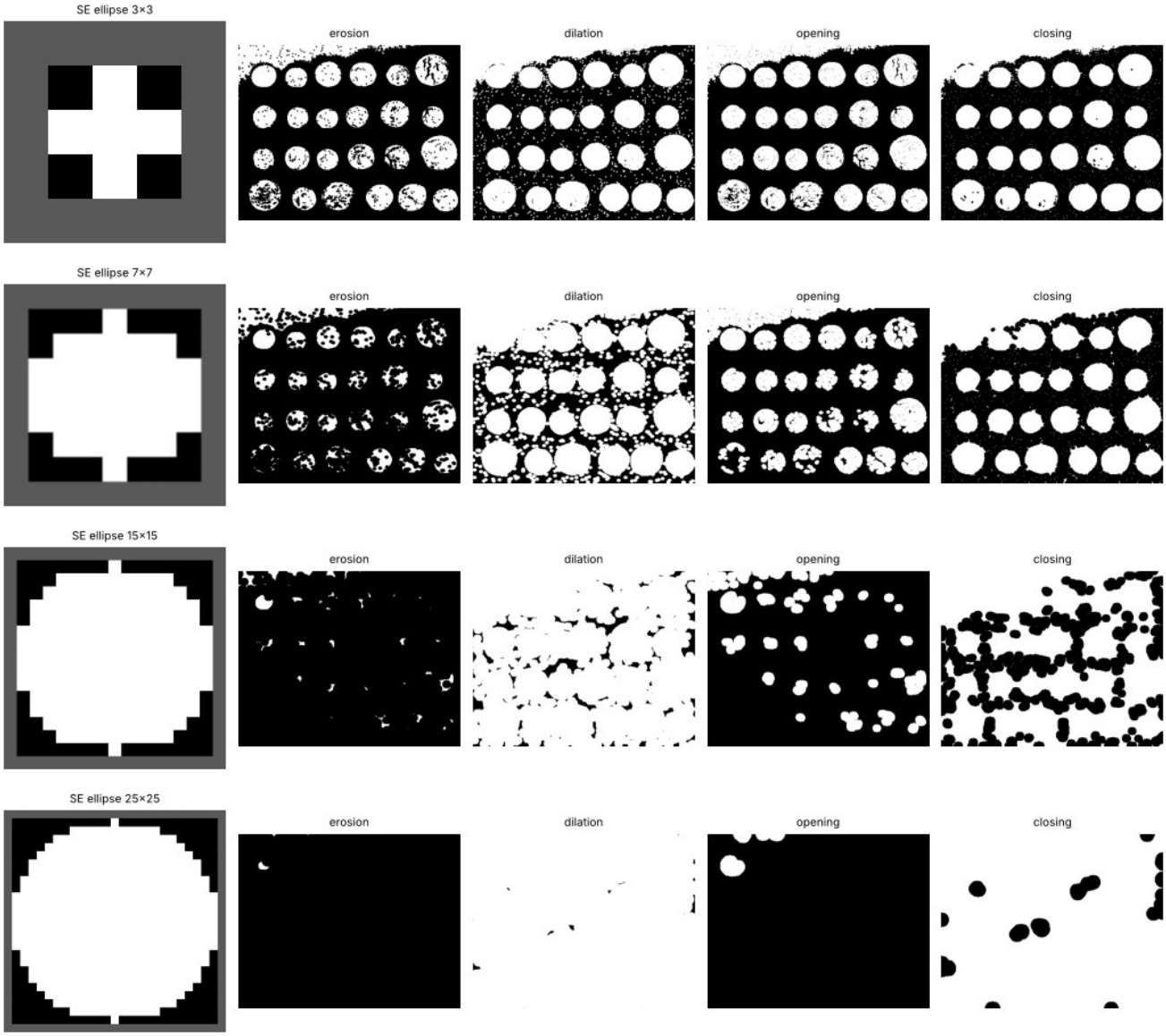
Mã nguồn

```
sizes = [3, 7, 15, 25]
imgs, tits = [], []
for sz in sizes:
    se = make_se("ellipse", sz)
    imgs += [se_img(se), erosion(noisy, se), dilation(noisy, se), opening(noisy, se), closing(noisy, se)]
    tits += [f"SE ellipse {sz}x{sz}", "erosion", "dilation", "opening", "closing"]
show(imgs, tits, ncols=5, cell=2.6, name="b2_se_sizes.png")

# bảng định lượng: diện tích vùng trắng (so với ảnh nhị phân sạch) và số thành phần liên thông
clean_area = int((bw > 0).sum())
n_clean = cv2.connectedComponents(bw, connectivity=8)[0] - 1
print(f"Ảnh sạch: diện tích = {clean_area}, số thành phần = {n_clean}")
print(f"{'SE':14s} {'phép':9s} {'diện tích':>10s} {'số TP liên thông':>17s} {'sai khác so với ảnh sạch (px)':>31s}")
for sh, sz in [("rect", 3), ("ellipse", 7), ("ellipse", 15), ("ellipse", 25), ("cross", 7), ("hline", 7)]:
    se = make_se(sh, sz)
```

```
    for op in ("opening", "closing"):
        o = (opening if op == "opening" else closing)(noisy, se)
        ncc = cv2.connectedComponents(o, connectivity=8)[0] - 1
        print(f"{sh + str(sz):14s} {op:9s} {int((o > 0).sum()):10d} {ncc:17d} {int((o != bw).sum()):31d}")
```

Kết quả (hình)



Kết quả (văn bản)

```
Ảnh sạch: diện tích = 45117, số thành phần = 96
SE             phép       diện tích  số TP liên thông   sai khác so với ảnh sạch (px)
rect3          opening        42772                36                           2395
rect3          closing        48140               583                           3053
ellipse7       opening        38118                39                           7023
ellipse7       closing        50234               464                           5145
ellipse15      opening        16097                28                          29020
ellipse15      closing        72610                45                          27505
ellipse25      opening         2177                 3                          42940
ellipse25      closing       108987                 1                          63870
cross7         opening        41558                27                           3625
cross7         closing        48517               652                           3464
hline7         opening        41940                50                           3245
hline7         closing        48565               528                           3492
```

Nhận xét 2.2

- **Hình dạng SE (7×7):** `rect` làm điểm nhiễu nở thành khối vuông góc nhọn; `ellipse` và `diamond` cho đốm tròn/thoi mượt hơn, ít làm méo hình tròn của đồng xu; `cross` có ít điểm nhất nên tác động nhẹ nhất (nhiễu nở thành dấu cộng mảnh); `hline`/`vline` có tính **định hướng**: nhiễu bị kéo thành vạch ngang/dọc và phép toán chỉ ảnh hưởng theo một phương (vd. erosion bằng `hline` chỉ cắt nét hẹp theo chiều ngang, các vệt xuất hiện song song nhau trong đồng xu). Chọn SE nên bám theo hình dạng đối tượng cần giữ.

- **Kích thước SE (ellipse) - opening:** 3×3: còn 36 thành phần, diện tích 42 772 px (lệch 2 395 px so với ảnh sạch) - diệt nhiễu muối tốt mà ít hỏng đối tượng; 7×7: 39 thành phần (lệch 7 023 px); 15×15: diện tích chỉ còn 16 097 px (lệch 29 020 px) vì mảng sáng nhỏ của đồng xu bị loại; 25×25: chỉ còn 3 thành phần (2 177 px) - SE lớn hơn đa số đối tượng thì opening xóa chúng.
- **Closing:** SE 3×3, 7×7 vẫn còn 583 và 464 thành phần vì closing chỉ lấp nhiễu tiêu (điểm đen), **không loại được nhiễu muối** (điểm trắng) - đối ngẫu của opening; 15×15: diện tích phình ra 72 610 px; 25×25: dính toàn bộ đồng xu và nền thành 1 thành phần (108 987 px). Kết luận: SE phải lớn hơn nhiễu nhưng nhỏ hơn chi tiết cần giữ; dùng opening để loại nhiễu muối/vật nhỏ, closing để lấp lỗ/khe nhỏ, và mở rồi đóng (opening -> closing) để xử lý nhiễu muối-tiêu.

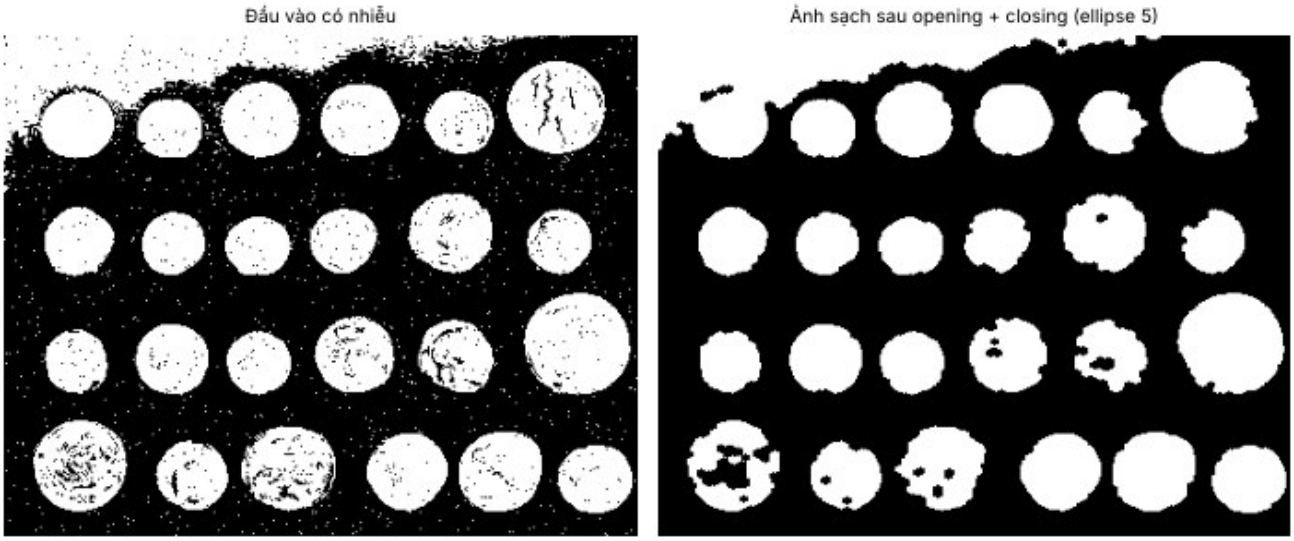
2.3 Các thuật toán hình thái học

Ảnh nhị phân đã làm sạch: `clean = closing(opening(noisy, ellipse 5), ellipse 5)`; đây là đầu vào cho các thuật toán dưới đây. Tự lập trình: **trích xuất hình bao**, **làm đầy lỗ**, **thành phần liên thông**. Dùng thư viện để đối chiếu: **convex hull**, **thinning**.

Mã nguồn

```
se5 = make_se("ellipse", 5)
clean = closing(opening(noisy, se5), se5)
show([noisy, clean], ["Đầu vào có nhiễu", "Ảnh sạch sau opening + closing (ellipse 5)"], ncols=2, cell=4.5, name="b2_clean.png")
```

Kết quả (hình)



(1) Trích xuất hình bao (boundary extraction)

β(A)=A-(A⊖ B) với B là SE 3×3 (hình bao dày 1 px). Tham số: SE `rect 3` (8-liên thông) và `cross 3` (4-liên thông).

Mã nguồn

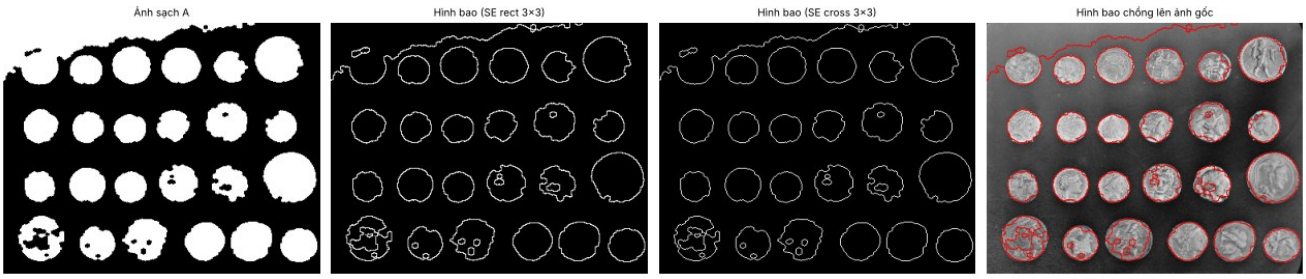
```
def boundary(A, se):
    return cv2.subtract(A, erosion(A, se))      # A - (A erosion B)

b8 = boundary(clean, make_se("rect", 3))
b4 = boundary(clean, make_se("cross", 3))
# đối chiếu: hình bao trong của scikit-image
ref8 = segmentation.find_boundaries(clean > 0, connectivity=2, mode="inner")
ref4 = segmentation.find_boundaries(clean > 0, connectivity=1, mode="inner")
print("Số pixel hình bao: SE rect3 =", int((b8 > 0).sum()), "| SE cross3 =", int((b4 > 0).sum()))
print("Khác biệt với skimage.find_boundaries (mode=inner): rect3 ->", int(((b8 > 0) != ref8).sum()),
      "px, cross3 ->", int(((b4 > 0) != ref4).sum()), "px")
overlay = cv2.cvtColor(coins, cv2.COLOR_GRAY2RGB)
overlay[b8 > 0] = (255, 0, 0)
show([clean, b8, b4, overlay], ["Ảnh sạch A", "Hình bao (SE rect 3x3)", "Hình bao (SE cross 3x3)", "Hình bao chồng lên ảnh
    gốc"],
     ncols=4, cell=4, name="b2_boundary.png")
```

Kết quả (văn bản)

```
Số pixel hình bao: SE rect3 = 5364 | SE cross3 = 3897
Khác biệt với skimage.find_boundaries (mode=inner): rect3 -> 0 px, cross3 -> 0 px
```

Kết quả (hình)



(2) Làm đầy lỗ (hole filling)

Cách tự động (không cần chọn điểm gốc): nền liên thông với biên ảnh được dựng lại bằng giãn có điều kiện X_k=(X_k-1⊕ B)∩ A^c với X_0 = các điểm nền nằm trên biên ảnh, B = SE chữ thập (4-liên thông). Các lỗ = A^c - X_∈fty; ảnh đã đầy = A ∪ lỗ. Lặp đến khi hội tụ.

Mã nguồn

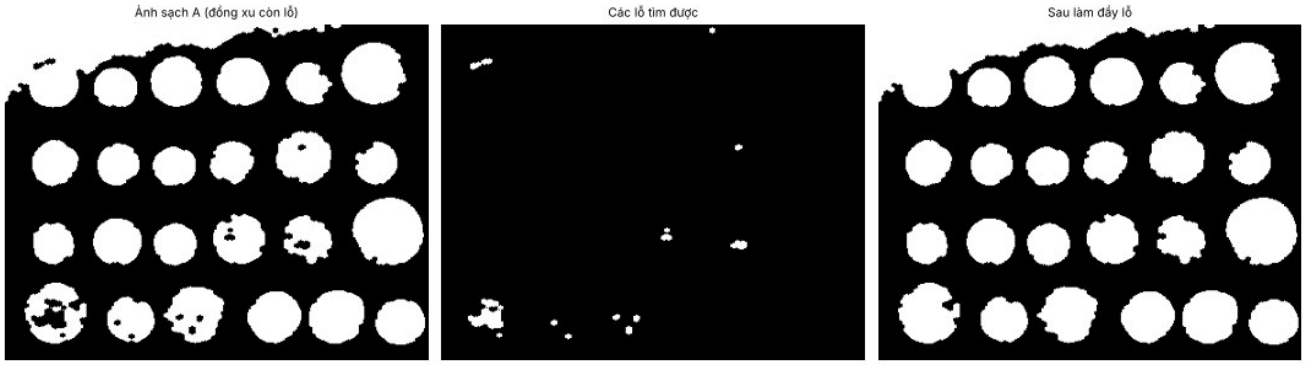
```
def fill_holes(A, se=None):
    se = make_se("cross", 3) if se is None else se
    Ac = 255 - A
    X = np.zeros_like(A)
    X[0, :], X[-1, :], X[:, 0], X[:, -1] = Ac[0, :], Ac[-1, :], Ac[:, 0], Ac[:, -1]   # nền chạm biên ảnh
    it = 0
    while True:
        Xn = cv2.bitwise_and(dilation(X, se), Ac)
        it += 1
        if np.array_equal(Xn, X):
            break
        X = Xn
    holes = cv2.subtract(Ac, X)
    return cv2.bitwise_or(A, holes), holes, it

from scipy import ndimage as ndi
filled, holes, n_it = fill_holes(clean)
ref = ndi.binary_fill_holes(clean > 0)
print(f"Số vòng lặp hội tụ: {n_it} | diện tích lỗ = {int((holes > 0).sum())} px | "
      f"khác scipy.ndimage.binary_fill_holes: {int(((filled > 0) != ref).sum())} px")
show([clean, holes, filled], ["Ảnh sạch A (đồng xu còn lỗ)", "Các lỗ tìm được", "Sau làm đầy lỗ"], ncols=3, cell=4.5,
    name="b2_fill.png")
```

Kết quả (văn bản)

```
Số vòng lặp hội tụ: 205 | diện tích lỗ = 956 px | khác scipy.ndimage.binary_fill_holes: 0 px
```

Kết quả (hình)



(3) Thành phần liên thông (connected components)

Lấy một điểm chưa gán nhãn làm hạt X_0, lặp X_k=(X_k-1⊕ B)∩ A đến khi hội tụ; X_∈fty là một thành phần. Lặp cho đến hết điểm trắng. B = `rect 3x3` (8-liên thông) hoặc `cross 3x3` (4-liên thông). Đối chiếu `cv2.connectedComponents`.

Mã nguồn

```
def connected_components(A, se):
    remaining = A > 0
    labels = np.zeros(A.shape, np.int32)
    n = 0
    while remaining.any():
        ys, xs = np.nonzero(remaining)
        X = np.zeros_like(A)
        X[ys[0], xs[0]] = 255
        while True:
            Xn = cv2.bitwise_and(dilation(X, se), A)
            if np.array_equal(Xn, X):
                break
            X = Xn
        n += 1
        labels[X > 0] = n
```

```
        remaining &= ~(X > 0)
    return n, labels

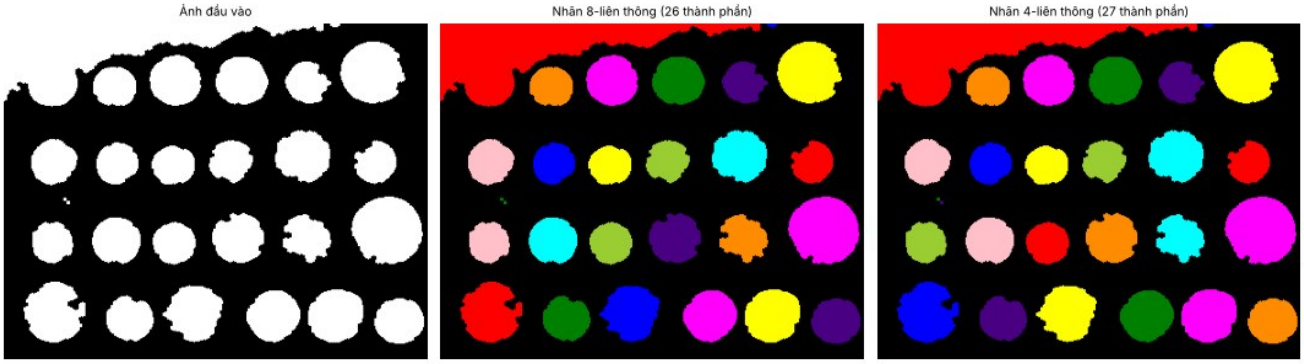
def same_partition(l1, l2):
    """Hai ảnh nhãn là một phép phân hoạch giống nhau (không phụ thuộc số thứ tự nhãn)."""
    pairs = set(zip(l1.ravel().tolist(), l2.ravel().tolist()))
    return len(pairs) == len(set(p[0] for p in pairs)) == len(set(p[1] for p in pairs))

# ảnh thử: ảnh sạch đã làm đầy lỗ, thêm vài vật nhỏ chạm chéo để thấy khác biệt 4/8-liên thông
test = filled.copy()
free = np.argwhere(erosion(255 - filled, make_se("rect", 9)) > 0)   # vùng nền trống 9x9
y0, x0 = free[len(free) // 2]
test[y0:y0 + 3, x0:x0 + 3] = 255
test[y0 + 3:y0 + 6, x0 + 3:x0 + 6] = 255   # hai khối 3x3 chỉ chạm nhau ở góc chéo
t0 = time.perf_counter()
n8, lab8 = connected_components(test, make_se("rect", 3))
n4, lab4 = connected_components(test, make_se("cross", 3))
t_my = time.perf_counter() - t0
cv8 = cv2.connectedComponents(test, connectivity=8)
cv4 = cv2.connectedComponents(test, connectivity=4)
print(f"Tự lập trình: 8-liên thông = {n8} TP, 4-liên thông = {n4} TP   (thời gian {t_my:.2f}s)")
print(f"Hai khối chéo đặt tại (y={y0}, x={x0}): 8-conn gộp thành 1 TP, 4-conn tách thành 2 TP")
print(f"OpenCV      : 8-liên thông = {cv8[0]-1} TP, 4-liên thông = {cv4[0]-1} TP")
print("Phân hoạch giống OpenCV: 8-conn =", same_partition(lab8, cv8[1]), "| 4-conn =", same_partition(lab4, cv4[1]))
props = measure.regionprops(lab8)
print("Diện tích các TP (8-conn):", sorted(int(p.area) for p in props))
rgb8 = (color.label2rgb(lab8, bg_label=0, bg_color=(0, 0, 0)) * 255).astype(np.uint8)
show([test, rgb8, (color.label2rgb(lab4, bg_label=0, bg_color=(0, 0, 0)) * 255).astype(np.uint8)],
     ["Ảnh đầu vào", f"Nhãn 8-liên thông ({n8} thành phần)", f"Nhãn 4-liên thông ({n4} thành phần)"], ncols=3, cell=4.5,
         name="b2_cc.png")
```

Kết quả (văn bản)

```
Tự lập trình: 8-liên thông = 26 TP, 4-liên thông = 27 TP   (thời gian 0.60s)
Hai khối chéo đặt tại (y=157, x=54): 8-conn gộp thành 1 TP, 4-conn tách thành 2 TP
OpenCV      : 8-liên thông = 26 TP, 4-liên thông = 27 TP
Phân hoạch giống OpenCV: 8-conn = True | 4-conn = True
Diện tích các TP (8-conn): [18, 23, 1074, 1075, 1078, 1097, 1107, 1116, 1121, 1138, 1295, 1302, 1335, 1448, 1455, 1620, 1623, 1663,
    1717, 1800, 1911, 1933, 2187, 2490, 3024, 8473]
```

Kết quả (hình)



(4) Convex hull

Với mỗi thành phần liên thông, so sánh `skimage.morphology.convex_hull_image` và `cv2.convexHull`; độ đặc (solidity) = diện tích / diện tích bao lồi.

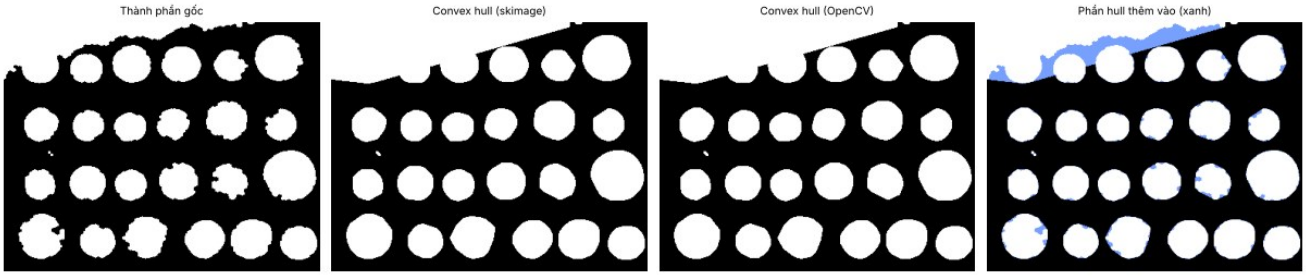
Mã nguồn

```
hull_sk = np.zeros(test.shape, bool)
hull_cv = np.zeros(test.shape, np.uint8)
sol = []
for p in props:
    m = lab8 == p.label
    h = morphology.convex_hull_image(m)
    hull_sk |= h
    cnt, _ = cv2.findContours(m.astype(np.uint8), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    pts = np.vstack(cnt)
    cv2.fillPoly(hull_cv, [cv2.convexHull(pts)], 255)
    sol.append(m.sum() / h.sum())
inter = (hull_sk & (hull_cv > 0)).sum()
union = (hull_sk | (hull_cv > 0)).sum()
print(f"IoU giữa hai cách tính convex hull = {inter/union:.4f}")
print("Solidity từng thành phần: min = %.3f, max = %.3f, TB = %.3f" % (min(sol), max(sol), np.mean(sol)))
ov = cv2.cvtColor(test, cv2.COLOR_GRAY2RGB)
ov[hull_sk & (test == 0)] = (120, 160, 255)
show([test, hull_sk, hull_cv, ov], ["Thành phần gốc", "Convex hull (skimage)", "Convex hull (OpenCV)", "Phần hull thêm vào
    (xanh)"],
     ncols=4, cell=3.8, name="b2_hull.png")
```

Kết quả (văn bản)

```
IoU giữa hai cách tính convex hull = 0.9968
Solidity từng thành phần: min = 0.636, max = 0.972, TB = 0.929
```

Kết quả (hình)



(5) Thinning (làm mảnh) - ảnh chữ in `page.png`

Ngưỡng thích nghi tách nét chữ, sau đó `skimage.morphology.thin` (thinning) và `skeletonize` (xương) để so sánh.

Mã nguồn

```
page = load_gray("page.png", data.page)
text = cv2.resize(page, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)        # phóng 3x để nét chữ đủ dày
tbw = cv2.adaptiveThreshold(text, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 51, 12)
n_cc, cc, st, _ = cv2.connectedComponentsWithStats(tbw, connectivity=8)
keep = st[:, cv2.CC_STAT_AREA] >= 30
keep[0] = False                                                                  # nhãn 0 = nền
tbw = keep[cc].astype(np.uint8) * 255                                            # bỏ hạt nhiễu < 30 px
thin = morphology.thin(tbw > 0)
skel = morphology.skeletonize(tbw > 0)
print("Số pixel nét chữ:", int((tbw > 0).sum()), "| sau thinning:", int(thin.sum()), "| skeletonize:", int(skel.sum()))
print("Tỉ lệ giữ lại: thinning = %.1f%%, skeleton = %.1f%%" % (100 * thin.sum() / (tbw > 0).sum(), 100 * skel.sum() / (tbw >
    0).sum()))
crop = (slice(0, 135), slice(0, 690))
show([text[crop], tbw[crop], thin[crop], skel[crop]], ["Ảnh gốc page.png (phóng 3x, vùng tiêu đề)", "Nhị phân nét chữ (adaptive
    51/12)", "Thinning", "Skeletonize"],
     ncols=2, cell=7, name="b2_thinning.png")
```

Kết quả (văn bản)

```
Số pixel nét chữ: 84224 | sau thinning: 14104 | skeletonize: 14489
Tỉ lệ giữ lại: thinning = 16.7%, skeleton = 17.2%
```

Kết quả (hình)



Nhận xét 2.3

- **Hình bao:** SE `rect 3×3` cho 5 364 px (đường biên dày, 4-liên thông), SE `cross 3×3` cho 3 897 px (mỏng hơn, 8-liên thông). Cả hai trùng 100% (0 pixel khác biệt) với `skimage.segmentation.find_boundaries(mode='inner')` tương ứng.
- **Làm đầy lỗ:** thuật toán dựng nền từ biên ảnh hội tụ sau 205 vòng, lấp 956 px lỗ (do vân đồng xu thấp hơn ngưỡng Otsu) và trùng 100% với `scipy.ndimage.binary_fill_holes`. Số vòng lớn vì nền lan 1 px mỗi vòng với SE chữ thập; không cần chọn điểm hạt cho từng lỗ.
- **Thành phần liên thông:** tự lập trình cho 26 thành phần (8-liên thông) và 27 thành phần (4-liên thông) - hơn 1 vì hai khối 3×3 đặt chạm nhau ở góc chéo chỉ dính nhau theo 8-liên thông; phân hoạch trùng `cv2.connectedComponents` ở cả hai kiểu. 23 thành phần có diện tích 1 074 - 3 024 px là các đồng xu; thành phần 8 473 px là vùng nền sáng ở mép trên **dính cả đồng xu góc trên-trái** (do ngưỡng Otsu); 18 px là cặp khối thử và 23 px là một hạt nhỏ. Bản morphological mất ~0,7 s cho hai lượt (lặp giãn theo từng thành phần) - chậm hơn nhiều so với duyệt hai lượt của OpenCV nhưng cho đúng kết quả.
- **Convex hull:** hai cách tính (skimage, OpenCV) trùng IoU = 0,997. Độ đặc (solidity) trung bình 0,929 (0,636 - 0,972): đồng xu gần lồi nên cao; vùng nền sáng mép trên có hình lõm nên thấp nhất (0,636); đồng xu bị khuyết/có vết lõm thấp hơn các đồng xu tròn.
- **Thinning (trang chữ in, phóng 3×):** 84 224 px nét chữ -> 14 104 px (16,7%) với `thin` và 14 489 px (17,2%) với `skeletonize`. Cả hai cho nét dày 1 px, giữ được hình dạng và tính liên thông của chữ; skeleton có thêm vài nhánh cụt nhỏ nên nhiều điểm hơn chút. Kết quả phụ thuộc chất lượng nhị phân

hóa (ảnh gốc chữ mảnh nên phải phóng 3× và dùng adaptive threshold, nếu dùng opening 3×3 như thử ban đầu thì các nét chữ mỏng bị xóa).

Ảnh chụp màn hình minh chứng - Bài 2

Chụp trên nền cửa sổ thư mục chứa mã sinh viên: (1) cửa sổ thư mục hiển thị tên thư mục/mã SV và file `bai2.ipynb`; (2) notebook đang chạy với kết quả hiển thị; (3) thời gian/tên tài khoản trên màn hình nếu có.

[Chèn ảnh chụp màn hình tại đây]

Đặt file vào thư mục **screenshots/** với tên `bai2_1.png`, `bai2_2.png`... rồi chạy lại `python tools/make_report.py`.

Bài 3. Phát hiện biên và phân ngưỡng

Nội dung	Tham số khảo sát
Sobel	ksize 3/5/7; Gauss σ 0/1/2; ngưỡng 20-25% max
Laplacian / LoG	ksize 1/3/5; LoG σ 1/2/3; ngưỡng zero-crossing 4% max
Canny	(low,high) = 20/60, 50/150, 100/200, 150/250; σ 0/1/2/3; apertureSize 3/5; L2gradient
Ngưỡng	cố định T = 60/107/150; Otsu tự lập vs OpenCV vs skimage; adaptive MEAN/GAUSSIAN, block 11/31/101, C 2/10
Nhiễu	Gauss σ_n = 10/25/50; muối-tiêu 2%/5%; bộ lọc Gauss 5×5, median 5; đo F1 (dung sai 1 px) và Dice

Nội dung: (1) Sobel, Laplacian (LoG), Canny và khảo sát tham số; (2) phân ngưỡng cố định, Otsu (tự lập trình, đối chiếu OpenCV / scikit-image) và adaptive thresholding; (3) ảnh hưởng của nhiễu (Gaussian, muối-tiêu) đến phát hiện biên và phân ngưỡng.

Ảnh đầu vào: `camera.png` (512×512, xám, ảnh mẫu scikit-image) cho phát hiện biên; `coins.png` (303×384) cho ngưỡng cố định/Otsu; `page.png` (191×384, trang chữ có chiếu sáng không đều) cho adaptive thresholding.

Mã nguồn - cài đặt và hàm tiện ích

```
# ===== Cài đặt & hàm tiện ích dùng chung =====
import os, time, warnings
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import data, color, filters, morphology, measure, segmentation, util
warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 80, "font.size": 9, "axes.titlesize": 9})

# Thư mục ảnh đầu vào / kết quả.
# Muốn dùng ảnh trong bộ dữ liệu LMS: chép ảnh vào images/ và đổi tên file ở ô "Ảnh đầu vào".
IMG_DIR, OUT_DIR = "images", "outputs/bai3"
for d in (IMG_DIR, OUT_DIR):
    os.makedirs(d, exist_ok=True)


def load_rgb(name, fallback, max_side=None):
    """Đọc ảnh màu RGB từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, cv2.cvtColor(fallback(), cv2.COLOR_RGB2BGR))
    img = cv2.cvtColor(cv2.imread(p, cv2.IMREAD_COLOR), cv2.COLOR_BGR2RGB)
    if max_side and max(img.shape[:2]) > max_side:
        s = max_side / max(img.shape[:2])
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def load_gray(name, fallback, max_side=None):
    """Đọc ảnh xám uint8 từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, fallback())
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if max_side and max(img.shape) > max_side:
        s = max_side / max(img.shape)
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def show(images, titles=None, ncols=4, name=None, cell=3.0, cmap="gray"):
    """Hiển thị lưới ảnh; ảnh 2-D hiển thị theo thang xám cố định; lưu vào OUT_DIR/name."""
    n = len(images)
    ncols = min(ncols, n)
    nrows = int(np.ceil(n / ncols))
    fig, axs = plt.subplots(nrows, ncols, figsize=(cell * ncols, (cell * images[0].shape[0] / images[0].shape[1] + 0.35) *
        nrows), squeeze=False)
    for i, ax in enumerate(axs.ravel()):
        ax.axis("off")
        if i >= n:
            continue
        im = images[i]
        if im.ndim == 2:
            if im.dtype == np.uint8:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=255)
            elif im.dtype == bool:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=1)
            else:
                ax.imshow(im, cmap=cmap)
        else:
            ax.imshow(im)
        if titles is not None:
            ax.set_title(titles[i])
    fig.tight_layout()
    if name:
```

```
        fig.savefig(os.path.join(OUT_DIR, name), dpi=110, bbox_inches="tight")
    plt.show()
```

3.0 Ảnh đầu vào

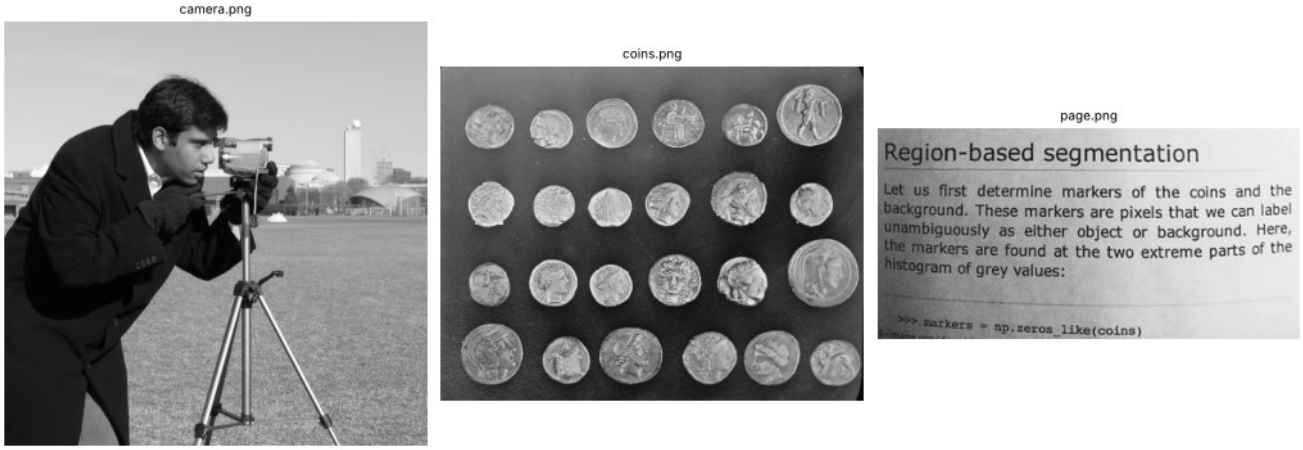
Mã nguồn

```
cam = load_gray("camera.png", data.camera)
coins = load_gray("coins.png", data.coins)
page = load_gray("page.png", data.page)
print("camera:", cam.shape, "| coins:", coins.shape, "| page:", page.shape)
show([cam, coins, page], ["camera.png", "coins.png", "page.png"], ncols=3, cell=4.3, name="b3_input.png")
```

Kết quả (văn bản)

```
camera: (512, 512) | coins: (303, 384) | page: (191, 384)
```

Kết quả (hình)



3.1 Phát hiện biên: Sobel, Laplacian, Canny

Sobel

Gradient theo x, y bằng nhân chập Sobel; độ lớn $|\nabla f|=\sqrt{(G_x^2+G_y^2)}$. Cài đặt bằng `cv2.filter2D` với nhân Sobel tự định nghĩa rồi đối chiếu `cv2.Sobel`. **Tham số khảo sát:** kích thước nhân `ksize` ∈ {3, 5, 7}; làm trơn Gaussian trước (σ ∈ {0, 1, 2}); ngưỡng nhị phân hóa độ lớn gradient.

Mã nguồn

```
Kx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], np.float32)
Ky = Kx.T
f = cam.astype(np.float32)
gx = cv2.filter2D(f, -1, Kx, borderType=cv2.BORDER_REFLECT_101)
gy = cv2.filter2D(f, -1, Ky, borderType=cv2.BORDER_REFLECT_101)
mag_mine = np.hypot(gx, gy)
mag_cv = np.hypot(cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3), cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3))
print("Sobel tự lập (filter2D) so với cv2.Sobel: sai khác lớn nhất =", float(np.abs(mag_mine - mag_cv).max()))

def sobel_mag(img, ksize=3, sigma=0):
    g = cv2.GaussianBlur(img, (0, 0), sigma) if sigma > 0 else img
    g = g.astype(np.float32)
    return np.hypot(cv2.Sobel(g, cv2.CV_32F, 1, 0, ksize=ksize), cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=ksize))

def norm8(m):
    return np.clip(m / m.max() * 255, 0, 255).astype(np.uint8)

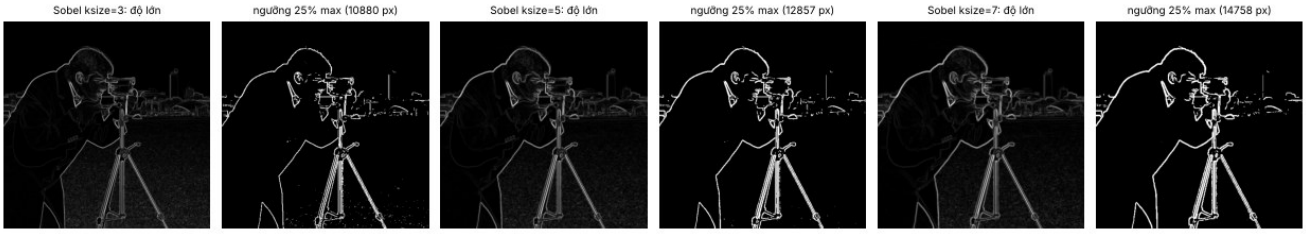
# (a) ảnh hưởng ksize (không làm trơn)
imgs, tits = [], []
for k in (3, 5, 7):
    m = sobel_mag(cam, k)
    imgs += [norm8(m), (m > 0.25 * m.max()).astype(np.uint8) * 255]
    tits += [f"Sobel ksize={k}: độ lớn", f"ngưỡng 25% max ({int((m > 0.25 * m.max()).sum())} px)"]
show(imgs, tits, ncols=6, cell=2.6, name="b3_sobel_ksize.png")

# (b) ảnh hưởng làm trơn Gaussian (ksize=3), ngưỡng cố định theo % của giá trị max
imgs, tits = [], []
for sg in (0, 1, 2):
    m = sobel_mag(cam, 3, sg)
    e = (m > 0.2 * m.max())
    imgs += [norm8(m), e]
    tits += [f"Sobel sigma={sg}: độ lớn", f"ngưỡng 20% max ({int(e.sum())} px)"]
show(imgs, tits, ncols=6, cell=2.6, name="b3_sobel_sigma.png")
```

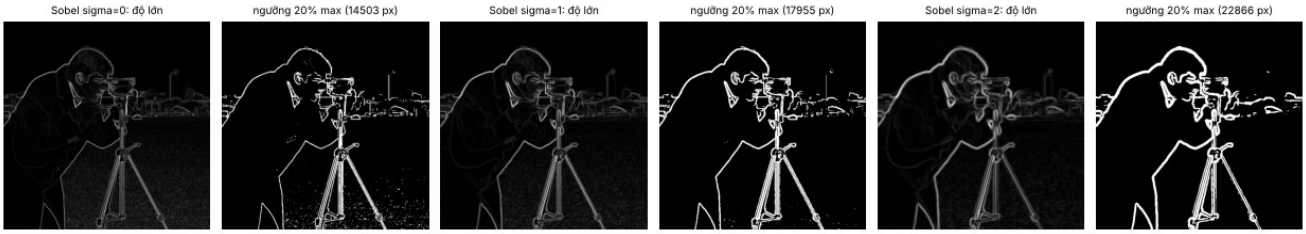
Kết quả (văn bản)

```
Sobel tự lập (filter2D) so với cv2.Sobel: sai khác lớn nhất = 0.0
```

Kết quả (hình)



Kết quả (hình)



Laplacian / LoG

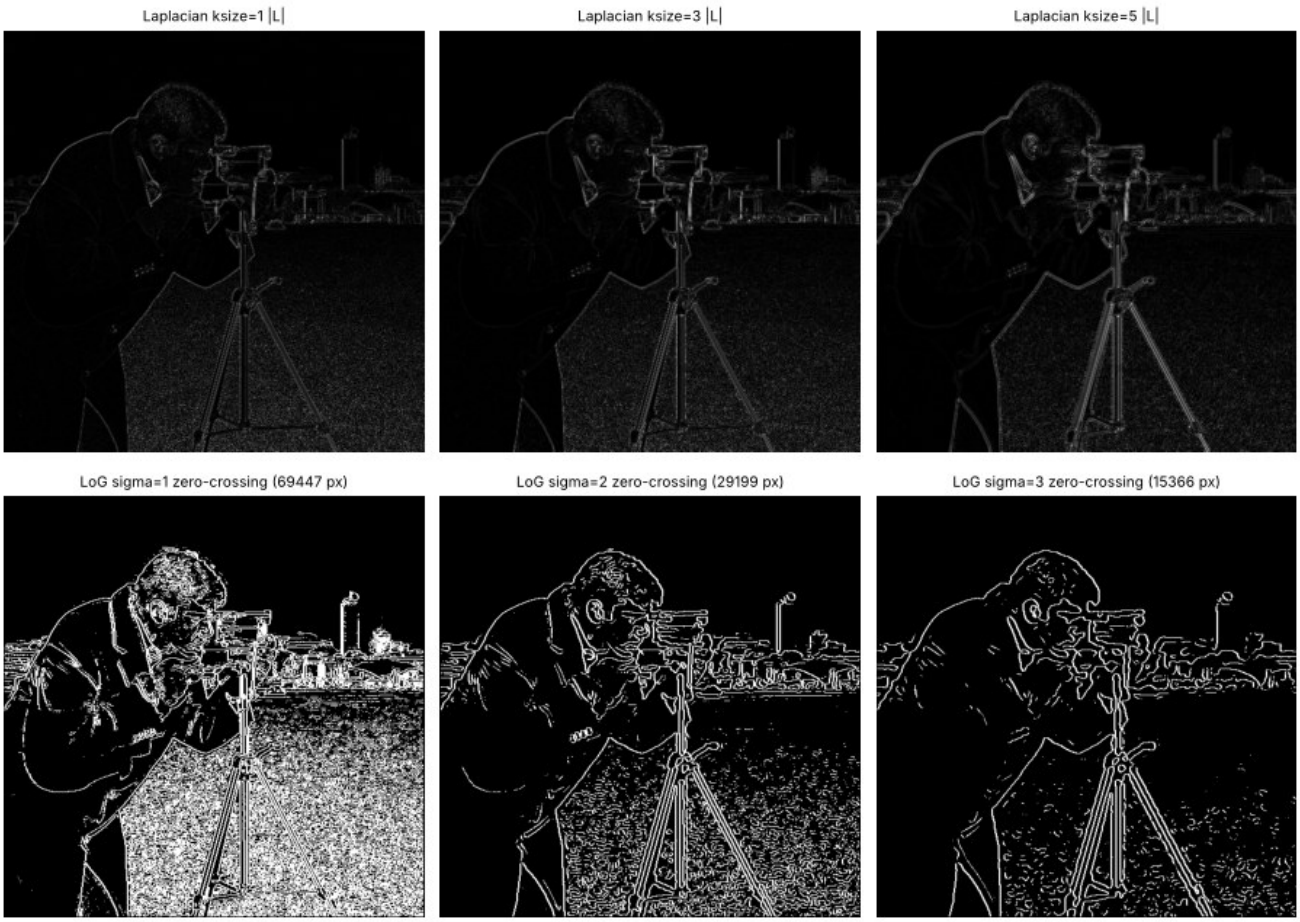
Laplacian là đạo hàm bậc hai (nhạy nhiễu) nên thường làm trơn Gaussian trước -> **LoG**. Biên là các điểm **zero-crossing** (đổi dấu) với biên độ đủ lớn. **Tham số khảo sát:** `ksize` của `cv2.Laplacian` ∈ {1, 3, 5}; σ của LoG ∈ {1, 2, 3}; ngưỡng zero-crossing.

Mã nguồn

```
def zero_crossing(L, thr):
    """Điểm đổi dấu giữa láng giềng (ngang/dọc/chéo) và chênh biên độ > thr (dịch ảnh bằng np.roll)."""
    zc = np.zeros(L.shape, bool)
    for dy, dx in [(0, 1), (1, 0), (1, 1), (1, -1)]:
        b = np.roll(L, (-dy, -dx), axis=(0, 1))
        zc |= ((L * b) < 0) & (np.abs(L - b) > thr)
    zc[0, :] = zc[-1, :] = False
    zc[:, 0] = zc[:, -1] = False
    return zc

# (a) cv2.Laplacian với các ksize (ảnh gốc, không làm trơn)
imgs, tits = [], []
for k in (1, 3, 5):
    L = cv2.Laplacian(cam.astype(np.float32), cv2.CV_32F, ksize=k)
    imgs += [norm8(np.abs(L))]
    tits += [f"Laplacian ksize={k} |L|"]
# (b) LoG với các sigma: làm trơn Gaussian rồi Laplacian + zero-crossing
for sg in (1, 2, 3):
    g = cv2.GaussianBlur(cam.astype(np.float32), (0, 0), sg)
    L = cv2.Laplacian(g, cv2.CV_32F, ksize=3)
    zc = zero_crossing(L, 0.04 * np.abs(L).max())
    imgs += [zc]
    tits += [f"LoG sigma={sg} zero-crossing ({int(zc.sum())} px)"]
show(imgs, tits, ncols=3, cell=3.8, name="b3_laplacian.png")
```

Kết quả (hình)



Canny

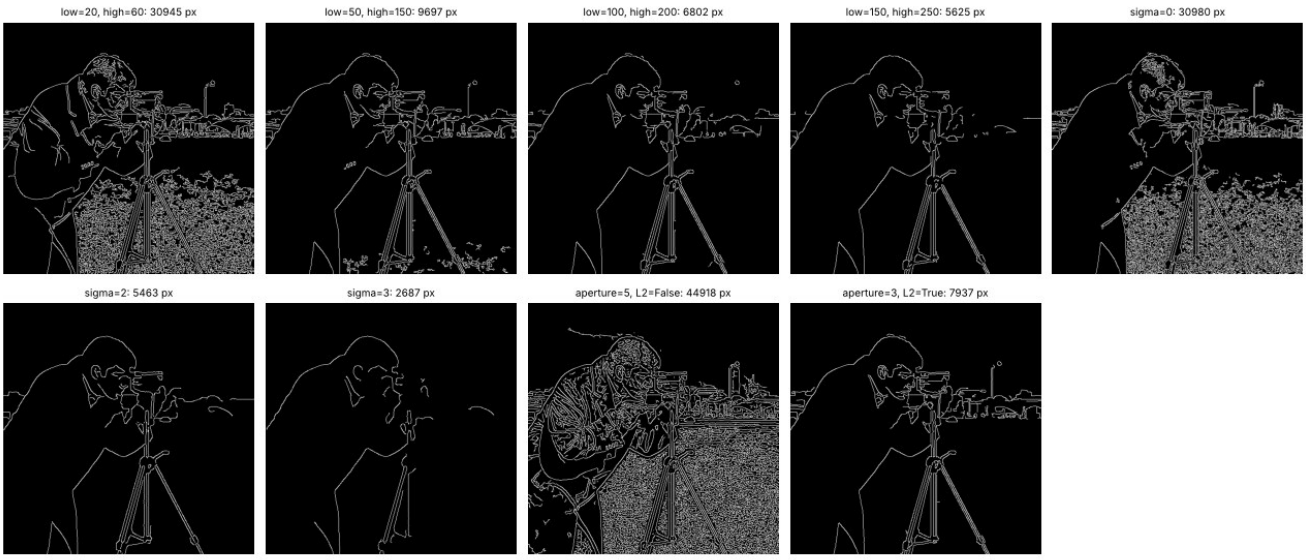
Làm trơn Gaussian -> gradient -> non-maximum suppression -> ngưỡng kép (`low`, `high`) và liên kết biên theo độ trễ. **Tham số khảo sát:** (low, high) ∈ {(20,60), (50,150), (100,200), (150,250)}; σ làm trơn ∈ {0, 1, 2, 3}; `apertureSize` ∈ {3, 5}; `L2gradient`.

Mã nguồn

```
def canny(img, lo, hi, sigma=0, ap=3, l2=False):
    g = cv2.GaussianBlur(img, (0, 0), sigma) if sigma > 0 else img
    return cv2.Canny(g, lo, hi, apertureSize=ap, L2gradient=l2)

# (a) ngưỡng (low, high), sigma = 1
pairs = [(20, 60), (50, 150), (100, 200), (150, 250)]
imgs = [canny(cam, lo, hi, 1) for lo, hi in pairs]
tits = [f"low={lo}, high={hi}: {int((e > 0).sum())} px" for (lo, hi), e in zip(pairs, imgs)]
# (b) sigma, ngưỡng (50, 150)
for sg in (0, 2, 3):
    e = canny(cam, 50, 150, sg)
    imgs.append(e); tits.append(f"sigma={sg}: {int((e > 0).sum())} px")
# (c) apertureSize / L2gradient
for ap, l2 in [(5, False), (3, True)]:
    e = canny(cam, 50, 150, 1, ap, l2)
    imgs.append(e); tits.append(f"aperture={ap}, L2={l2}: {int((e > 0).sum())} px")
show(imgs, tits, ncols=5, cell=3.2, name="b3_canny_params.png")
```

Kết quả (hình)



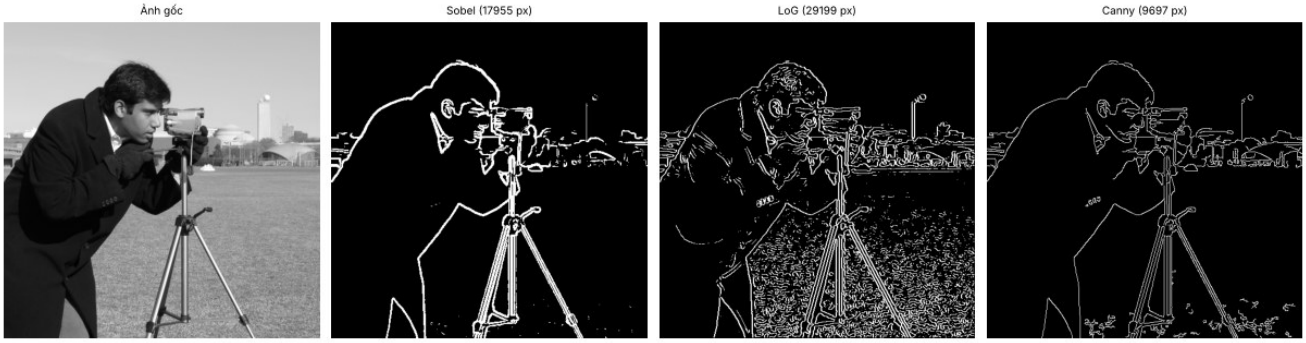
So sánh ba bộ dò biên trên cùng ảnh

Sobel (σ=1, ngưỡng 20% max), LoG (σ=2) và Canny (σ=1, 50/150).

Mã nguồn

```
m = sobel_mag(cam, 3, 1)
e_sobel = m > 0.2 * m.max()
g2 = cv2.GaussianBlur(cam.astype(np.float32), (0, 0), 2)
L2 = cv2.Laplacian(g2, cv2.CV_32F, ksize=3)
e_log = zero_crossing(L2, 0.04 * np.abs(L2).max())
e_canny = canny(cam, 50, 150, 1) > 0
show([cam, e_sobel, e_log, e_canny],
     ["Ảnh gốc", f"Sobel ({int(e_sobel.sum())} px)", f"LoG ({int(e_log.sum())} px)", f"Canny ({int(e_canny.sum())} px)"],
     ncols=4, cell=3.8, name="b3_edge_compare.png")
```

Kết quả (hình)



Nhận xét 3.1

- **Sobel:** cài đặt bằng `filter2D` trùng hoàn toàn `cv2.Sobel` (sai khác lớn nhất 0,0). Tăng `ksize` 3 -> 5 -> 7 (ngưỡng 25% max) làm số điểm biên tăng 10 880 -> 12 857 -> 14 758 và biên dày hơn; nhiễu hạt trên bãi cỏ giảm vì nhân lớn tự làm trơn, nhưng chi tiết nhỏ (tòa nhà xa) bị nhòe. Làm trơn Gaussian trước (σ = 0, 1, 2; ngưỡng 20% max): 14 503 -> 17 955 -> 22 866 điểm; hạt cỏ biến mất nhưng đường biên to bè, định vị kém. Sobel luôn cho biên dày nhiều pixel và phụ thuộc ngưỡng chọn tay.
- **Laplacian / LoG:** `cv2.Laplacian` trên ảnh gốc gần như chỉ thấy nhiễu hạt vì đạo hàm bậc hai khuếch đại nhiễu. LoG + zero-crossing: σ = 1, 2, 3 cho 69 447, 29 199, 15 366 điểm - σ tăng thì biên liền mạch, bớt nhiễu nhưng mất chi tiết. Ưu điểm là biên mảnh và khép kín; nhược điểm là sinh nhiều biên giả ở vùng phẳng (cỏ) và không cho biết hướng/độ mạnh biên.
- **Canny:** số điểm biên giảm dần khi tăng ngưỡng (low/high) 20/60 -> 50/150 -> 100/200 -> 150/250: 30 945 -> 9 697 -> 6 802 -> 5 625; ngưỡng thấp lấy cả kết cấu cỏ, ngưỡng cao chỉ giữ biên chính nhưng bắt đầu làm đứt đường chân trời. σ = 0, 1, 2, 3 (ngưỡng 50/150): 30 980, 9 697, 5 463, 2 687 điểm - σ lớn tạo biên sạch nhưng mất chi tiết nhỏ. `apertureSize = 5` cho 44 918 điểm vì đạo hàm nhân lớn không chuẩn hóa có biên độ lớn hơn nên cùng ngưỡng sẽ lọt nhiều điểm hơn; `L2gradient = True` cho 7 937 điểm (ít hơn) do độ lớn L2 ≤ L1.

- **So sánh:** Canny cho biên mảnh 1 px, liên tục, ít biên giả nhất (NMS + ngưỡng trễ); Sobel dày; LoG khép kín nhưng nhiều biên giả. Cùng một ảnh: Sobel (σ=1) 17 955, LoG (σ=2) 29 199, Canny (σ=1, 50/150) 9 697 điểm.

3.2 Phân ngưỡng: cố định, Otsu, adaptive

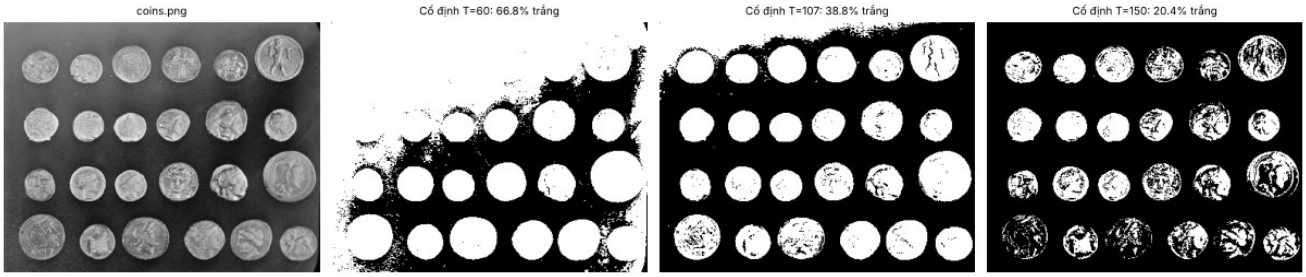
Ngưỡng cố định

Khảo sát T∈{60,107,150} trên `coins.png` (107 xấp xỉ ngưỡng Otsu).

Mã nguồn

```
imgs, tits = [coins], ["coins.png"]
for T in (60, 107, 150):
    _, b = cv2.threshold(coins, T, 255, cv2.THRESH_BINARY)
    imgs.append(b); tits.append(f"Cố định T={T}: {100 * (b > 0).mean():.1f}% trắng")
show(imgs, tits, ncols=4, cell=3.8, name="b3_fixed.png")
```

Kết quả (hình)



Otsu - tự lập trình

Chọn T cực đại phương sai giữa hai lớp: σ_B^2(T)=((μ_T ω(T)-μ(T))^2)/(ω(T) (1-ω(T))) với ω(T)=Σ_i≤ Tp_i, μ(T)=Σ_i≤ Ti p_i, μ_T=μ(255). So sánh với `cv2.THRESH_OTSU` và `skimage.filters.threshold_otsu`.

Mã nguồn

```
def otsu_threshold(img):
    hist = np.bincount(img.ravel(), minlength=256).astype(np.float64)
    p = hist / hist.sum()
    omega = np.cumsum(p)                        # xác suất lớp nền (<= T)
    mu = np.cumsum(p * np.arange(256))          # trung bình tích lũy
    mu_t = mu[-1]
    denom = omega * (1 - omega)
    sigma_b2 = np.zeros(256)
    ok = denom > 1e-12
    sigma_b2[ok] = (mu_t * omega[ok] - mu[ok]) ** 2 / denom[ok]
    return int(np.argmax(sigma_b2)), sigma_b2

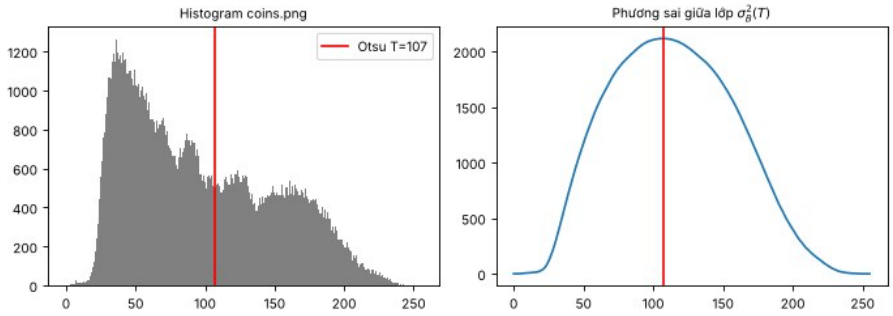
print(f"{'ảnh':10s} {'Otsu tự lập':>12s} {'OpenCV':>8s} {'skimage':>9s} {'điểm khác biệt (tự lập vs OpenCV)':>36s}")
for name, im in [("coins", coins), ("camera", cam), ("page", page)]:
    t_my, _ = otsu_threshold(im)
    t_cv, b_cv = cv2.threshold(im, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    t_sk = filters.threshold_otsu(im)
    _, b_my = cv2.threshold(im, t_my, 255, cv2.THRESH_BINARY)
    print(f"{name:10s} {t_my:12d} {int(t_cv):8d} {t_sk:9.1f} {int((b_my != b_cv).sum()):36d}")

t_my, sb2 = otsu_threshold(coins)
fig, ax = plt.subplots(1, 3, figsize=(13, 3.2))
ax[0].bar(range(256), np.bincount(coins.ravel(), minlength=256), width=1.0, color="gray")
ax[0].axvline(t_my, color="r", label=f"Otsu T={t_my}"); ax[0].set_title("Histogram coins.png"); ax[0].legend()
ax[1].plot(sb2); ax[1].axvline(t_my, color="r"); ax[1].set_title(r"Phương sai giữa lớp $\sigma_B^2(T)$")
ax[2].imshow(cv2.threshold(coins, t_my, 255, cv2.THRESH_BINARY)[1], cmap="gray"); ax[2].set_title("Kết quả Otsu tự lập trình");
    ax[2].axis("off")
fig.tight_layout(); fig.savefig(os.path.join(OUT_DIR, "b3_otsu.png"), dpi=110, bbox_inches="tight"); plt.show()
```

Kết quả (văn bản)

```
ảnh         Otsu tự lập   OpenCV   skimage    điểm khác biệt (tự lập vs OpenCV)
coins               107      107     107.0                                    0
camera              102      102     102.0                                    0
page                157      157     157.0                                    0
```

Kết quả (hình)



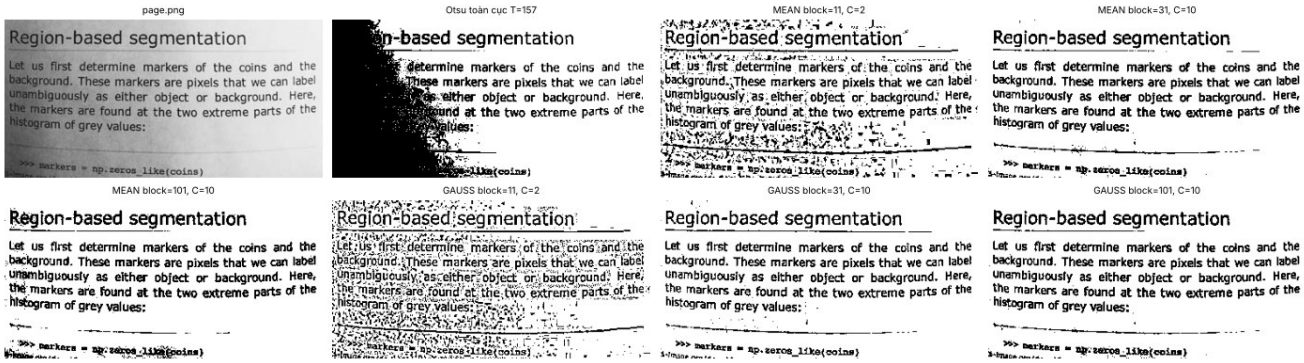
Adaptive thresholding

Ngưỡng cục bộ T(x,y)=mean/gauss_block(x,y)-C. Ảnh `page.png` chiếu sáng không đều: so sánh với Otsu toàn cục. **Tham số khảo sát:** `blockSize` ∈ {11, 31, 101}, `C` ∈ {2, 10}, phương pháp MEAN / GAUSSIAN.

Mã nguồn

```
t_p, _ = otsu_threshold(page)
imgs = [page, cv2.threshold(page, t_p, 255, cv2.THRESH_BINARY)[1]]
tits = ["page.png", f"Otsu toàn cục T={t_p}"]
for meth, mn in [(cv2.ADAPTIVE_THRESH_MEAN_C, "MEAN"), (cv2.ADAPTIVE_THRESH_GAUSSIAN_C, "GAUSS")]:
    for bs, C in [(11, 2), (31, 10), (101, 10)]:
        imgs.append(cv2.adaptiveThreshold(page, 255, meth, cv2.THRESH_BINARY, bs, C))
        tits.append(f"{mn} block={bs}, C={C}")
show(imgs, tits, ncols=4, cell=4.4, name="b3_adaptive.png")
```

Kết quả (hình)



Nhận xét 3.2

- **Ngưỡng cố định** (coins): T = 60 làm 66,8% pixel thành trắng - nền sáng ở góc trên bị gộp vào đối tượng; T = 150 chỉ còn 20,4%, đồng xu chỉ còn các mảng sáng và bị thủng nhiều; T = 107 (≈ Otsu, 38,8%) tách tốt nhất, nhưng vùng nền ở góc trên vẫn dính vào do chiếu sáng không đều. Ngưỡng cố định phải chọn tay và chỉ đúng với một điều kiện chiếu sáng.
- **Otsu tự lập trình** trùng tuyệt đối với OpenCV và scikit-image: coins T = 107, camera T = 102, page T = 157, số pixel khác biệt giữa mặt nạ tự lập và OpenCV = 0. Histogram coins có hai đỉnh (nền tối, đồng xu sáng) nên Otsu hoạt động tốt. Với `page.png` (bị bóng/chiếu sáng không đều) Otsu **toàn cục** (T = 157) làm mất cả nửa trái của trang thành khối đen, chữ không đọc được.
- **Adaptive thresholding** giải quyết vấn đề này vì ngưỡng tính theo vùng lân cận. `blockSize` nhỏ (11) với C = 2 cho nhiễu hạt dày đặc ở nền và viền chữ bị vỡ, vì ngưỡng cục bộ bám theo nhiễu; `blockSize` 31 hoặc 101 với C = 10 cho chữ rõ, nền sạch (GAUSSIAN mượt hơn MEAN, MEAN 101 còn vệt ở mép trái/đáy). C lớn loại nhiễu nền nhưng có nguy cơ đứt nét mảnh. Adaptive không phù hợp khi vật thể lớn hơn cửa sổ: trên `coins.png` (block 31 < đường kính đồng xu) nó chỉ giữ hoa văn/cạnh, thân đồng xu bị rỗng (Dice với mặt nạ Otsu chỉ 0,78 ngay cả khi ảnh sạch).

3.3 Ảnh hưởng của nhiễu

Thêm nhiễu Gaussian (σ_n = 10, 25, 50) và nhiễu muối-tiêu (2%, 5%) vào `camera.png` / `coins.png`.

- **Biên:** ảnh biên tham chiếu = Canny (σ=1, 50/150) trên ảnh sạch. Đo precision/recall/F1 với dung sai 1 px của các bộ dò: Sobel (ngưỡng 20% max), Canny không làm trơn, Canny σ=1.5, và Canny sau khử nhiễu (Gaussian 5×5 hoặc median 5).
- **Ngưỡng:** so với mặt nạ Otsu của ảnh sạch bằng Dice; theo dõi ngưỡng Otsu. So sánh Otsu, adaptive (block 31, C=10) và Otsu sau khử nhiễu.

Mã nguồn

```
def add_gauss(img, sigma, seed=0):
    r = np.random.default_rng(seed)
    return np.clip(img.astype(np.float32) + r.normal(0, sigma, img.shape), 0, 255).astype(np.uint8)

def add_sp(img, amount, seed=0):
    r = np.random.default_rng(seed)
    o = img.copy(); u = r.random(img.shape)
    o[u < amount / 2] = 0; o[u > 1 - amount / 2] = 255
    return o

noises = {"sạch": lambda im: im,
          "Gauss σ=10": lambda im: add_gauss(im, 10), "Gauss σ=25": lambda im: add_gauss(im, 25), "Gauss σ=50": lambda im:
              add_gauss(im, 50),
          "Muối-tiêu 2%": lambda im: add_sp(im, 0.02), "Muối-tiêu 5%": lambda im: add_sp(im, 0.05)}

def edge_prf(pred, ref, tol=1):
    k = np.ones((2 * tol + 1, 2 * tol + 1), np.uint8)
    pred, ref = pred > 0, ref > 0
    ref_d, pred_d = cv2.dilate(ref.astype(np.uint8), k) > 0, cv2.dilate(pred.astype(np.uint8), k) > 0
    prec = (pred & ref_d).sum() / max(pred.sum(), 1)
    rec = (ref & pred_d).sum() / max(ref.sum(), 1)
    return prec, rec, 2 * prec * rec / max(prec + rec, 1e-9)

def sobel_edges(im):
    m = sobel_mag(im, 3, 0)
    return m > 0.2 * m.max()

detectors = {
    "Sobel":                lambda im: sobel_edges(im),
    "Canny 50/150":         lambda im: canny(im, 50, 150, 0),
    "Canny σ=1.5":          lambda im: canny(im, 50, 150, 1.5),
    "Gauss5 + Canny":       lambda im: canny(cv2.GaussianBlur(im, (5, 5), 0), 50, 150, 0),
    "Median5 + Canny":      lambda im: canny(cv2.medianBlur(im, 5), 50, 150, 0),
}
ref_edge = canny(cam, 50, 150, 1)
noisy_cam = {k: fn(cam) for k, fn in noises.items()}

rows = {}
print(f"{'nhiễu':14s} " + " ".join(f"{d:>16s}" for d in detectors))
for nk, nim in noisy_cam.items():
    row = []
    for dn, dfn in detectors.items():
        _, _, f1 = edge_prf(dfn(nim), ref_edge)
        row.append(f1)
    rows[nk] = row
    print(f"{nk:14s} " + " ".join(f"{v:16.3f}" for v in row) + "   (F1)")
print("\nSố điểm biên phát hiện (ảnh sạch có", int((ref_edge > 0).sum()), "điểm tham chiếu):")
for nk, nim in noisy_cam.items():
    print(f"{nk:14s} " + " ".join(f"{int((dfn(nim) > 0).sum()):16d}" for dfn in detectors.values()))

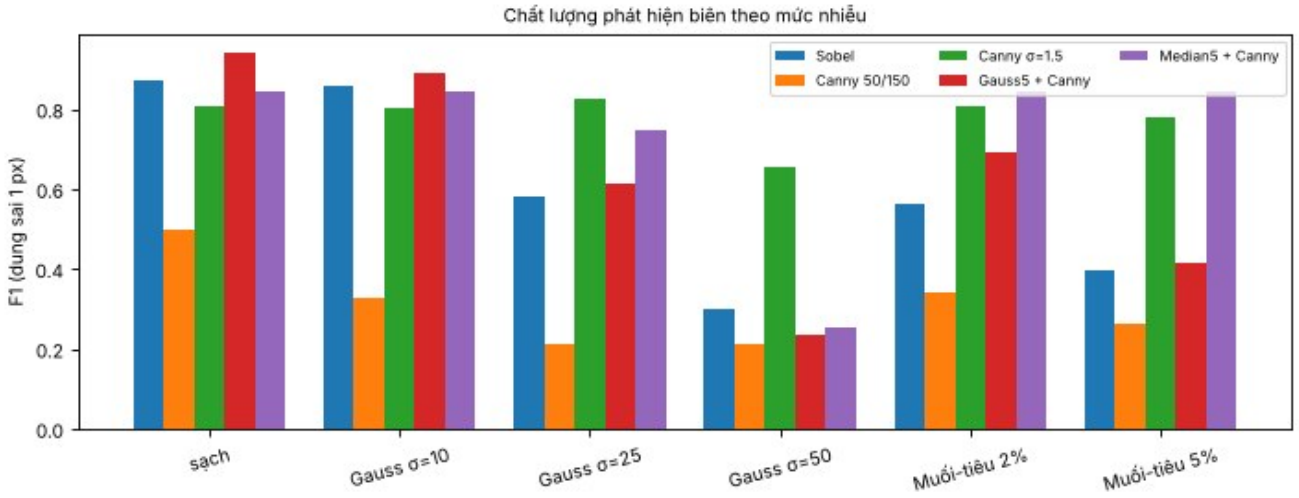
fig, ax = plt.subplots(figsize=(9, 3.6))
w = 0.16
for i, dn in enumerate(detectors):
    ax.bar(np.arange(len(noises)) + i * w, [rows[nk][i] for nk in noises], w, label=dn)
ax.set_xticks(np.arange(len(noises)) + 2 * w); ax.set_xticklabels(noises.keys(), rotation=15)
ax.set_ylabel("F1 (dung sai 1 px)"); ax.set_title("Chất lượng phát hiện biên theo mức nhiễu"); ax.legend(fontsize=7, ncol=3)
fig.tight_layout(); fig.savefig(os.path.join(OUT_DIR, "b3_noise_f1.png"), dpi=110, bbox_inches="tight"); plt.show()
```

Kết quả (văn bản)

```
nhiễu                     Sobel     Canny 50/150      Canny σ=1.5   Gauss5 + Canny  Median5 + Canny
sạch                      0.870            0.499            0.807            0.940            0.843   (F1)
Gauss σ=10                0.859            0.326            0.804            0.891            0.843   (F1)
Gauss σ=25                0.580            0.211            0.824            0.615            0.745   (F1)
Gauss σ=50                0.299            0.214            0.657            0.233            0.255   (F1)
Muối-tiêu 2%              0.560            0.342            0.808            0.690            0.845   (F1)
Muối-tiêu 5%              0.396            0.264            0.778            0.416            0.842   (F1)

Số điểm biên phát hiện (ảnh sạch có 9697 điểm tham chiếu):
sạch                         14503            30980             6505             9135             8221
Gauss σ=10                   15754            54396             6504            10112             8130
Gauss σ=25                   36478            95134             6979            20105            11686
Gauss σ=50                  104008            97500            13243            76274            66452
Muối-tiêu 2%                 33899            52209             6919            16865             8207
Muối-tiêu 5%                 56792            73719             7933            36376             8389
```

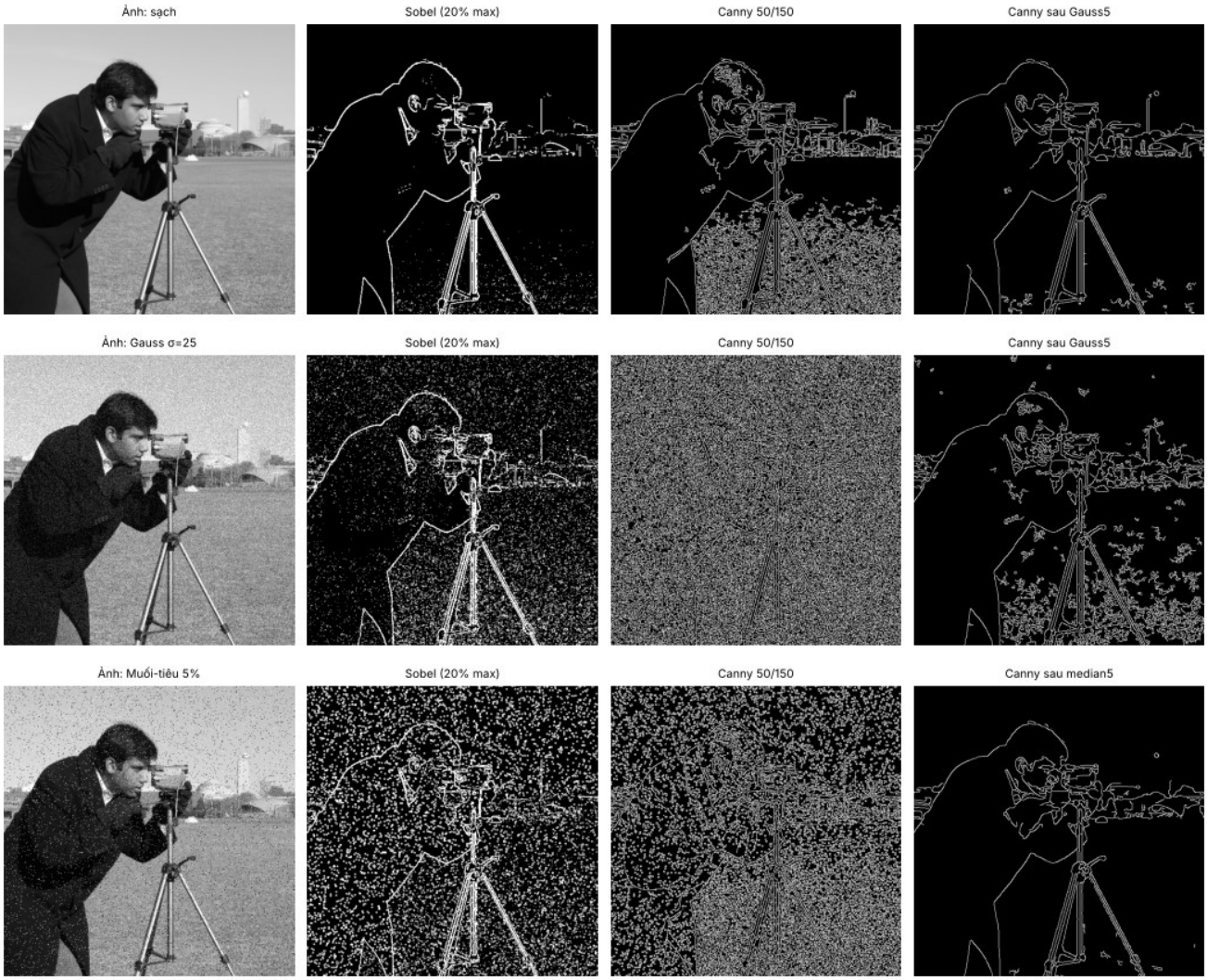
Kết quả (hình)



Mã nguồn

```
sel = ["sạch", "Gauss σ=25", "Muối-tiêu 5%"]
imgs, tits = [], []
for nk in sel:
    nim = noisy_cam[nk]
    imgs += [nim, sobel_edges(nim), canny(nim, 50, 150, 0), canny(cv2.medianBlur(nim, 5) if "Muối" in nk else
        cv2.GaussianBlur(nim, (5, 5), 0), 50, 150, 0)]
    tits += [f"Ảnh: {nk}", "Sobel (20% max)", "Canny 50/150", "Canny sau " + ("median5" if "Muối" in nk else "Gauss5")]
show(imgs, tits, ncols=4, cell=3.3, name="b3_noise_edges.png")
```

Kết quả (hình)



Ảnh hưởng của nhiễu đến phân ngưỡng (coins.png)

Mã nguồn

```
t0, _ = otsu_threshold(coins)
ref_mask = cv2.threshold(coins, t0, 255, cv2.THRESH_BINARY)[1] > 0
def dice(a, b):
    return 2 * (a & b).sum() / max(a.sum() + b.sum(), 1)

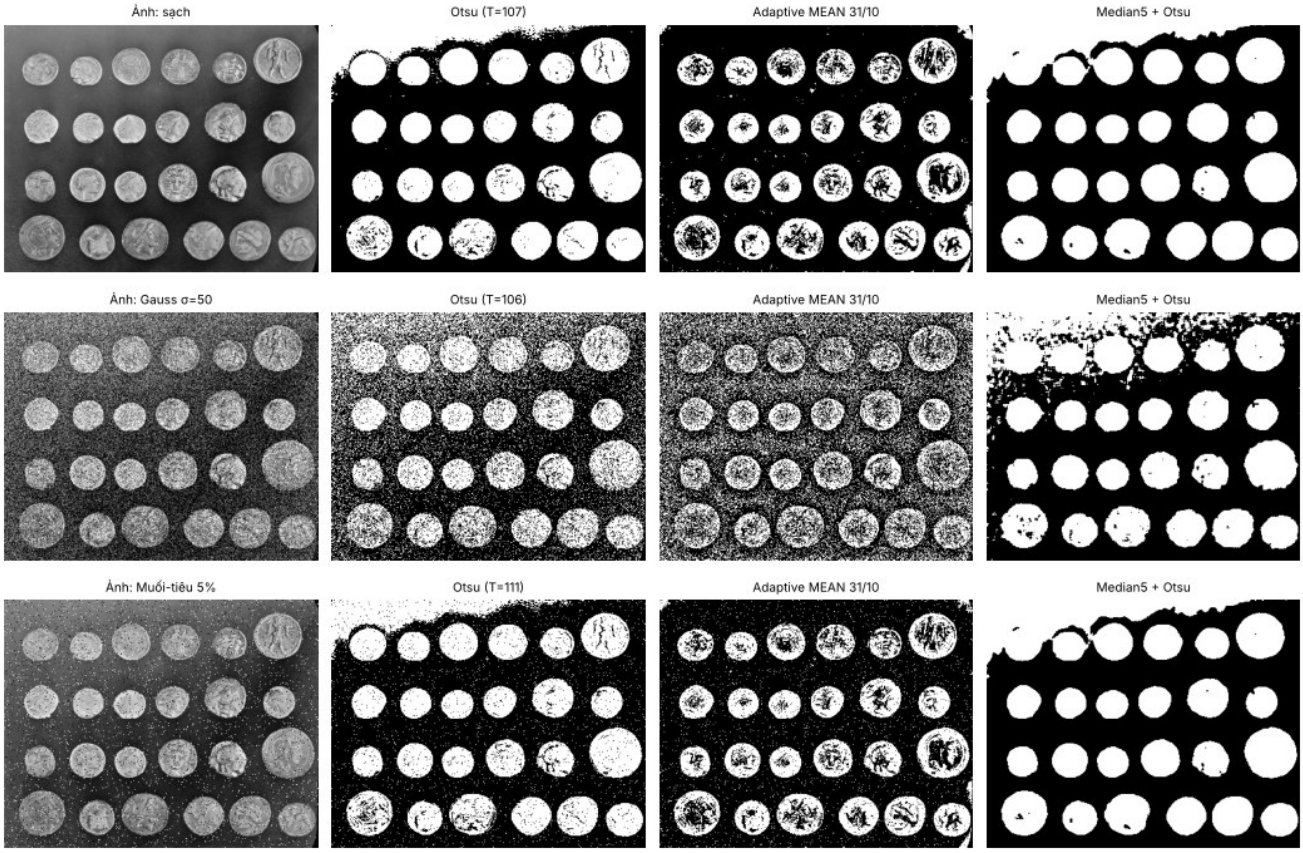
methods = {
    "Otsu":                lambda im: cv2.threshold(im, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU),
    "Adaptive MEAN 31/10":  lambda im: (None, cv2.adaptiveThreshold(im, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 31,
        -10)),
    "Median5 + Otsu":       lambda im: cv2.threshold(cv2.medianBlur(im, 5), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU),
    "Gauss5 + Otsu":        lambda im: cv2.threshold(cv2.GaussianBlur(im, (5, 5), 0), 0, 255, cv2.THRESH_BINARY +
        cv2.THRESH_OTSU),
}
noisy_coins = {k: fn(coins) for k, fn in noises.items()}
print(f"Ngưỡng Otsu ảnh sạch = {t0}\n")
print(f"{'nhiễu':14s} " + " ".join(f"{m:>22s}" for m in methods) + "   (Dice so với mặt nạ Otsu ảnh sạch)")
dice_rows = {}
for nk, nim in noisy_coins.items():
    vals, ts = [], []
    for mn, mf in methods.items():
        t, b = mf(nim)
        vals.append(dice(b > 0, ref_mask)); ts.append(t)
    dice_rows[nk] = vals
    print(f"{nk:14s} " + " ".join(f"{v:22.3f}" for v in vals) + f"   T_Otsu(thô) = {int(ts[0])}")
sel = ["sạch", "Gauss σ=50", "Muối-tiêu 5%"]
imgs, tits = [], []
for nk in sel:
    nim = noisy_coins[nk]
    imgs += [nim, methods["Otsu"](nim)[1], methods["Adaptive MEAN 31/10"](nim)[1], methods["Median5 + Otsu"](nim)[1]]
    tits += [f"Ảnh: {nk}", f"Otsu (T={int(methods['Otsu'](nim)[0])})", "Adaptive MEAN 31/10", "Median5 + Otsu"]
show(imgs, tits, ncols=4, cell=3.3, name="b3_noise_thresh.png")
```

Kết quả (văn bản)

```
Ngưỡng Otsu ảnh sạch = 107

nhiễu                            Otsu    Adaptive MEAN 31/10         Median5 + Otsu          Gauss5 + Otsu   (Dice so với mặt nạ
    Otsu ảnh sạch)
sạch                           1.000                 0.784                 0.967                 0.961   T_Otsu(thô) = 107
Gauss σ=10                     0.952                 0.764                 0.963                 0.959   T_Otsu(thô) = 106
Gauss σ=25                     0.875                 0.688                 0.945                 0.947   T_Otsu(thô) = 106
Gauss σ=50                     0.751                 0.603                 0.913                 0.919   T_Otsu(thô) = 106
Muối-tiêu 2%                   0.981                 0.778                 0.966                 0.951   T_Otsu(thô) = 108
Muối-tiêu 5%                   0.946                 0.767                 0.964                 0.940   T_Otsu(thô) = 111
```

Kết quả (hình)



Nhận xét 3.3 (biên tham chiếu = Canny σ=1, 50/150 của ảnh sạch, nên F1 chỉ mang tính so sánh tương đối, không phải ground-truth do người vẽ)

- **Phát hiện biên:** Sobel (ngưỡng 20% max cố định) có số điểm biên tăng 14 503 -> 36 478 (Gauss σ=25) -> 104 008 (σ=50) và F1 giảm 0,87 -> 0,58 -> 0,30, vì nhiễu làm gradient tăng trên toàn ảnh.

Canny **không làm trơn** là kém nhất: ngay với σ_n = 10 số điểm biên tăng 30 980 -> 54 396 và F1 xuống 0,33; với σ_n = 25 chỉ còn 0,21 (biên thật bị chìm trong biên giả). Canny σ = 1,5 bền nhất với nhiễu Gauss (F1 0,81 -> 0,82 -> 0,66 ở σ_n = 0/25/50); Gauss 5×5 + Canny tốt nhất khi ảnh sạch (0,94) nhưng giảm nhanh hơn khi nhiễu mạnh (0,23 ở σ_n = 50).

- **Nhiễu muối-tiêu:** làm trơn Gauss không hiệu quả (F1 chỉ 0,42 ở 5%) vì nó lan điểm nhiễu thành đốm nhỏ; **median 5×5 + Canny** gần như không bị ảnh hưởng (F1 0,84 ở cả 0% và 5%) vì median loại điểm ngoại lai mà không làm nhòe biên. Với nhiễu Gauss, median 5×5 cũng không kém Gauss 5×5 (σ_n = 25: 0,75 so với 0,62; σ_n = 50: 0,26 so với 0,23, cả hai đều thấp) nhưng thua ở ảnh sạch (0,84 so với 0,94). **Chọn bộ lọc khử nhiễu theo loại nhiễu** là điều quan trọng nhất.
- **Phân ngưỡng** (Dice so với mặt nạ Otsu của ảnh sạch): ngưỡng Otsu T gần như không đổi (106 - 111 so với 107) nhưng chất lượng giảm vì nhiễu làm mỗi lớp xuất hiện pixel thuộc lớp kia: Otsu Dice 0,95 / 0,88 / 0,75 với Gauss σ_n = 10 / 25 / 50 và 0,98 / 0,95 với muối-tiêu 2% / 5%. Khử nhiễu trước (median hoặc Gauss 5×5) giữ Dice ≥ 0,91 ở σ_n = 50 và ≥ 0,94 với muối-tiêu 5% (median 0,96), dù ảnh sạch chỉ đạt 0,96 - 0,97 vì làm trơn làm lệch nhẹ đường biên. Adaptive MEAN 31/10 kém nhất (0,78 ở ảnh sạch, 0,60 ở σ_n = 50) vì cửa sổ nhỏ bám theo nhiễu.

Ảnh chụp màn hình minh chứng - Bài 3

Chụp trên nền cửa sổ thư mục chứa mã sinh viên: (1) cửa sổ thư mục hiển thị tên thư mục/mã SV và file `bai3.ipynb`; (2) notebook đang chạy với kết quả hiển thị; (3) thời gian/tên tài khoản trên màn hình nếu có.

[Chèn ảnh chụp màn hình tại đây]

Đặt file vào thư mục **screenshots/** với tên `bai3_1.png`, `bai3_2.png`... rồi chạy lại `python tools/make_report.py`.

Bài 4. Phân đoạn ảnh dựa trên vùng

Nội dung	Tham số khảo sát
Region growing	seed A-D; T = 5/10/20/40 (xám), T = 10/20/30/45 (Lab); mode mean/seed; 8-liên thông; Gauss σ=1
K-means	K = 2, 3, 4, 5 (xám & Lab), KMEANS_PP_CENTERS, attempts=5; RGB vs Lab; elbow K = 1..8
SLIC	n_segments 50/200/800 × compactness 1/10/40; sigma 0/1/3
Felzenszwalb	scale 50/200/800 × sigma 0,5/1,5; min_size 50
So sánh	số vùng, số thành phần liên thông, độ đồng nhất Lab, thời gian

Nội dung: (1) tự lập trình region growing, khảo sát vị trí seed và ngưỡng đồng nhất; (2) K-means (K = 2, 3, 4, 5) và SLIC superpixels với nhiều bộ tham số; (3) thêm phương pháp **graph-based segmentation (Felzenszwalb)**, rồi so sánh các phương pháp.

Ảnh đầu vào: `coins.png` (303×384, xám) cho region growing và K-means xám; `coffee.png` (ảnh màu mẫu scikit-image, thu nhỏ còn 200×300 để chạy nhanh) cho region growing trong không gian màu, K-means, SLIC, Felzenszwalb.

Mã nguồn - cài đặt và hàm tiện ích

```
# ===== Cài đặt & hàm tiện ích dùng chung =====
import os, time, warnings
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import data, color, filters, morphology, measure, segmentation, util
warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 80, "font.size": 9, "axes.titlesize": 9})

# Thư mục ảnh đầu vào / kết quả.
# Muốn dùng ảnh trong bộ dữ liệu LMS: chép ảnh vào images/ và đổi tên file ở ô "Ảnh đầu vào".
IMG_DIR, OUT_DIR = "images", "outputs/bai4"
for d in (IMG_DIR, OUT_DIR):
    os.makedirs(d, exist_ok=True)


def load_rgb(name, fallback, max_side=None):
    """Đọc ảnh màu RGB từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, cv2.cvtColor(fallback(), cv2.COLOR_RGB2BGR))
    img = cv2.cvtColor(cv2.imread(p, cv2.IMREAD_COLOR), cv2.COLOR_BGR2RGB)
    if max_side and max(img.shape[:2]) > max_side:
        s = max_side / max(img.shape[:2])
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def load_gray(name, fallback, max_side=None):
    """Đọc ảnh xám uint8 từ images/<name>; nếu chưa có thì lấy ảnh mẫu rồi lưu lại."""
    p = os.path.join(IMG_DIR, name)
    if not os.path.exists(p):
        cv2.imwrite(p, fallback())
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if max_side and max(img.shape) > max_side:
        s = max_side / max(img.shape)
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img


def show(images, titles=None, ncols=4, name=None, cell=3.0, cmap="gray"):
    """Hiển thị lưới ảnh; ảnh 2-D hiển thị theo thang xám cố định; lưu vào OUT_DIR/name."""
    n = len(images)
    ncols = min(ncols, n)
    nrows = int(np.ceil(n / ncols))
    fig, axs = plt.subplots(nrows, ncols, figsize=(cell * ncols, (cell * images[0].shape[0] / images[0].shape[1] + 0.35) *
        nrows), squeeze=False)
    for i, ax in enumerate(axs.ravel()):
        ax.axis("off")
        if i >= n:
            continue
        im = images[i]
        if im.ndim == 2:
            if im.dtype == np.uint8:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=255)
            elif im.dtype == bool:
                ax.imshow(im, cmap=cmap, vmin=0, vmax=1)
            else:
                ax.imshow(im, cmap=cmap)
        else:
            ax.imshow(im)
        if titles is not None:
            ax.set_title(titles[i])
    fig.tight_layout()
    if name:
```

```
        fig.savefig(os.path.join(OUT_DIR, name), dpi=110, bbox_inches="tight")
    plt.show()
```

4.0 Ảnh đầu vào

Mã nguồn

```
coins = load_gray("coins.png", data.coins)
coffee = load_rgb("coffee.png", data.coffee)
coffee_s = cv2.resize(coffee, (300, 200), interpolation=cv2.INTER_AREA)       # 200 x 300
coffee_lab = cv2.cvtColor(coffee_s, cv2.COLOR_RGB2LAB)
print("coins:", coins.shape, "| coffee gốc:", coffee.shape, "| coffee dùng:", coffee_s.shape)
show([coins, coffee_s], ["coins.png", "coffee.png (200x300)"], ncols=2, cell=5, name="b4_input.png")
```

Kết quả (văn bản)

```
coins: (303, 384) | coffee gốc: (400, 600, 3) | coffee dùng: (200, 300, 3)
```

Kết quả (hình)





4.1 Region growing (tự lập trình)

Thuật toán: bắt đầu từ điểm hạt (seed); duyệt BFS các láng giềng (8-liên thông); thêm điểm q vào vùng nếu ‖f(q)-f̄_R‖≤ T với f̄_R là **trung bình hiện thời của vùng** (`mode="mean"`) hoặc giá trị của seed (`mode="seed"`). Chạy được cho ảnh xám (khoảng cách = |Δ|) và ảnh màu (khoảng cách Euclid trong không gian màu, ở đây CIELAB).

Tham số: vị trí seed, ngưỡng đồng nhất T, kiểu so sánh (`mean`/`seed`); ảnh được làm trơn Gaussian σ=1 trước.

Mã nguồn

```
from collections import deque

def region_growing(img, seed, T, mode="mean", conn=8):
    """img: HxW hoặc HxWxC (float); seed=(y, x); trả về mặt nạ bool."""
    f = img.astype(np.float32)
    if f.ndim == 2:
        f = f[..., None]
    H, W, _ = f.shape
    nbrs = [(-1, 0), (1, 0), (0, -1), (0, 1)] + ([(-1, -1), (-1, 1), (1, -1), (1, 1)] if conn == 8 else [])
    mask = np.zeros((H, W), bool)
    mask[seed] = True
    total = f[seed].astype(np.float64).copy()
    n = 1
    seed_val = f[seed].copy()
    q = deque([seed])
    while q:
        y, x = q.popleft()
        for dy, dx in nbrs:
            yy, xx = y + dy, x + dx
            if 0 <= yy < H and 0 <= xx < W and not mask[yy, xx]:
                ref = total / n if mode == "mean" else seed_val
                if np.sqrt(((f[yy, xx] - ref) ** 2).sum()) <= T:
                    mask[yy, xx] = True
                    total += f[yy, xx]
                    n += 1
                    q.append((yy, xx))
    return mask

# chọn seed có ý nghĩa dựa trên ảnh nhị phân: trong đồng xu lớn, trên đường biên đồng xu, ở nền, đồng xu khác
g = cv2.GaussianBlur(coins, (0, 0), 1)
_, bw = cv2.threshold(coins, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
bw = cv2.morphologyEx(bw, cv2.MORPH_OPEN, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)))
lab, nlab = measure.label(bw > 0, return_num=True)
```

```
props = [p for p in measure.regionprops(lab) if 800 < p.area < 5000 and p.solidity > 0.9]
big = max(props, key=lambda p: p.area)              # đồng xu lớn nhất
other = sorted(props, key=lambda p: p.area)[len(props) // 2]   # một đồng xu cỡ trung bình
cy, cx = map(int, big.centroid)
bnd = np.argwhere(segmentation.find_boundaries(lab == big.label, mode="inner"))
by, bx = bnd[len(bnd) // 4]                         # một điểm trên đường biên đồng xu
free = np.argwhere(cv2.erode(255 - bw, np.ones((9, 9), np.uint8)) > 0)
fy, fx = free[len(free) * 3 // 4]                   # một điểm nền
oy, ox = map(int, other.centroid)
seeds = {"A: tâm đồng xu lớn": (cy, cx), "B: trên biên đồng xu": (int(by), int(bx)),
         "C: nền": (int(fy), int(fx)), "D: tâm đồng xu khác": (oy, ox)}
for k, v in seeds.items():
    print(f"seed {k:24s} = {v}, giá trị xám = {int(g[v])}")

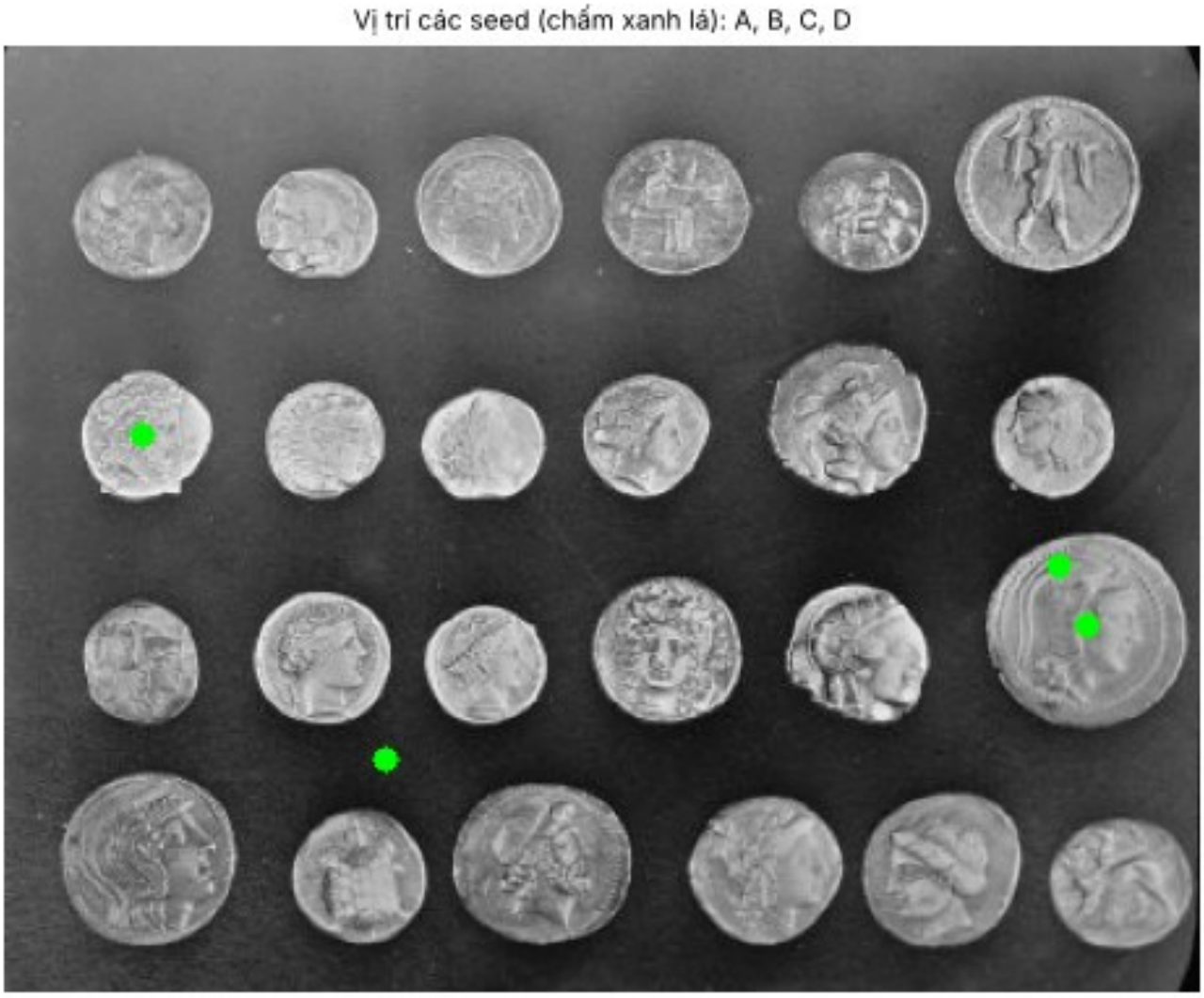
def overlay_mask(gray, mask, seed_pts=(), col=(255, 0, 0)):
    o = cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)
    o[mask] = (0.45 * o[mask] + 0.55 * np.array(col)).astype(np.uint8)
    for (yy, xx) in seed_pts:
        cv2.circle(o, (int(xx), int(yy)), 4, (0, 255, 0), -1)
    return o

show([overlay_mask(coins, np.zeros_like(bw, bool), seeds.values())], ["Vị trí các seed (chấm xanh lá): A, B, C, D"], ncols=1,
    cell=6, name="b4_rg_seeds.png")
```

Kết quả (văn bản)

```
seed A: tâm đồng xu lớn       = (185, 347), giá trị xám = 154
seed B: trên biên đồng xu     = (166, 338), giá trị xám = 129
seed C: nền                   = (228, 122), giá trị xám = 39
seed D: tâm đồng xu khác      = (124, 44), giá trị xám = 188
```

Kết quả (hình)



(a) Khảo sát ngưỡng đồng nhất T (seed A, so với trung bình vùng)

Mã nguồn

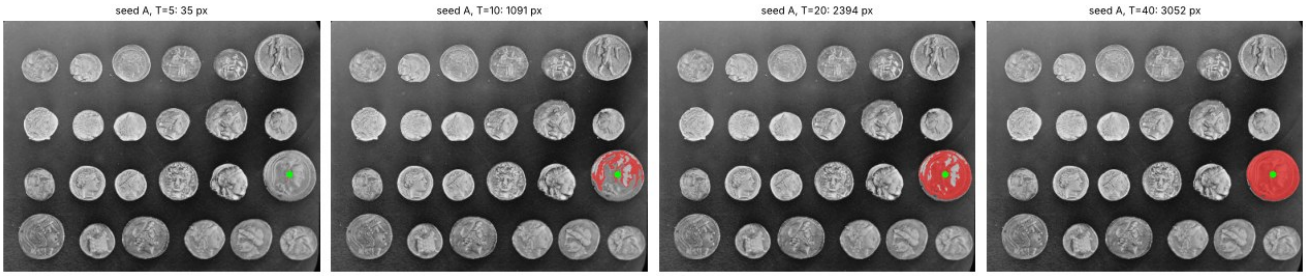
```
imgs, tits = [], []
print(f"{'T':>4s} {'diện tích':>10s} {'thời gian(s)':>13s}")
for T in (5, 10, 20, 40):
    t0 = time.perf_counter()
    m = region_growing(g, seeds["A: tâm đồng xu lớn"], T)
    dt = time.perf_counter() - t0
```

```
    imgs.append(overlay_mask(coins, m, [seeds["A: tâm đồng xu lớn"]]))
    tits.append(f"seed A, T={T}: {int(m.sum())} px")
    print(f"{T:4d} {int(m.sum()):10d} {dt:13.2f}")
show(imgs, tits, ncols=4, cell=4, name="b4_rg_thresholds.png")
```

Kết quả (văn bản)

```
   T  diện tích  thời gian(s)
   5         35          0.00
  10       1091          0.02
  20       2394          0.03
  40       3052          0.03
```

Kết quả (hình)



(b) Khảo sát vị trí seed (T = 15) và kiểu so sánh `mean` / `seed`

Mã nguồn

```
imgs, tits = [], []
print(f"{'seed':26s} {'mode':5s} {'diện tích':>10s}")
for name, sd in seeds.items():
    for mode in ("mean", "seed"):
        m = region_growing(g, sd, 15, mode=mode)
        print(f"{name:26s} {mode:5s} {int(m.sum()):10d}")
        if mode == "mean":
            imgs.append(overlay_mask(coins, m, [sd])); tits.append(f"{name} (mean): {int(m.sum())} px")
        else:
            imgs.append(overlay_mask(coins, m, [sd], (0, 120, 255))); tits.append(f"{name} (seed): {int(m.sum())} px")
show(imgs, tits, ncols=4, cell=4, name="b4_rg_seeds_modes.png")
```

Kết quả (văn bản)

```
seed                       mode   diện tích
A: tâm đồng xu lớn         mean        1886
A: tâm đồng xu lớn         seed        1889
B: trên biên đồng xu       mean        1897
B: trên biên đồng xu       seed          38
```

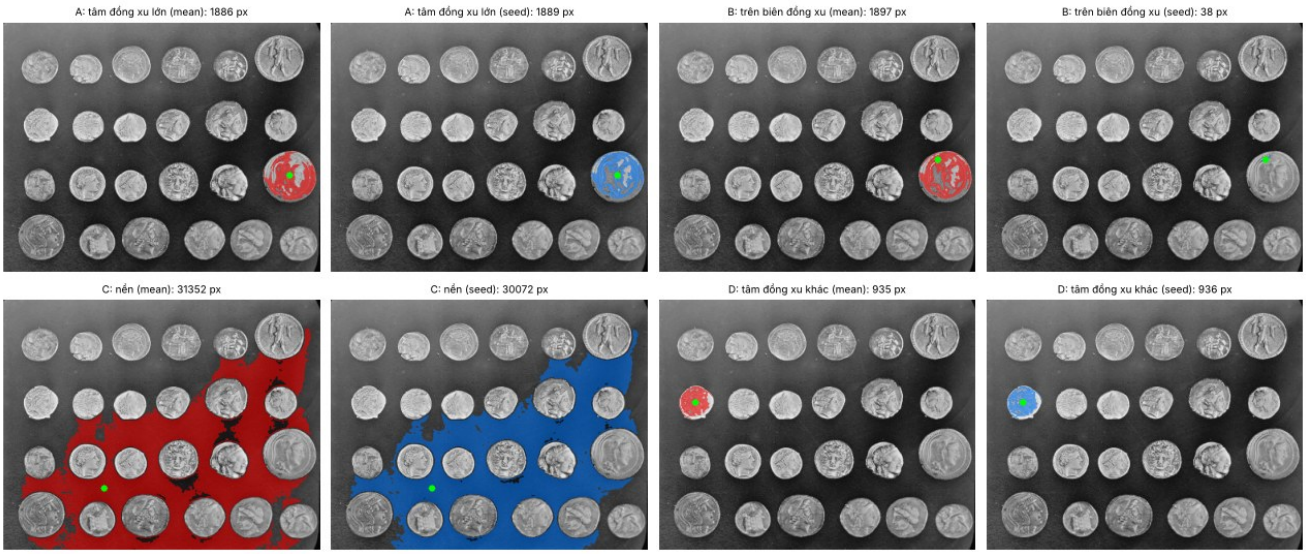
Kết quả (văn bản)

```
C: nền                     mean       31352
```

Kết quả (văn bản)

```
C: nền                     seed       30072
D: tâm đồng xu khác        mean         935
D: tâm đồng xu khác        seed         936
```

Kết quả (hình)



(c) Region growing trên ảnh màu (CIELAB, khoảng cách Euclid), seed trên đĩa sứ đỏ

Mã nguồn

```
lab_s = cv2.GaussianBlur(coffee_lab, (0, 0), 1)
```

```
SEED_C = (100, 62)                      # (y, x) nằm trên đĩa sứ đỏ, phần được chiếu sáng
print("giá trị Lab tại seed (đĩa đỏ):", lab_s[SEED_C])
imgs, tits = [], []
for T in (10, 20, 30, 45):
    t0 = time.perf_counter()
    m = region_growing(lab_s, SEED_C, T)
    o = coffee_s.copy(); o[m] = (0.4 * o[m] + 0.6 * np.array([0, 255, 0])).astype(np.uint8)
    cv2.circle(o, (SEED_C[1], SEED_C[0]), 4, (255, 255, 0), -1)
    imgs.append(o); tits.append(f"Lab, T={T}: {int(m.sum())} px ({time.perf_counter() - t0:.1f}s)")
show(imgs, tits, ncols=4, cell=3.8, name="b4_rg_color.png")
```

Kết quả (văn bản)

```
giá trị Lab tại seed (đĩa đỏ): [ 96 178 173]
```

Kết quả (hình)



Nhận xét 4.1

- **Ngưỡng T (seed A, so với trung bình vùng):** T = 5, 10, 20, 40 cho 35, 1 091, 2 394, 3 052 px. T = 5 dừng gần như ngay vì vân nổi của đồng xu khác trung bình vùng hơn 5 mức xám (thiếu vùng); T = 10 còn nhiều lỗ; T = 20 phủ gần hết mặt đồng xu; T = 40 phủ trọn đồng xu và dừng đúng ở biên vì nền tối (≈ 39) cách đồng xu (≈ 154) rất xa.
- **Vị trí seed (T = 15):** seed A (tâm đồng xu lớn) 1 886 px (mode `mean`) / 1 889 px (`seed`); seed D (tâm đồng xu khác) 935 / 936 px - ở vùng đồng nhất, hai kiểu so sánh gần như giống nhau. Seed B (trên biên đồng xu, mức xám 129 khác xa phần thân ≈ 154) cho kết quả rất khác nhau: `mean` cho 1 897 px (trung bình vùng trôi dần về mức xám thân đồng xu nên lan được hết đồng xu) nhưng `seed` chỉ 38 px vì mọi điểm bị so với 129. Seed C (nền) cho vùng nền liên thông rất lớn, 31 352 px (`mean`) / 30 072 px (`seed`) ≈ 27% ảnh, vì nền chiếu sáng biến thiên chậm. Vậy kết quả **phụ thuộc mạnh vào việc seed có đại diện cho vùng hay không**; `mean` linh hoạt hơn nhưng phụ thuộc thứ tự duyệt/độ trôi, `seed` ổn định nhưng nhạy với seed xấu.
- **Ảnh màu (Lab, seed trên đĩa đỏ):** T = 10, 20, 30, 45 cho 2 803, 4 735, 8 796, 30 680 px. T nhỏ chỉ lấy một phần đĩa (bóng đổ và vùng sáng phản chiếu chia cắt đĩa); T = 30 bắt đầu lấn sang vài mảng gỗ cùng tông; T = 45 **rò rỉ** ra toàn bộ mặt bàn (≈ 51% ảnh). Region growing đơn giản, kết quả liên thông và trọn vẹn, nhưng chỉ có một ngưỡng toàn cục nên nhạy với seed, thứ tự duyệt và dễ rò rỉ qua biên mờ.

4.2 K-means (K = 2, 3, 4, 5)

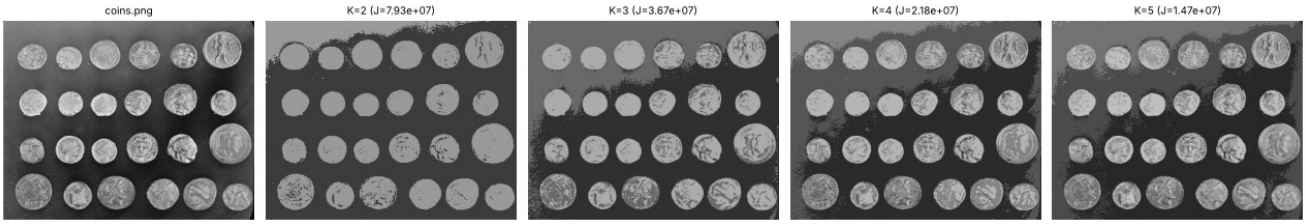
Gom cụm các pixel theo vector đặc trưng (ảnh xám: cường độ; ảnh màu: Lab) bằng `cv2.kmeans` (`KMEANS_PP_CENTERS`, 5 lần khởi tạo, tối đa 100 vòng lặp, ε=0.2, `cv2.setRNGSeed(0)` để lặp lại được). Tổng bình phương khoảng cách (compactness) giảm khi K tăng -> dùng đánh giá khuỷu tay (elbow).

Mã nguồn

```
def kmeans_seg(feat, K, attempts=5):
    cv2.setRNGSeed(0)
    X = feat.reshape(-1, feat.shape[-1] if feat.ndim == 3 else 1).astype(np.float32)
    crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
    comp, labels, centers = cv2.kmeans(X, K, None, crit, attempts, cv2.KMEANS_PP_CENTERS)
    return labels.reshape(feat.shape[:2]), centers, float(comp)

# (a) ảnh xám coins
imgs, tits = [coins], ["coins.png"]
for K in (2, 3, 4, 5):
    lb, ce, comp = kmeans_seg(coins, K)
    imgs.append(ce[lb, 0].astype(np.uint8)); tits.append(f"K={K} (J={comp:.2e})")
    if K == 2:
        k2 = lb
show(imgs, tits, ncols=5, cell=3.2, name="b4_kmeans_gray.png")
# K=2 so với Otsu
hi = np.argmax(kmeans_seg(coins, 2)[1].ravel())
mk = (k2 == hi)
mo = cv2.threshold(coins, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1] > 0
print("Dice(K-means K=2, Otsu) trên coins =", round(2 * (mk & mo).sum() / (mk.sum() + mo.sum()), 4))
```

Kết quả (hình)



Kết quả (văn bản)

```
Dice(K-means K=2, Otsu) trên coins = 1.0
```

Mã nguồn

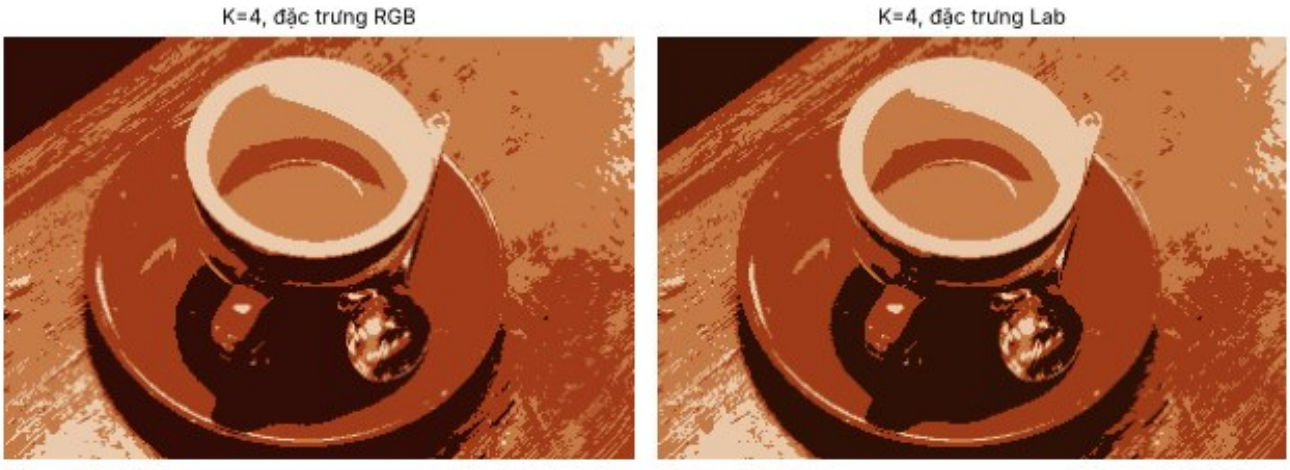
```
# (b) ảnh màu coffee, không gian Lab
imgs, tits, J = [coffee_s], ["coffee.png"], {}
for K in (2, 3, 4, 5):
    lb, ce, comp = kmeans_seg(coffee_lab, K)
    rgb_c = cv2.cvtColor(ce.astype(np.uint8).reshape(1, K, 3), cv2.COLOR_LAB2RGB).reshape(K, 3)
    imgs.append(rgb_c[lb]); tits.append(f"K-means Lab, K={K}")
    J[K] = comp
show(imgs, tits, ncols=5, cell=3.2, name="b4_kmeans_color.png")

# (c) so sánh không gian đặc trưng RGB / Lab với K=4 và đường elbow
imgs, tits = [], []
lb_rgb, ce_rgb, _ = kmeans_seg(coffee_s, 4)
imgs.append(ce_rgb.astype(np.uint8)[lb_rgb]); tits.append("K=4, đặc trưng RGB")
lb_lab, ce_lab, _ = kmeans_seg(coffee_lab, 4)
imgs.append(cv2.cvtColor(ce_lab.astype(np.uint8).reshape(1, 4, 3), cv2.COLOR_LAB2RGB).reshape(4, 3)[lb_lab]); tits.append("K=4,
    đặc trưng Lab")
show(imgs, tits, ncols=2, cell=4, name="b4_kmeans_rgb_lab.png")
Ks = list(range(1, 9))
Js = [kmeans_seg(coffee_lab, K)[2] for K in Ks]
fig, ax = plt.subplots(figsize=(4.5, 2.8))
ax.plot(Ks, Js, "o-"); ax.set_xlabel("K"); ax.set_ylabel("compactness J"); ax.set_title("Elbow - K-means Lab trên coffee")
fig.tight_layout(); fig.savefig(os.path.join(OUT_DIR, "b4_kmeans_elbow.png"), dpi=110, bbox_inches="tight"); plt.show()
print("J theo K:", {K: f"{j:.3e}" for K, j in zip(Ks, Js)})
# số thành phần liên thông của từng nhãn: K-means không dùng thông tin vị trí nên vùng bị phân mảnh
for K in (2, 3, 4, 5):
    lb = kmeans_seg(coffee_lab, K)[0]
    ncc = sum(cv2.connectedComponents((lb == k).astype(np.uint8), connectivity=8)[0] - 1 for k in range(K))
    print(f"K={K}: {ncc} thành phần liên thông cho {K} cụm")
```

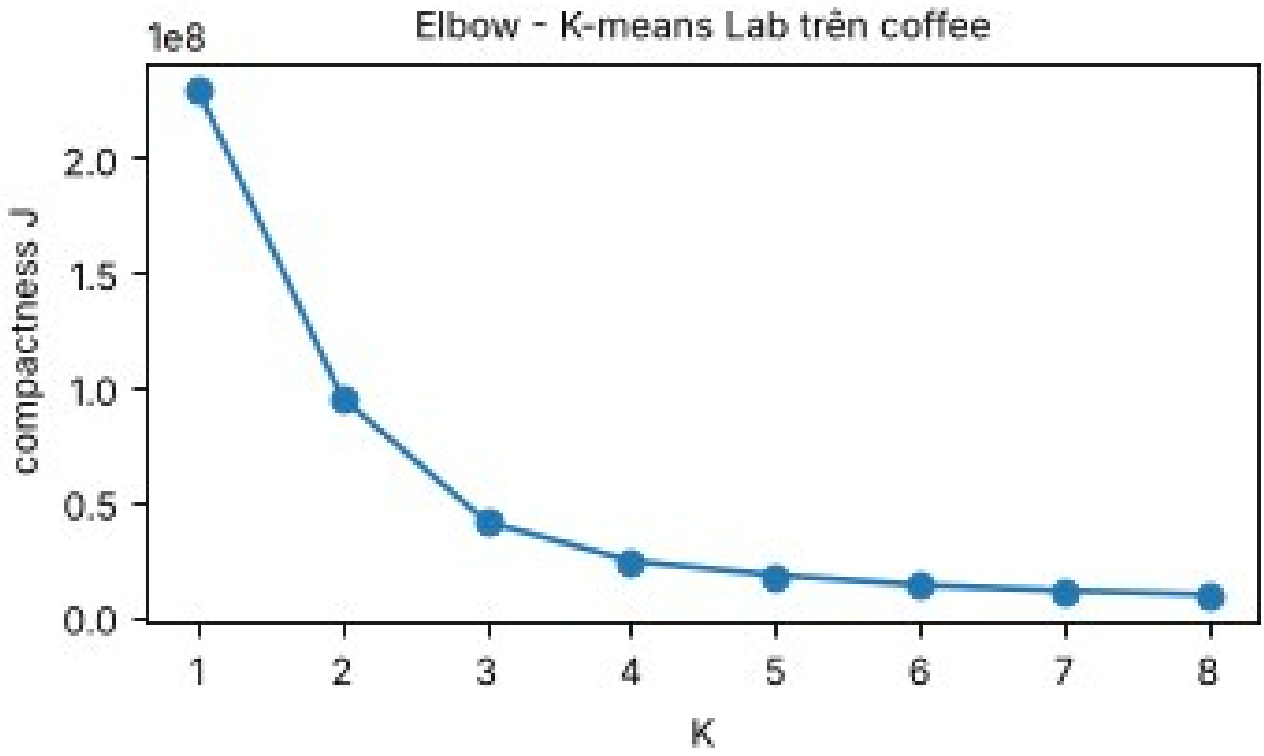
Kết quả (hình)



Kết quả (hình)



Kết quả (hình)



Kết quả (văn bản)

```
J theo K: {1: '2.302e+08', 2: '9.542e+07', 3: '4.127e+07', 4: '2.412e+07', 5: '1.796e+07', 6: '1.361e+07', 7: '1.115e+07', 8:
    '9.481e+06'}
K=2: 125 thành phần liên thông cho 2 cụm
K=3: 358 thành phần liên thông cho 3 cụm
K=4: 747 thành phần liên thông cho 4 cụm
K=5: 1068 thành phần liên thông cho 5 cụm
```

Nhận xét 4.2

- **Ảnh xám (`coins.png`):** K = 2 trùng **hoàn toàn** với Otsu (Dice = 1,0) vì Otsu thực chất là K-means 2 cụm một chiều (cùng cực tiểu phương sai trong lớp). K = 3, 4, 5 chia thêm các mức xám của vân đồng xu và độ chuyển sáng của nền.
- **Ảnh màu (Lab, `coffee.png`):** K = 2 chỉ tách sáng/tối (bóng đổ riêng một cụm); K = 3 tách thêm vành/lòng cốc sáng; K = 4 tách được bóng tối, đĩa đỏ, gỗ (gộp cả cà phê vì cùng tông), vành cốc trắng; K = 5 tách thêm vùng cà phê trong cốc. Compactness J giảm mạnh đến K = 4 ($2{,}30\cdot10^{8}$ -> $9{,}54\cdot10^{7}$ -> $4{,}13\cdot10^{7}$ -> $2{,}41\cdot10^{7}$) rồi chậm lại ($1{,}80\cdot10^{7}$ ở K = 5, $1{,}36\cdot10^{7}$ ở K = 6) - khuỷu quanh K = 4 - 5. Đặc trưng RGB và Lab cho K = 4 gần như giống nhau trên ảnh này.
- **Hạn chế:** K-means không dùng vị trí nên một cụm bị chia thành rất nhiều mảnh rời: K = 2, 3, 4, 5 cho 125, 358, 747, 1 068 thành phần liên thông; cùng màu thì gộp dù là hai vật khác nhau (bóng đổ trên bàn và bóng dưới đĩa, thìa kim loại cùng cụm tối). Phải chọn K trước.

4.3 SLIC superpixels

`skimage.segmentation.slic`: K-means cục bộ trong không gian 5-D (Lab + vị trí). **Tham số:** `n_segments` ∈ {50, 200, 800}, `compactness` ∈ {1, 10, 40} (lớn -> superpixel vuông vức, đều; nhỏ -> bám biên màu), `sigma` làm trơn.

Mã nguồn

```
imgs, tits = [], []
slic_info = {}
print(f"{'n_segments':>11s} {'compactness':>12s} {'số SP thực':>11s} {'thời gian(s)':>13s}")
for ns in (50, 200, 800):
    for cp in (1, 10, 40):
        t0 = time.perf_counter()
        sp = segmentation.slic(coffee_s, n_segments=ns, compactness=cp, sigma=1, start_label=0)
        dt = time.perf_counter() - t0
        slic_info[(ns, cp)] = sp
        print(f"{ns:11d} {cp:12d} {sp.max() + 1:11d} {dt:13.3f}")
        imgs.append(segmentation.mark_boundaries(coffee_s, sp, color=(1, 1, 0)))
        tits.append(f"n={ns}, c={cp}: {sp.max() + 1} SP")
show(imgs, tits, ncols=3, cell=3.6, name="b4_slic_grid.png")
```

Kết quả (văn bản)

```
 n_segments  compactness  số SP thực  thời gian(s)
```

Kết quả (văn bản)

```
         50            1          27         0.088
         50           10          44         0.056
```

Kết quả (văn bản)

```
         50           40          54         0.056
        200            1         112         0.070
```

Kết quả (văn bản)

```
        200           10         178         0.070
```

Kết quả (văn bản)

```
        200           40         216         0.068
        800            1         432         0.079
```

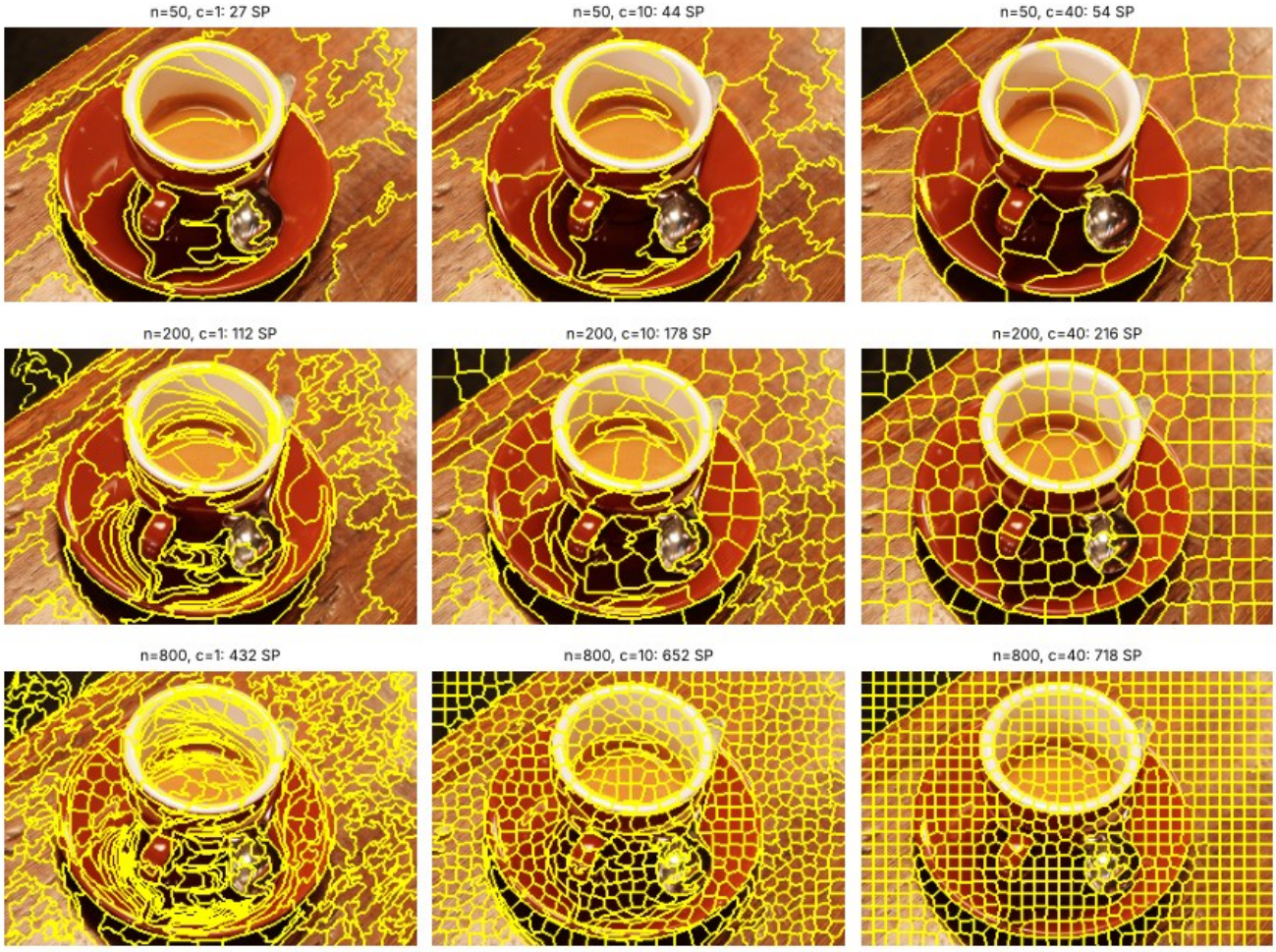
Kết quả (văn bản)

```
        800           10         652         0.073
```

Kết quả (văn bản)

```
        800           40         718         0.072
```

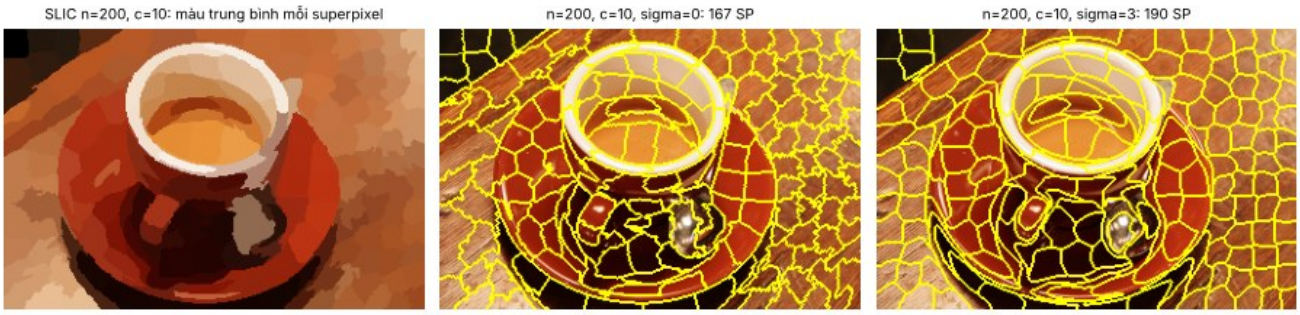
Kết quả (hình)



Mã nguồn

```
# hiển thị superpixel dưới dạng màu trung bình + ảnh hưởng của sigma (n=200, c=10)
sp = slic_info[(200, 10)]
avg = color.label2rgb(sp, coffee_s, kind="avg")
imgs, tits = [avg], ["SLIC n=200, c=10: màu trung bình mỗi superpixel"]
for sg in (0, 3):
    s2 = segmentation.slic(coffee_s, n_segments=200, compactness=10, sigma=sg, start_label=0)
    imgs.append(segmentation.mark_boundaries(coffee_s, s2, color=(1, 1, 0))); tits.append(f"n=200, c=10, sigma={sg}: {s2.max() +
        1} SP")
show(imgs, tits, ncols=3, cell=3.8, name="b4_slic_avg_sigma.png")
```

Kết quả (hình)



Nhận xét 4.3

- `n_segments` = 50, 200, 800 với `compactness` = 1 / 10 / 40 cho số superpixel thực 27 / 44 / 54, 112 / 178 / 216 và 432 / 652 / 718: compactness nhỏ làm số vùng thực thấp hơn nhiều so với yêu cầu (các mảnh nhỏ bị gộp để đảm bảo liên thông), compactness lớn thì gần đúng yêu cầu. Thời gian chỉ ≈ 0,06 - 0,09 s, gần như không đổi.
- `compactness` = 1: superpixel bám biên màu tốt (đường viền cốc, bóng đổ) nhưng hình dạng méo, kích thước không đều; = 40: gần như lưới đều, ít bám biên, cắt ngang vành cốc; = 10 là cân bằng tốt (vừa đều, vừa theo biên). `n_segments` tăng thì biên chính xác hơn nhưng vùng nhỏ hơn.
- `sigma` = 0, 1, 3 (n = 200, c = 10) cho 167, 178, 190 superpixel: làm trơn mạnh hơn loại bỏ kết cấu hạt của vân gỗ nên vùng đều đặn hơn và số vùng gần giá trị yêu cầu. Ảnh "màu trung bình" cho thấy superpixel giữ được hình cốc/đĩa với chỉ ~180 vùng -> dùng làm tầng tiền xử lý rất tốt.

4.4 Phương pháp bổ sung: graph-based segmentation (Felzenszwalb-Huttenlocher)

Ảnh là đồ thị lưới, cạnh có trọng số = độ khác biệt màu; gộp hai thành phần nếu cạnh nhỏ nhất nối chúng nhẹ hơn ngưỡng nội tại (phụ thuộc `scale`/kích thước thành phần). **Tham số:** `scale` ∈ {50, 200, 800}, `sigma` ∈ {0.5, 1.5}, `min_size` = 50.

Mã nguồn

```
imgs, tits = [], []
felz = {}
print(f"{'scale':>6s} {'sigma':>6s} {'số vùng':>8s} {'thời gian(s)':>13s}")
for sg in (0.5, 1.5):
    for sc in (50, 200, 800):
        t0 = time.perf_counter()
        fl = segmentation.felzenszwalb(coffee_s, scale=sc, sigma=sg, min_size=50)
        dt = time.perf_counter() - t0
        felz[(sc, sg)] = fl
        print(f"{sc:6d} {sg:6.1f} {fl.max() + 1:8d} {dt:13.3f}")
        imgs.append(segmentation.mark_boundaries(coffee_s, fl, color=(1, 1, 0)))
        tits.append(f"scale={sc}, sigma={sg}: {fl.max() + 1} vùng")
show(imgs, tits, ncols=3, cell=3.6, name="b4_felz_grid.png")
```

Kết quả (văn bản)

```
 scale  sigma  số vùng  thời gian(s)
    50    0.5      167         0.066
   200    0.5       74         0.066
```

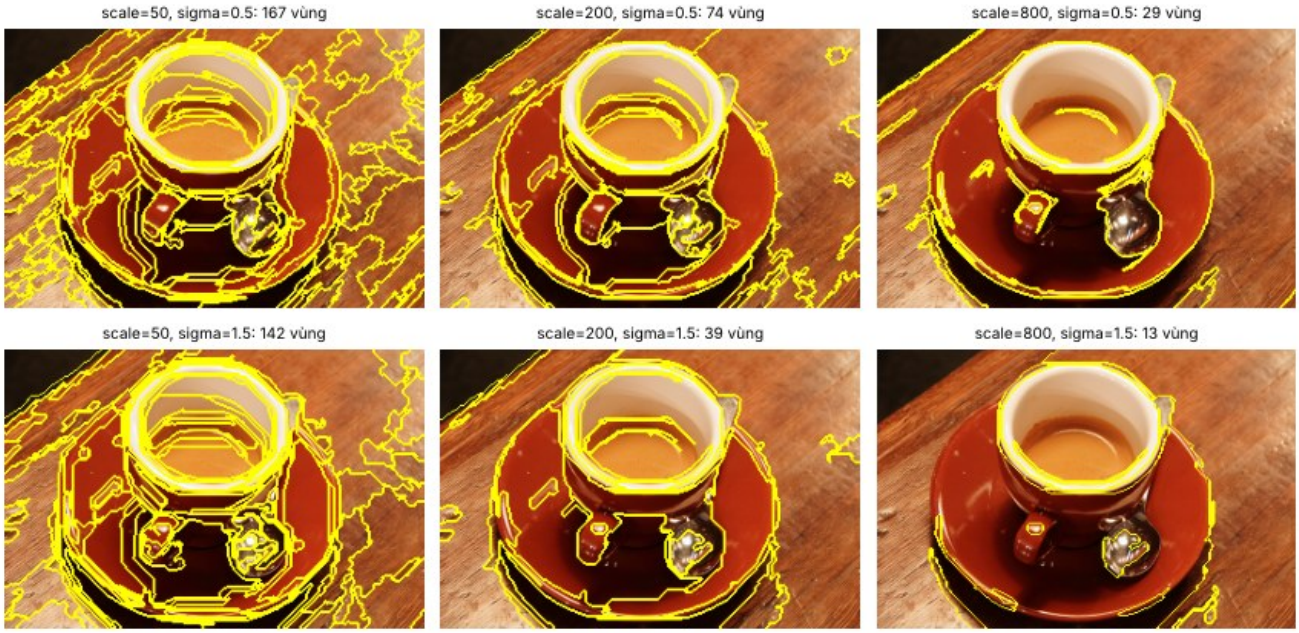
Kết quả (văn bản)

```
   800    0.5       29         0.071
    50    1.5      142         0.085
   200    1.5       39         0.074
```

Kết quả (văn bản)

```
   800    1.5       13         0.078
```

Kết quả (hình)



4.5 So sánh các phương pháp trên cùng ảnh `coffee.png`

Không có ground-truth nên dùng các số đo nội tại: **số vùng**, **số thành phần liên thông** (vùng rời rạc), **độ đồng nhất** = độ lệch Euclid trung bình trong Lab của pixel so với màu trung bình vùng của nó (thấp = đồng nhất hơn), và thời gian chạy. Lưu ý độ đồng nhất tất nhiên giảm khi số vùng tăng -> chỉ so sánh công bằng giữa các phương pháp có số vùng gần nhau.

Mã nguồn

```
def homogeneity(labels, lab_img):
    lab_f = lab_img.astype(np.float32)
    err = np.zeros(labels.shape, np.float32)
    for k in np.unique(labels):
        m = labels == k
        err[m] = np.sqrt(((lab_f[m] - lab_f[m].mean(0)) ** 2).sum(1))
    return float(err.mean())

def n_regions_connected(labels):
    return sum(cv2.connectedComponents((labels == k).astype(np.uint8), connectivity=8)[0] - 1 for k in np.unique(labels))

cands = {}
t0 = time.perf_counter(); lb4 = kmeans_seg(coffee_lab, 4)[0]; cands["K-means K=4 (Lab)"] = (lb4, time.perf_counter() - t0)
t0 = time.perf_counter(); lb8 = kmeans_seg(coffee_lab, 8)[0]; cands["K-means K=8 (Lab)"] = (lb8, time.perf_counter() - t0)
t0 = time.perf_counter(); s1 = segmentation.slic(coffee_s, n_segments=50, compactness=10, sigma=1, start_label=0); cands["SLIC
    n=50, c=10"] = (s1, time.perf_counter() - t0)
t0 = time.perf_counter(); s2 = segmentation.slic(coffee_s, n_segments=200, compactness=10, sigma=1, start_label=0); cands["SLIC
    n=200, c=10"] = (s2, time.perf_counter() - t0)
t0 = time.perf_counter(); f1 = segmentation.felzenszwalb(coffee_s, scale=800, sigma=0.5, min_size=50); cands["Felzenszwalb
    scale=800"] = (f1, time.perf_counter() - t0)
t0 = time.perf_counter(); f2 = segmentation.felzenszwalb(coffee_s, scale=200, sigma=0.5, min_size=50); cands["Felzenszwalb
    scale=200"] = (f2, time.perf_counter() - t0)

print(f"{'phương pháp':32s} {'số nhãn':>8s} {'số TP liên thông':>17s} {'độ đồng nhất ↓':>15s} {'thời gian(s)':>13s}")
imgs, tits = [coffee_s], ["Ảnh gốc"]
for name, (lb, dt) in cands.items():
    print(f"{name:32s} {len(np.unique(lb)):8d} {n_regions_connected(lb):17d} {homogeneity(lb, coffee_lab):15.2f} {dt:13.3f}")
    if "K-means" in name:
        cl = np.array([coffee_s[lb == k].mean(0) for k in range(lb.max() + 1)]).astype(np.uint8)
        imgs.append(cl[lb])
    else:
        imgs.append(segmentation.mark_boundaries(coffee_s, lb, color=(1, 1, 0)))
    tits.append(name)
show(imgs, tits, ncols=3, cell=4.2, name="b4_compare.png")
```

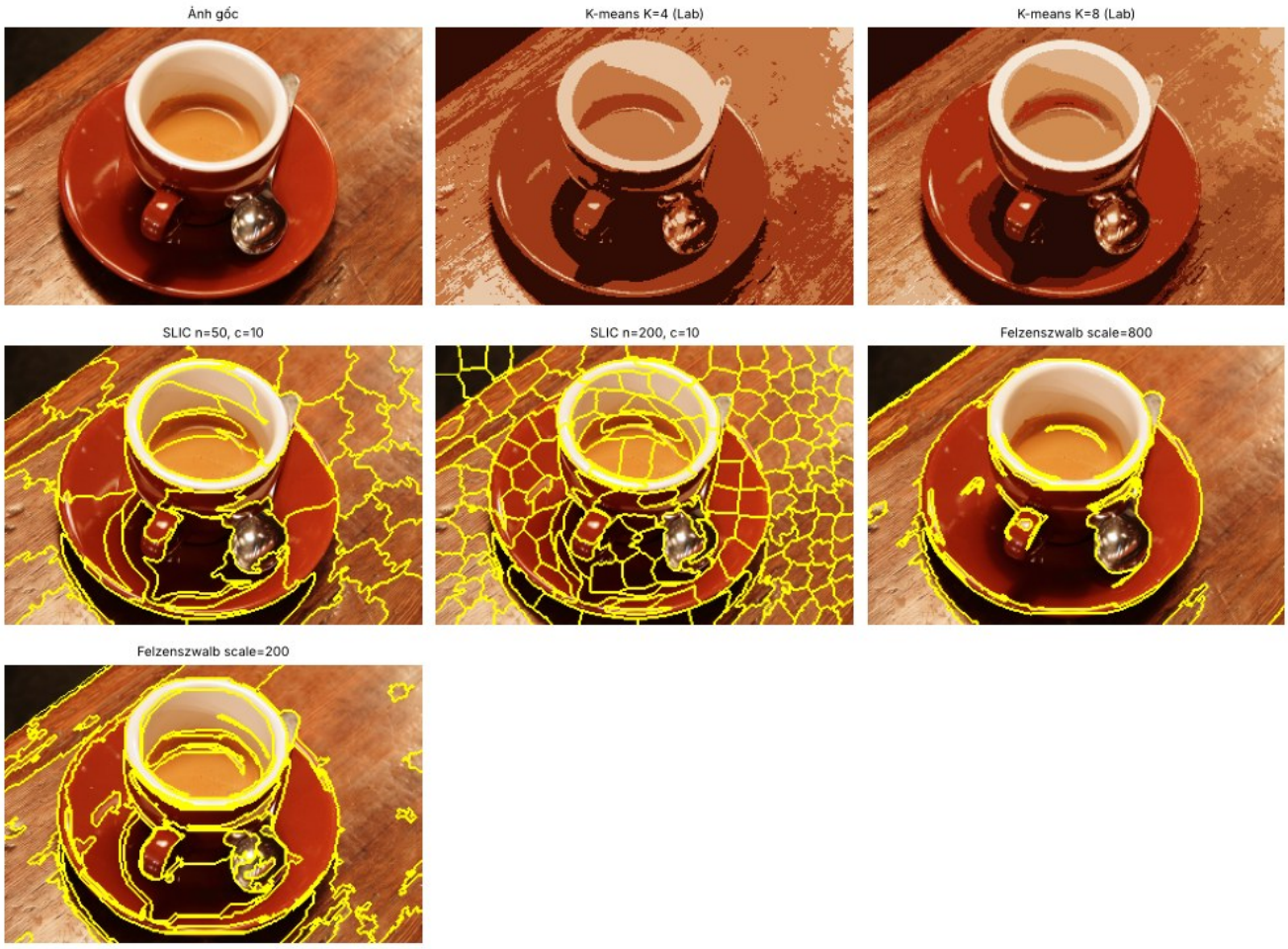
Kết quả (văn bản)

```
phương pháp                      số nhãn  số TP liên thông  độ đồng nhất ↓  thời gian(s)
K-means K=4 (Lab)                      4               747           17.94         0.048
K-means K=8 (Lab)                      8              1860           11.07         0.113
SLIC n=50, c=10                       44                44           16.15         0.060
SLIC n=200, c=10                     178               178           13.47         0.071
Felzenszwalb scale=800                29                29           36.39         0.067
```

Kết quả (văn bản)

```
Felzenszwalb scale=200                74                74           17.83         0.067
```

Kết quả (hình)



Nhận xét 4.4 - 4.5

- **Felzenszwalb (graph-based):** `scale` điều khiển cỡ vùng: scale = 50, 200, 800 (σ = 0,5) cho 167, 74, 29 vùng; σ = 1,5 cho 142, 39, 13 vùng. Biên của vùng bám theo vật thể (đường viền cốc, đĩa, thìa) và kích thước vùng **không đều**: vùng phẳng (bàn gỗ, mặt đĩa) lớn, vùng nhiều chi tiết (thìa) nhỏ. scale = 800 gần với phân đoạn theo vật thể nhất (cốc - đĩa - bàn) dù độ đồng nhất số học kém nhất.
- **So sánh (độ đồng nhất ↓ tốt hơn, đo bằng khoảng cách Lab TB tới màu trung bình vùng):** K-means K = 4: 17,94 (4 nhãn nhưng 747 mảnh rời); K = 8: 11,07 (1 860 mảnh); SLIC 44 vùng: 16,15; SLIC 178 vùng: 13,47; Felzenszwalb 74 vùng: 17,83; Felzenszwalb 29 vùng: 36,39. Ở số vùng tương đương (khoảng 40 - 75), SLIC (16,15) đồng nhất hơn Felzenszwalb (17,83) vì ép các vùng kích thước đều; K-means đồng nhất rất tốt ngay với K nhỏ nhưng phải trả giá bằng hàng trăm mảnh không liên thông. Cả ba chạy rất nhanh (≈ 0,05 - 0,14 s trên ảnh 200×300). Số đo này là nội tại, chỉ đánh giá màu đồng đều chứ không phải độ đúng theo ngữ nghĩa.
- **Chọn phương pháp:** K-means - phân lớp theo màu toàn cục, không cần vị trí, nhưng vùng phân mảnh; SLIC - tiền xử lý/superpixel đều, nhanh, ổn định nhưng không theo vật thể; Felzenszwalb - vùng thích nghi theo nội dung, gần vật thể hơn nhưng khó kiểm soát số vùng; Region growing - chỉ lấy một vật quanh seed, kết quả liên thông nhưng cần chọn seed và T cẩn thận (đã thấy rò rỉ ở T = 45). Region growing không đưa vào bảng so sánh vì chỉ phân đoạn một đối tượng.

Ảnh chụp màn hình minh chứng - Bài 4

Chụp trên nền cửa sổ thư mục chứa mã sinh viên: (1) cửa sổ thư mục hiển thị tên thư mục/mã SV và file `bai4.ipynb`; (2) notebook đang chạy với kết quả hiển thị; (3) thời gian/tên tài khoản trên màn hình nếu có.

[Chèn ảnh chụp màn hình tại đây]

Đặt file vào thư mục **screenshots/** với tên `bai4_1.png`, `bai4_2.png`... rồi chạy lại `python tools/make_report.py`.